

Cloud und Edge Computing

Prof. Dr. Eckhart Körner

Email: e.koerner@th-mannheim.de





1. Einführung

- Cloud, Edge und Fog Computing
- Verteilte Systemarchitekturen
- Anwendungsentwicklung: .NET und C#

2 . Cloud Computing

- Grundlagen des Cloud Computing
- Infrastructure as a Service (IaaS)
- Container as a Service (CaaS)
- Platform as a Service (PaaS)

3. Middleware Layer

- Hypertext Transfer Protocol (HTTP)
- Domain Name System (DNS)
- Distributed Hash Table (DHT)



M. Scott Kingsley, “Cloud Technologies and Services“, Springer, 1. Auflage, 2024. ISBN 9783031336683

Open Access: <https://link.springer.com/book/10.1007/978-3-031-33669-0>

Huawei Technologies Co., Ltd, “Cloud Computing Technology“, Springer, 1. Auflage, 2023. ISBN 9789811930256

Open Access: <https://link.springer.com/book/10.1007/978-981-19-3026-3>

David Jensen, “Beginning Azure IoT Edge Computing: Extending the Cloud to the Intelligent Edge“, Apress, 1. Auflage, 2019. ISBN 1484245350

Naresh Kumar Sehgal , Manoj Saxena , Dhaval N. Shah, “AI on the Edge with Security“, Springer, 2025, ISBN 9783031782725, 2025.

<https://link.springer.com/book/10.1007/978-3-031-78272-5>



Labor

Ort:

Institut für Softwaretechnik und Datenkommunikation, Bau S,
Raum 113 oder 117 (PC-Pool)

Themen:

- Cloud Computing with Microsoft Azure
 - Azure IaaS
 - Azure PaaS
 - Azure Machine Learning (ML)
 - Azure Internet of Things (IoT)

Übungen

Aufgaben zu Kapitel 2 und 3

Prüfung

- Miniprojekte zum Cloud Computing zur Bearbeitung in Teams
- Klausur



Alle Arbeitsmaterialien sind verfügbar auf

<https://moodle.th-mannheim.de>



1. Einführung

2. Cloud Computing

3. Middleware Layer

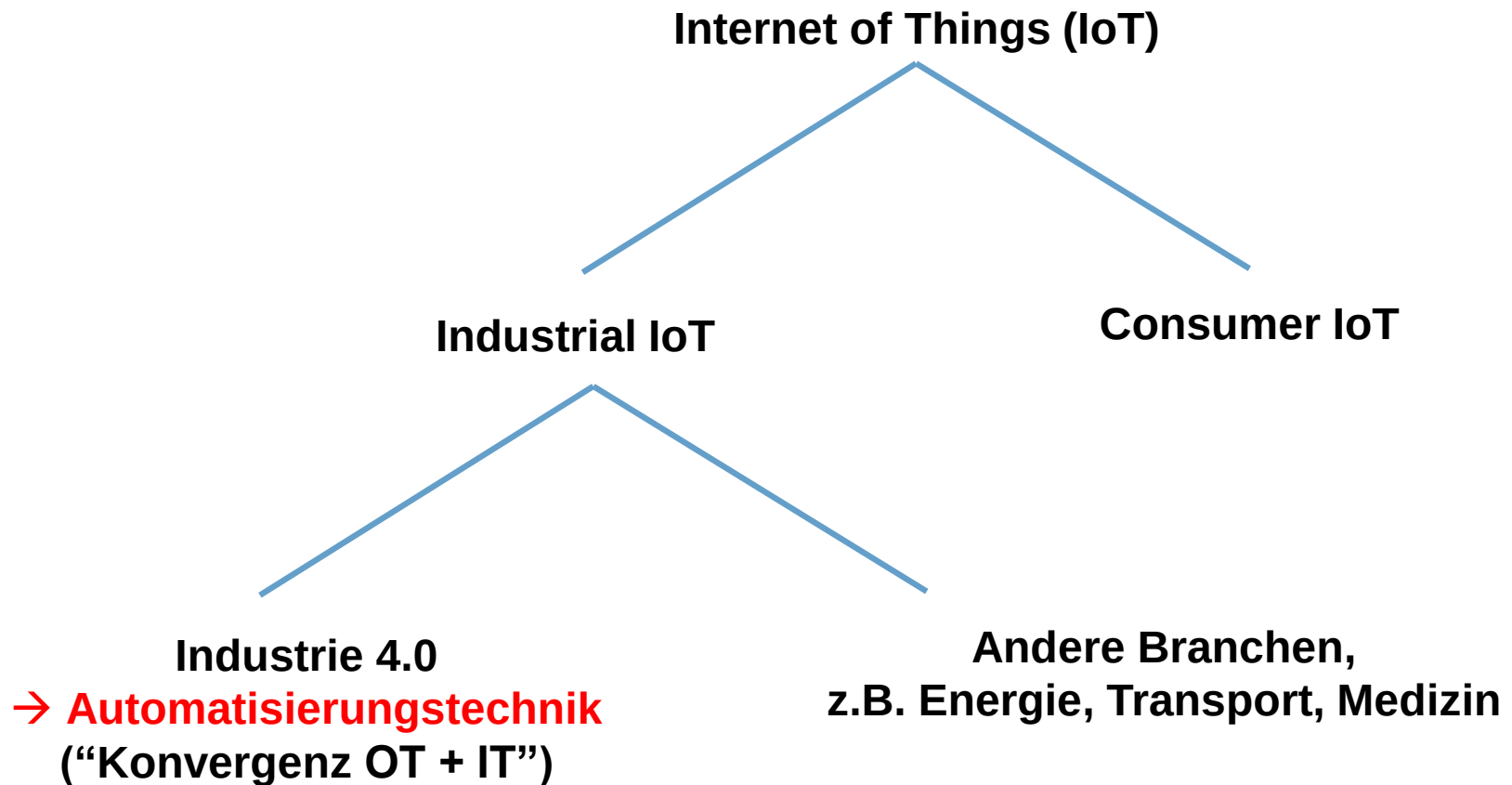
1. Einführung



1.1 Cloud, Edge und Fog Computing

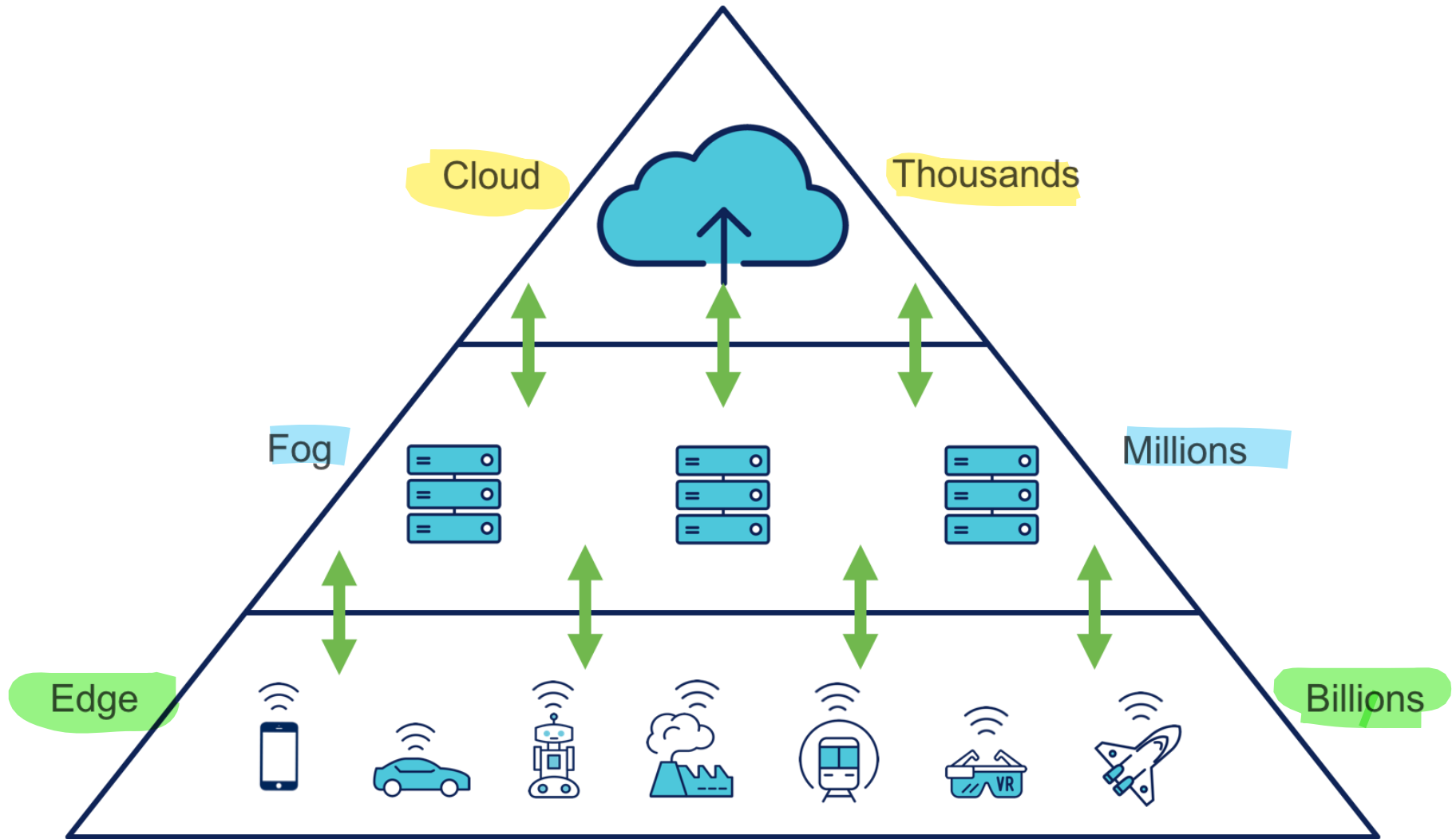
1.2 Verteilte Systemarchitekturen

1.3 Anwendungsentwicklung: .NET und C#



OT= Operational Technology
IT = Information Technology

OT bewegt die wirkliche Welt (Maschinen, Ventile, Motoren).
IT bewegt Daten





- **Cloud computing** is the on-demand availability of computer system resources, especially data storage and computing power, without direct active management by the user. The term is generally used to describe data centers available to many users over the Internet.
[Quelle: Amazon AWS]
- **Edge computing** broadly is all computing outside the cloud happening at the edge of the network, and more specifically in applications where real-time processing of data is required.
[Quelle: IEEE DAC 2014 Keynote]
- **Fog computing**, also known as fogging, is an architecture that uses edge devices to carry out a substantial amount of computation, storage, and communication locally and routed over the internet backbone.
[Quelle: Bonomi, Flavio, ISCA2011]

1. Einführung

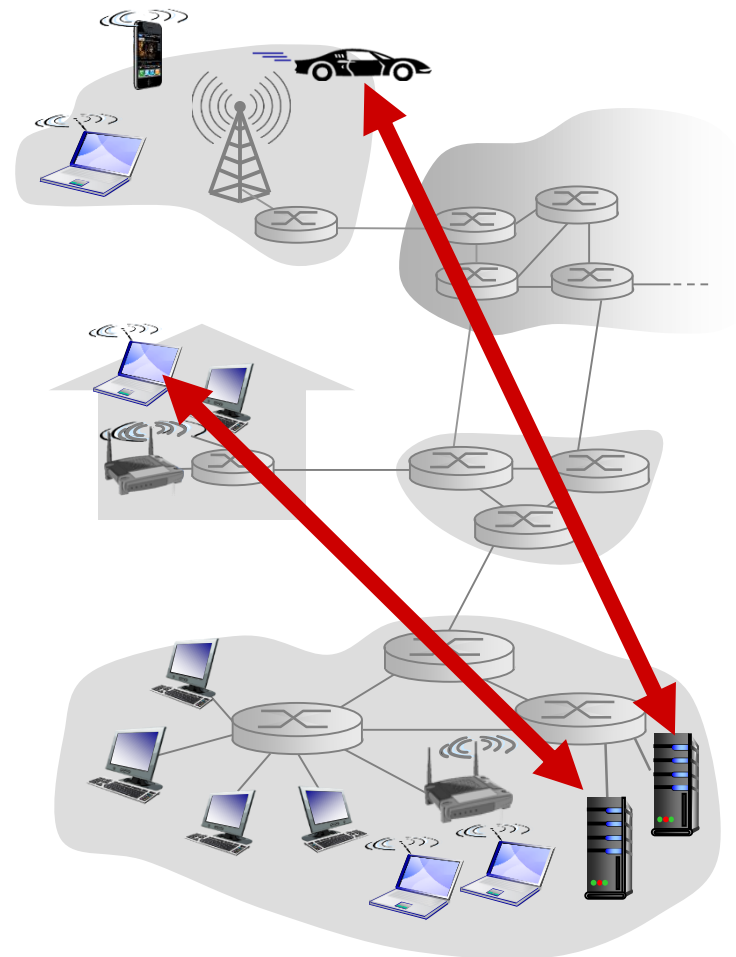


1.1 Cloud, Edge und Fog Computing

1.2 Verteilte Systemarchitekturen

1.3 Anwendungsentwicklung:
.NET und C#

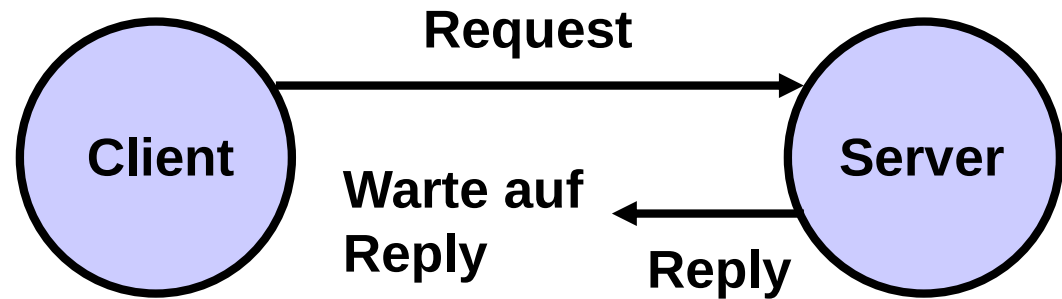
- **Client**
initiiierender Prozess
(stellt Anfragen, erteilt Aufträge)
- **Server**
reagierender Prozess
(bearbeitet Anfragen und Aufträge; i. a. mit besonderen Ressourcen ausgestattet)



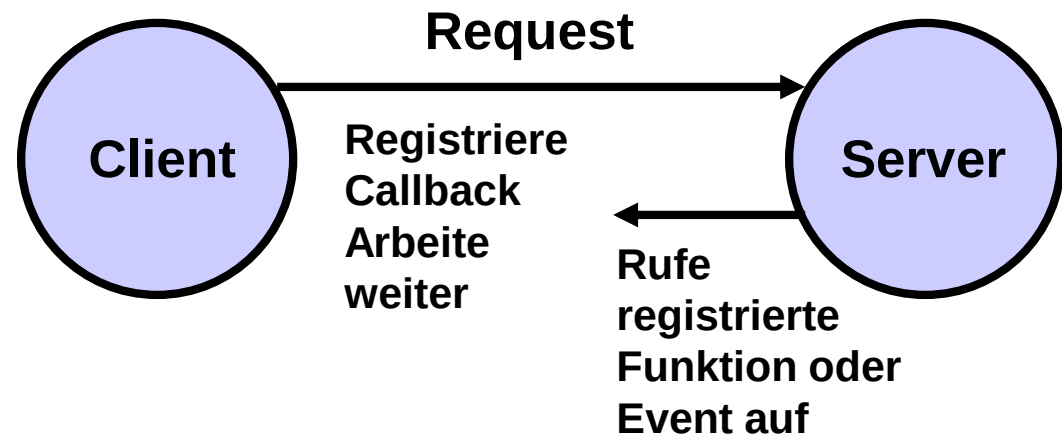


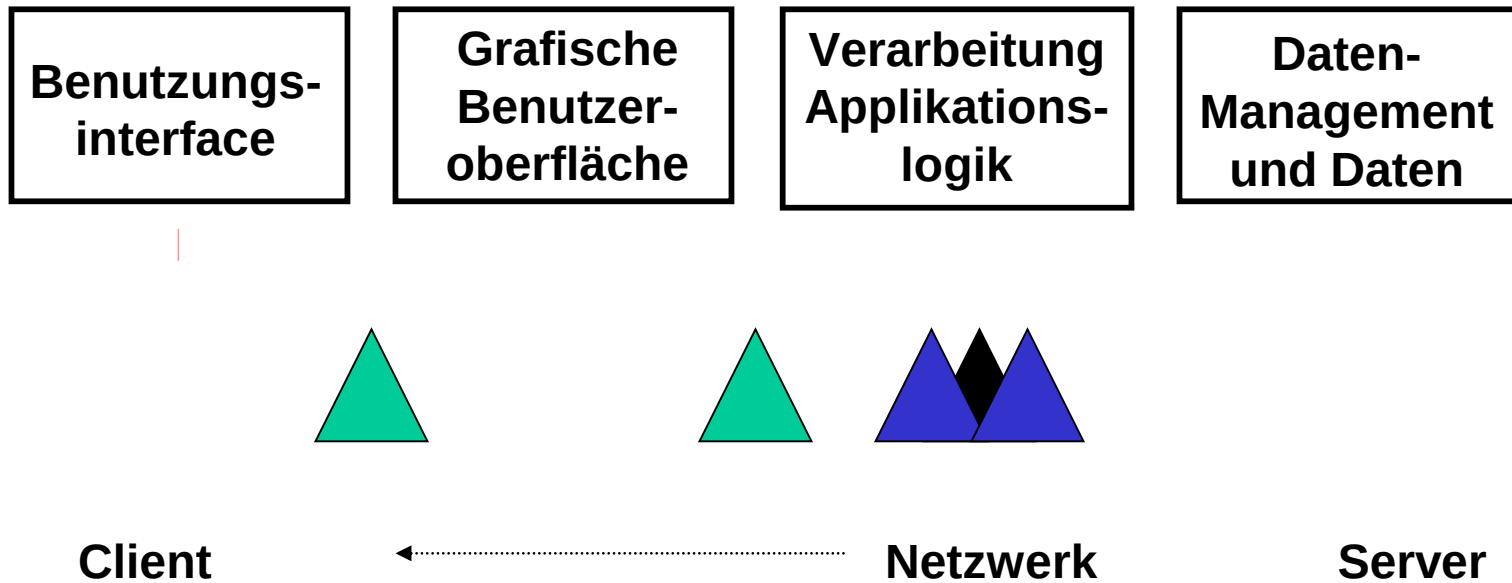
- **Populär wegen des eingängigen Modells**
 - entspricht Geschäftsvorgängen in unserer Dienstleistungsgesellschaft
 - gewohntes Muster --> intuitive Struktur, gute Überschaubarkeit
- **Effizienz durch spezialisierte „Dienstleister“**
 - verfügen ggf. über Spezialhardware
 - großzügige Ausstattung (CPU- Leistung, Speicherkapazität usw.)
 - bestückt mit Spezialsoftware (Datenbank etc.)
- **Kosteneffektivität durch bessere Auslastung wertvoller Ressourcen**
 - einzelner Client kann Ressourcen nicht dauerhaft auslasten
- **Passend für viele Kooperationsbeziehungen, z. B.**
 - schwacher Client erbittet Hilfe vom „großen Bruder“
 - gefährdete Clients geben wertvolle Daten in Obhut des (gegen Mißbrauch, Brand, Diebstahl usw.) hoch gesicherten Servers

**Synchrone
Kommunikation**

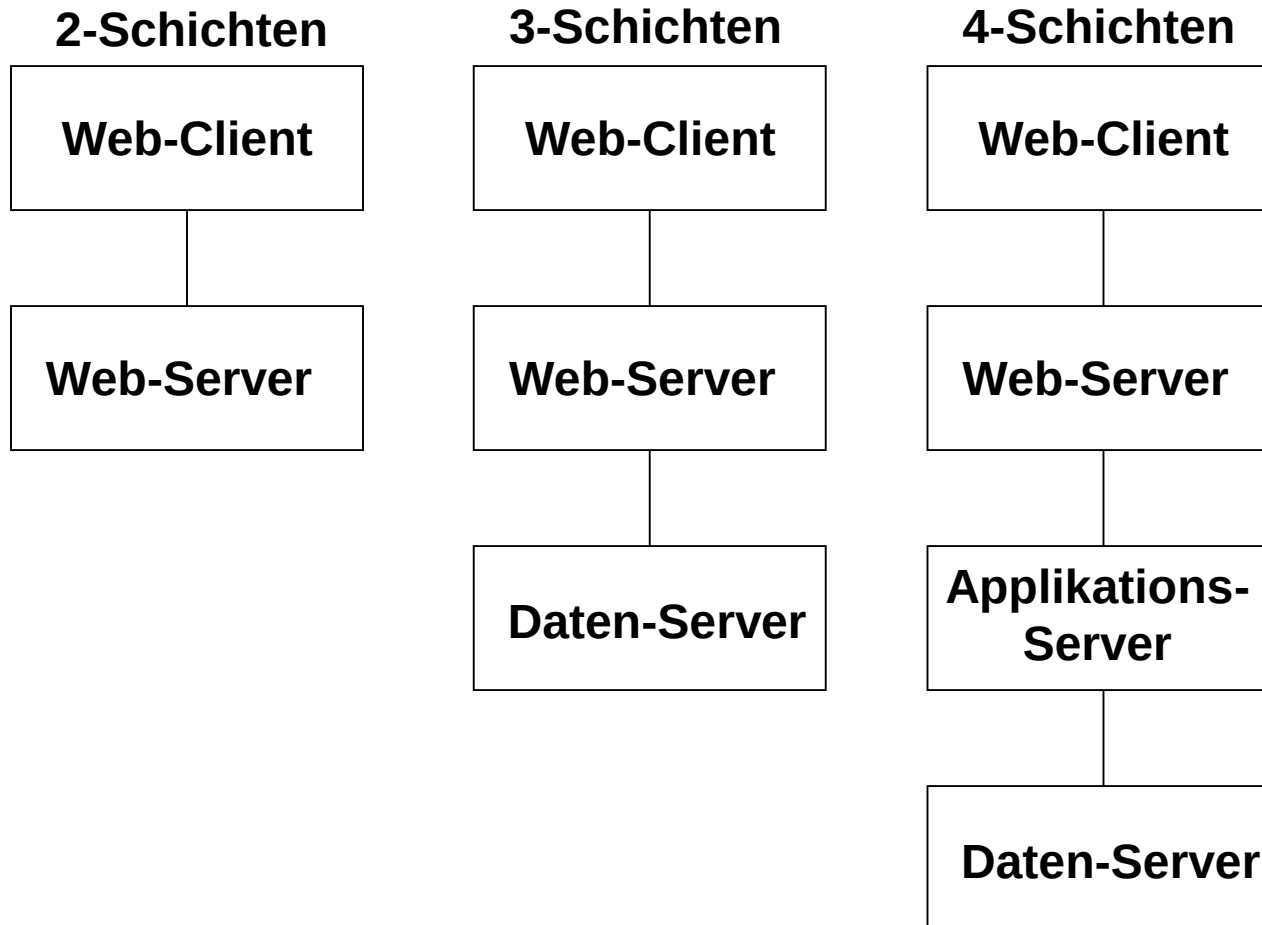


**Asynchrone
Kommunikation**

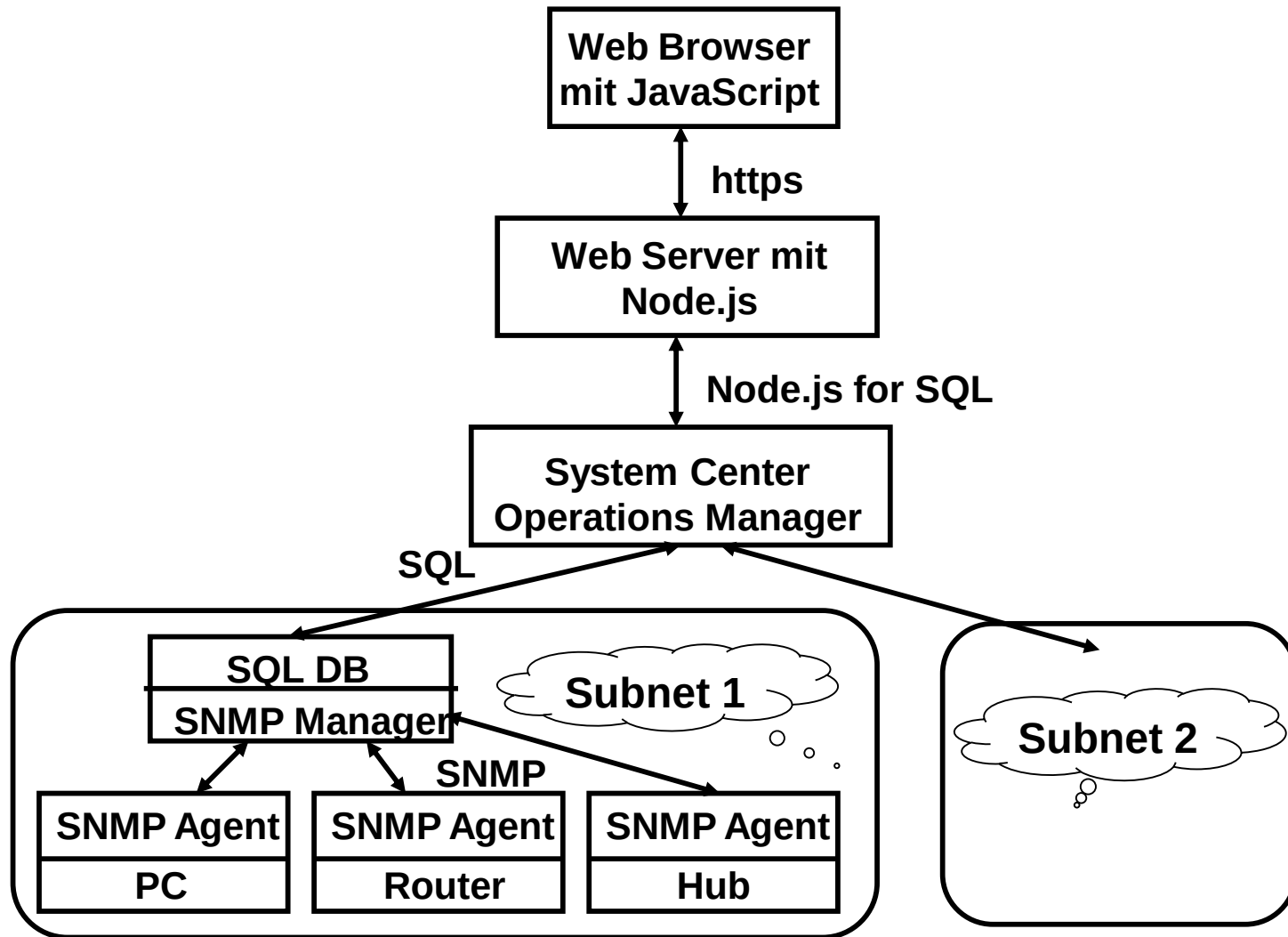




Unterscheidung “Thin Client” und “Fat Client”

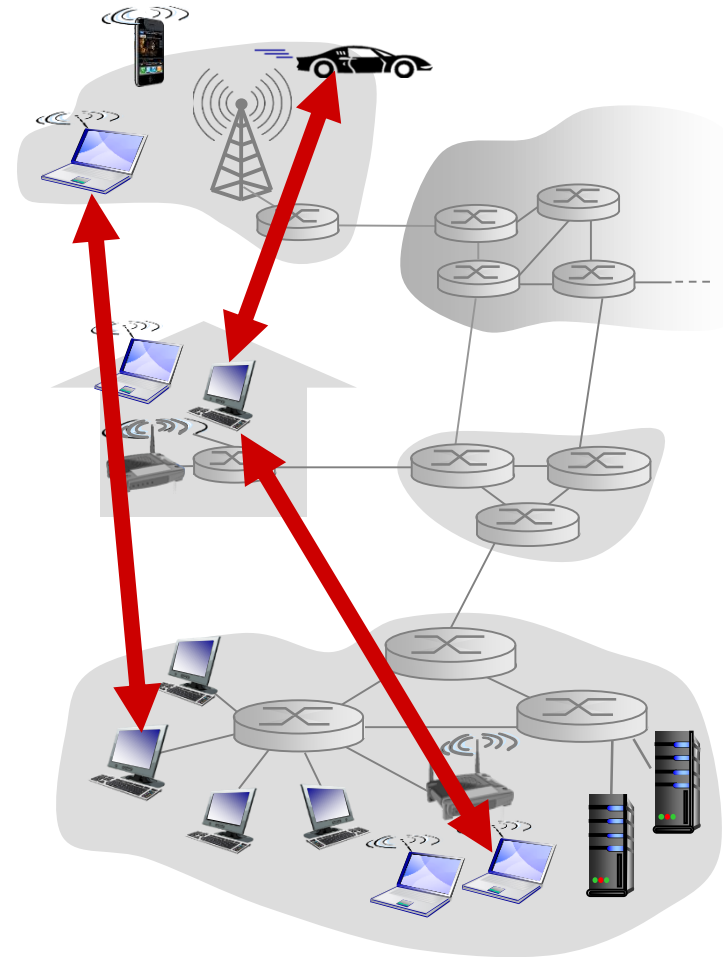


Beispiel: Asset Management System

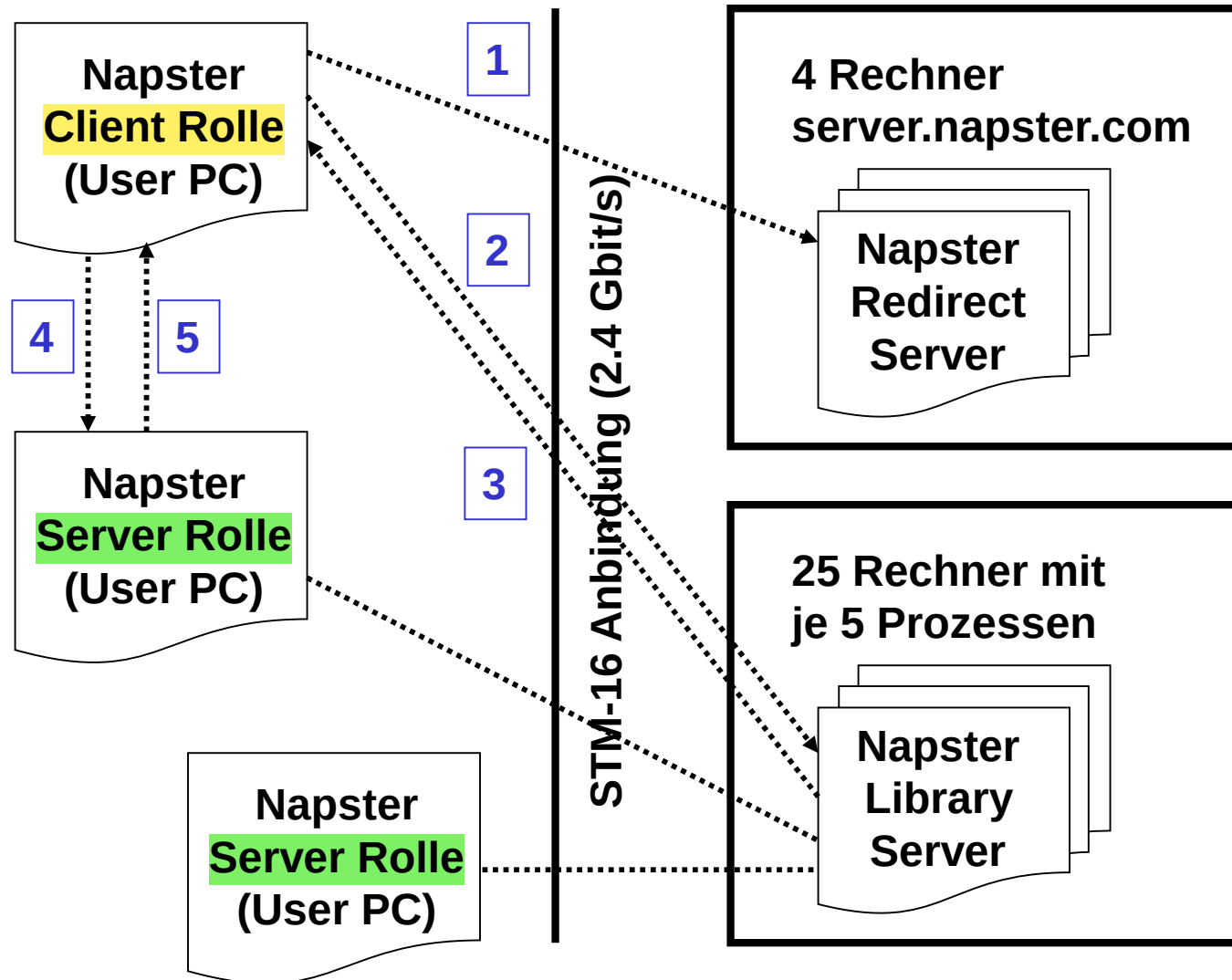


Peer-to-Peer Modell

- **Dezentrales System**, vermeidet dedizierte Server
- Gleichberechtigte Peer-Knoten bilden ein logisches „**Overlay Netzwerk**“
- Vorteile:
 - Keine Anschaffung und Wartung von Servern
 - Geringer administrativer Aufwand („**selbstorganisierend**“)
 - **Robust** (kein Single Point of Failure)
 - Skaliert mit Anwenderanzahl

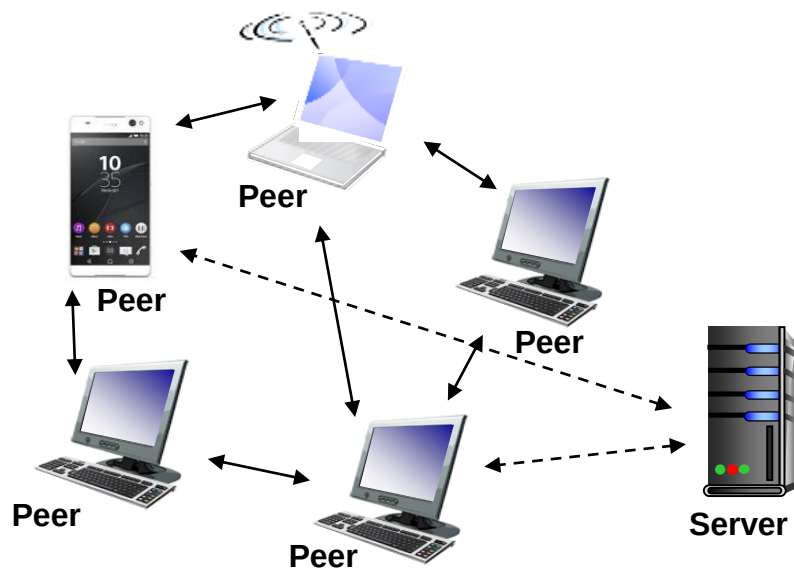


Ursprung: Napster (bis 2001)



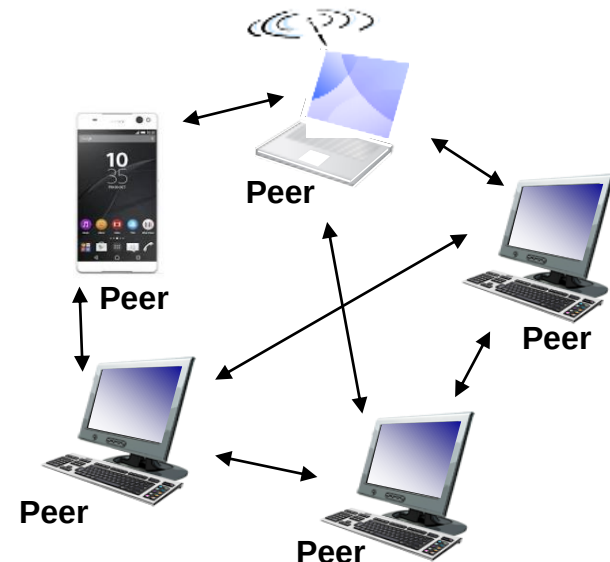
Peer-to-Peer-Netze: Hybrides oder reines Modell

Hybrides P2P Modell

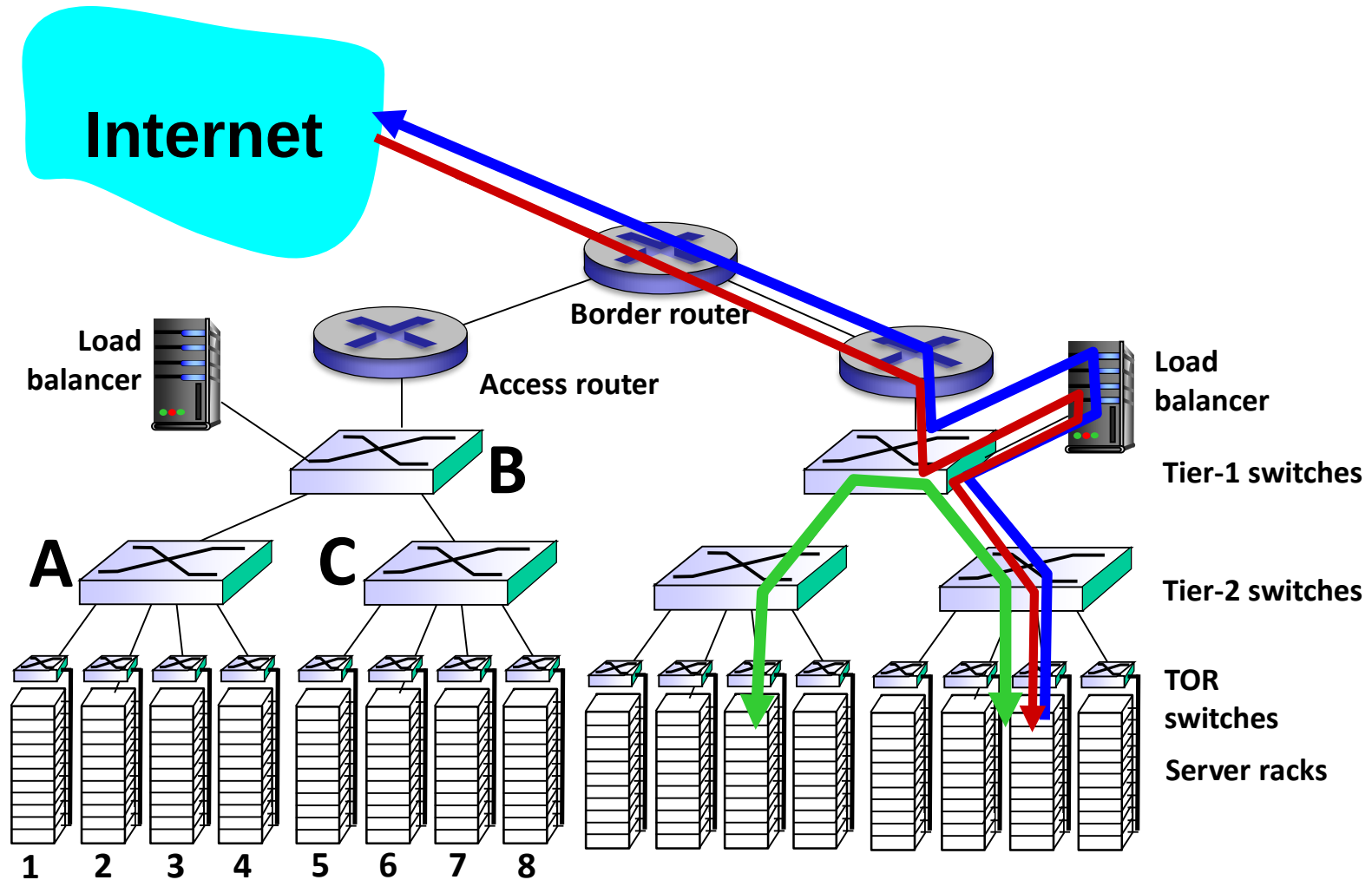


- Napster
- BitTorrent (Tracker-basiert)

- Gnutella
- BitTorrent (DHT-basiert)
- Blockchain

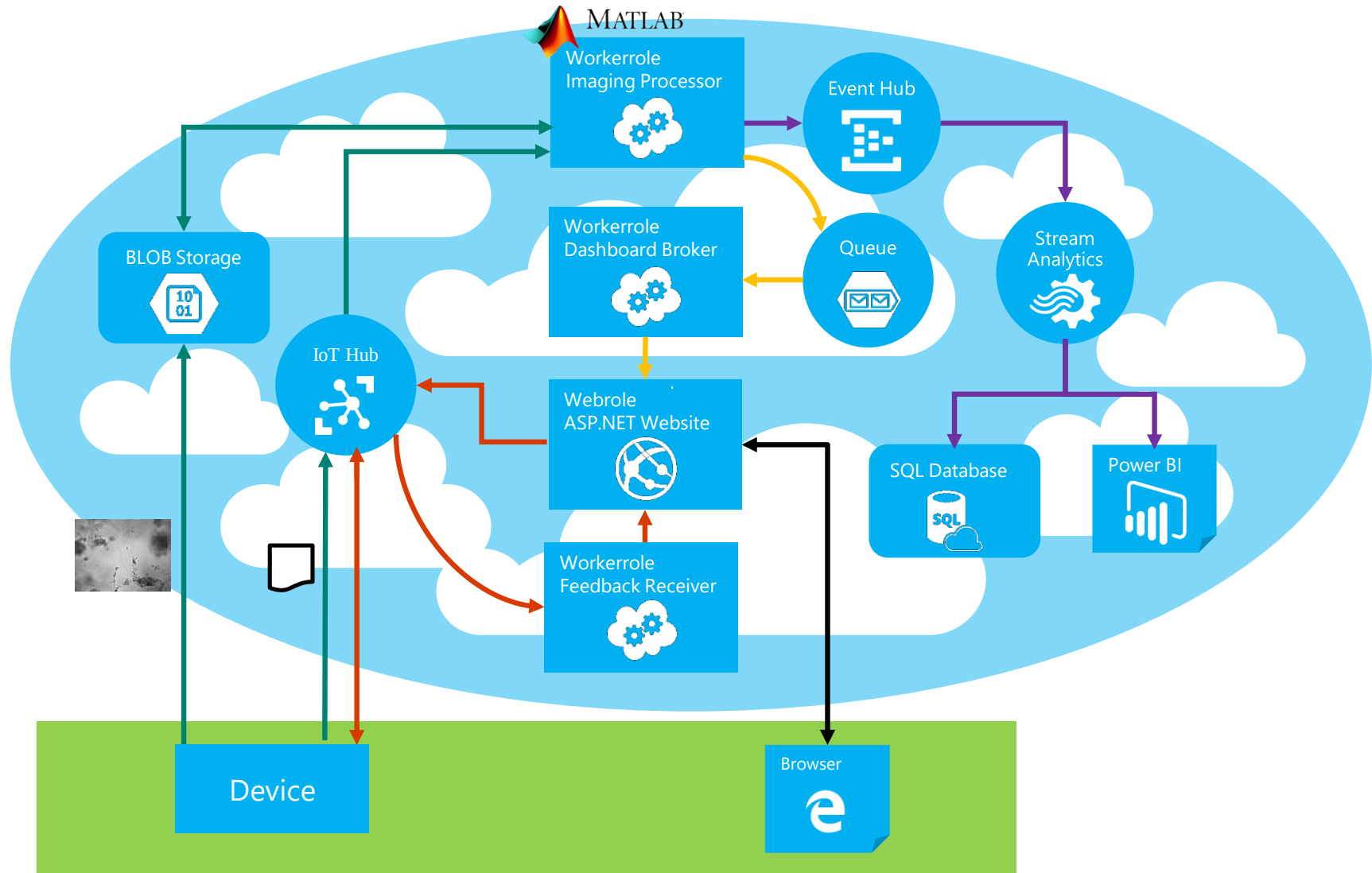


Reines P2P Modell

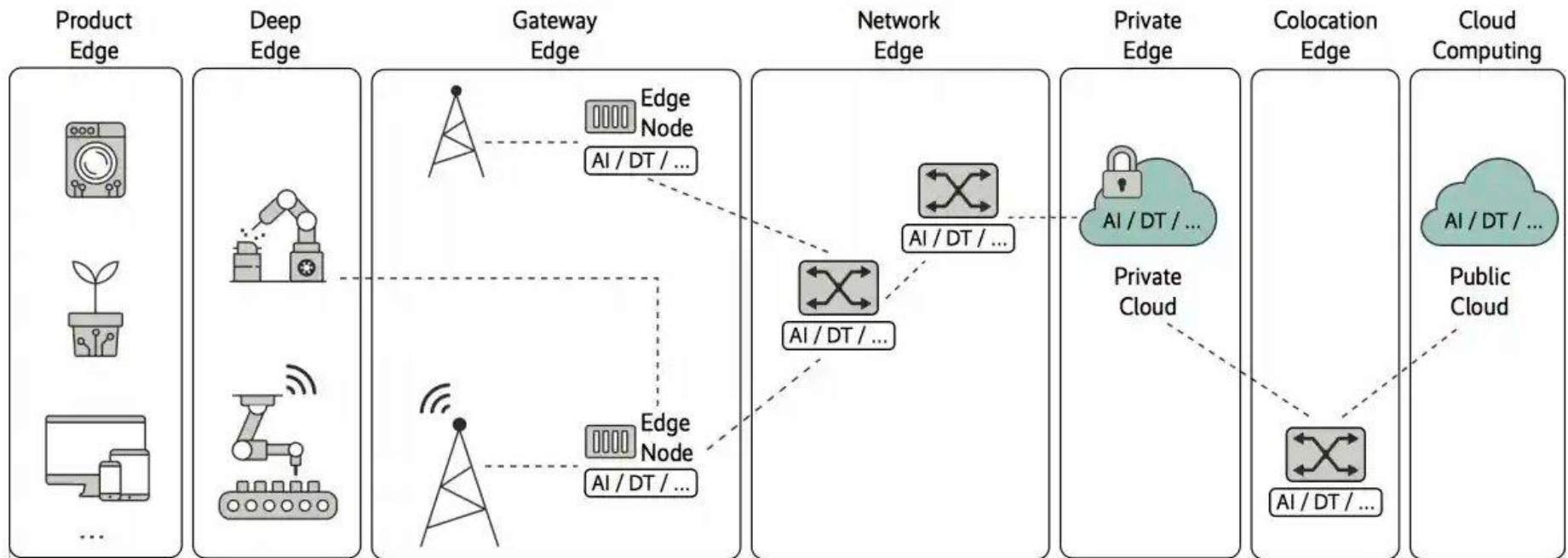




Verarbeitung von Bilddaten einer Kläranlage im Internet der Dinge



Edge Computing



Latency, Jitter, Network Dependency, Backhaul Traffic

Data Privacy and Protection, Context Awareness, Autonomy

Energy, Value of Data

Costs

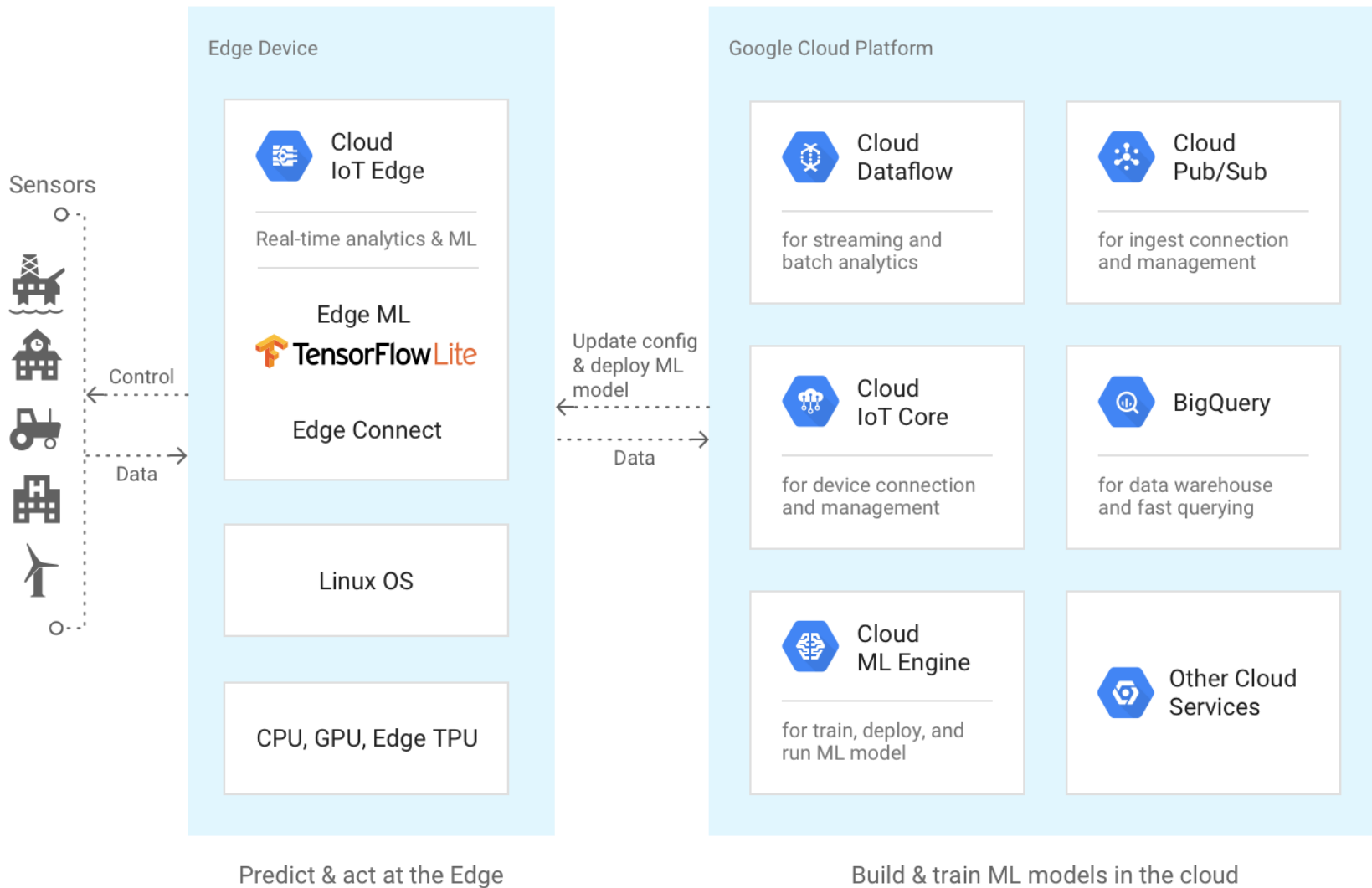


- **Colocation Edge: Amazon AWS Wavelength**
- **Private Edge: Microsoft Azure Stack, Azure local**
- **Network Edge: Betriebssystem Cisco IOx**
- **Gateway Edge: Dell Gateway Edge**
- ...

Referenzarchitektur für Edge Computing



Beispiel für Cloud IoT Edge Architektur



1. Einführung

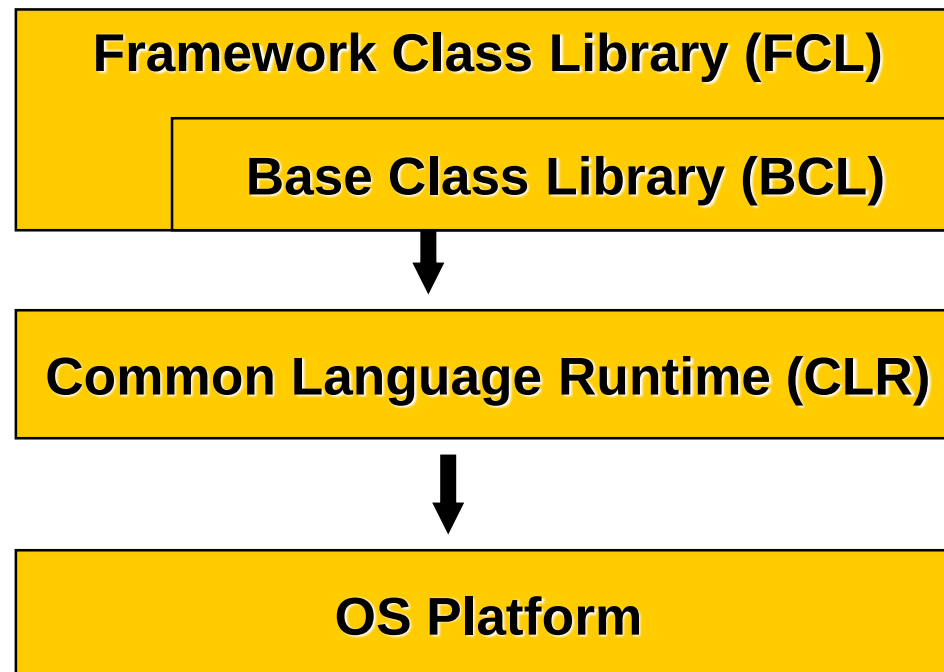


1.1 Cloud, Edge und Fog Computing

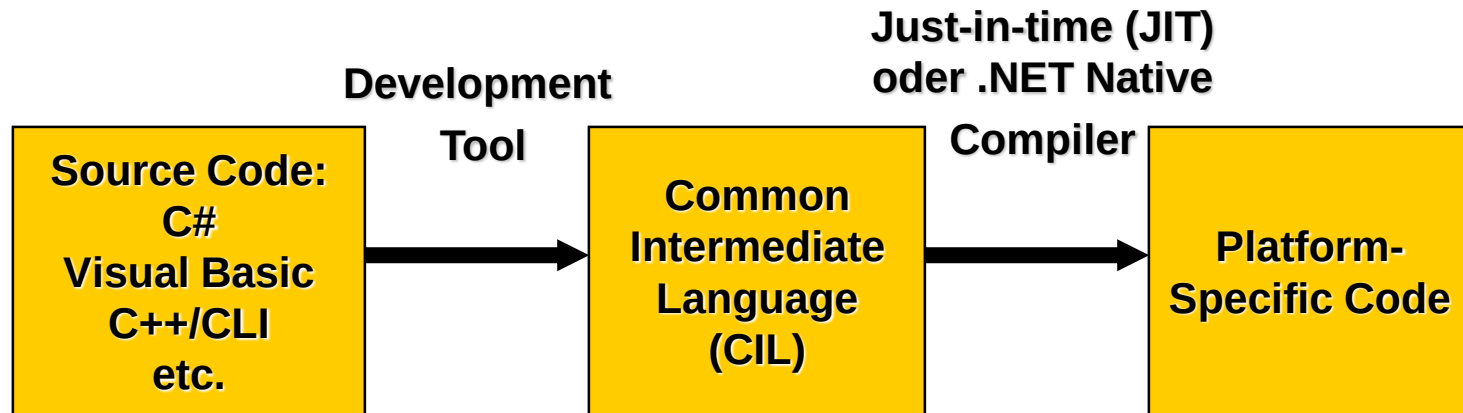
1.2 Verteilte Systemarchitekturen

**1.3 Anwendungsentwicklung:
.NET und C#**

- **.NET Framework**
 - Objektorientierte Klassenbibliothek
 - Laufzeitumgebung
- **.NET Grobarchitektur:**



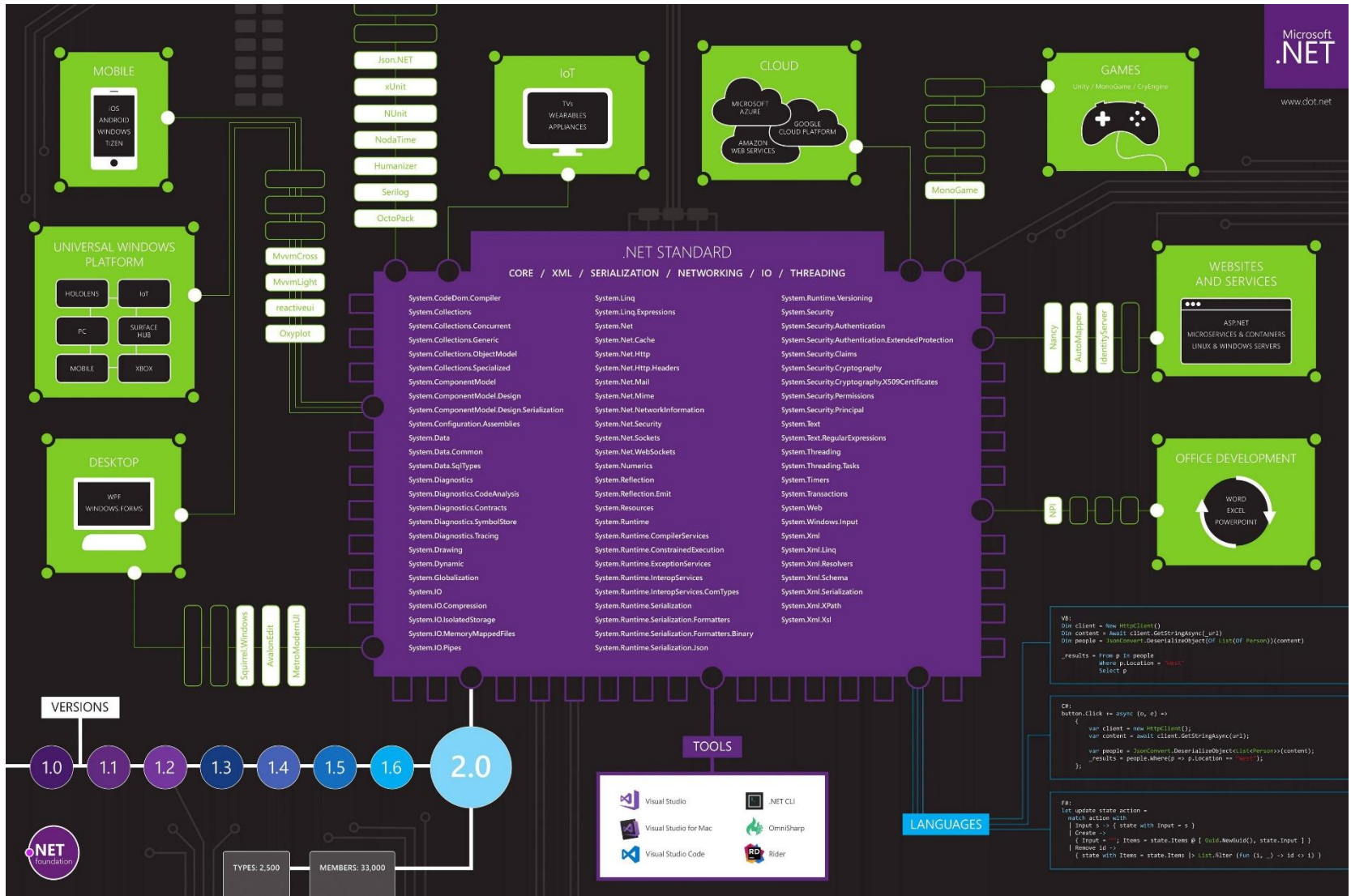
Prinzip der Common Language Runtime (CLR)



Grundlegende Eigenschaften:

- Interoperabilität zwischen verschiedenen Programmiersprachen, aber mit C# als Zielsprache
- Automatische Speicherbereinigung (Garbage Collection)
- Versionierung
- Sicherheitsmechanismen

Framework Class Library (FCL)



.NET – A unified platform





- **C/C++ - ähnliche komponenten-orientierte Sprache**
- **Garbage Collection**
 - **Keine Speicherlecks oder „wilde Pointer“**
- **Exceptions**
 - **Error Handling von Anfang an bedacht**
- **Typsicherheit**
 - **Keine uninitialisierten Variablen**
 - **Keine unsicheren Casts**

„Hello World“ in C#



```
// Datei Hello.cs

using System;

class Hello {
    public static void Main() {
        Console.WriteLine("Hello World");
    }
}
```



- **Namespaces**
 - **Enthalten Typdefinitionen und Namespaces**
- **Typdefinitionen**
 - **Klassen, Strukturen, Interfaces, ...**
- **Elemente von Typen**
 - **Konstanten, Felder, Methoden, Properties, Indexer, Events, Operatoren, Konstruktoren, Destruktoren**
- **Organisation der Dateien**
 - **Keine Header-Dateien, Programmcode ist “inline”**
 - **Die Reihenfolge der Deklarationen ist ohne Bedeutung**

```
using System;

namespace System.Collections
{
    public class Stack
    {
        Entry top;

        public void Push(object data) {
            top = new Entry(top, data);
        }

        public object Pop() {
            if (top == null) throw new InvalidOperationException();
            object result = top.data;
            top = top.next;
            return result;
        }
    }
}
```



- `if`, `while`, `do` benötigen eine boolesche Bedingung
- `switch` Statement
 - Kein “fall-through“ im Switch Statement
 - `break`, `goto case` or `goto default` notwendig
- Checked und unchecked Statement
- Ausdrücke müssen etwas tun

```
if (value)    //Fehler  
    WriteLine("true");
```

```
if (value != 0)    //ok  
    WriteLine("true");
```

- Iteration von Arrays

```
public static void Main(string[] args)
{
    foreach (string s in args) Console.WriteLine(s);
}
```

- Iteration durch selbst definierte Collections

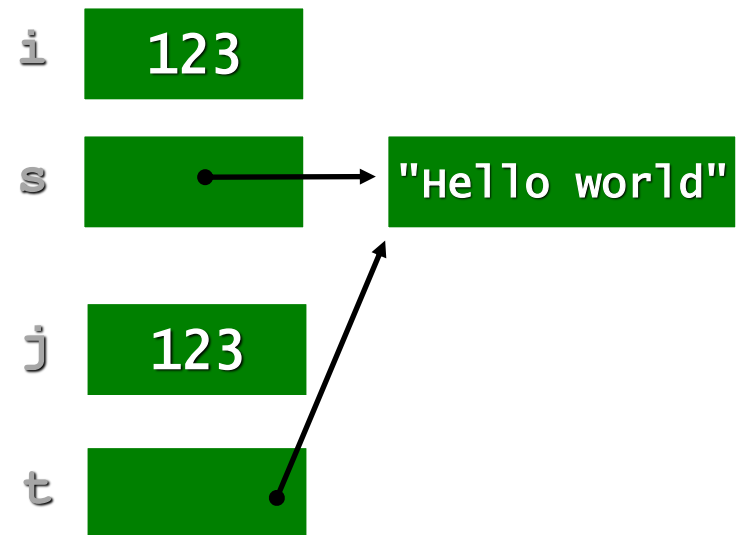
```
foreach (Customer c in customers.OrderBy(c=>c.Name))
{
    if (c.Orders.Count != 0)
    {
        ...
    }
}
```



	Value (struct)	Reference (class)
Variable enthält	Wert	Referenz
Speicher	Stack	Heap
Initialisiert mit	Alles 0	Konstante: null
Zuweisung	kopiert Wert	kopiert Referenz

```
int i = 123;
string s = "Hello world";
```

```
int j = i;
string t = s;
```





- **Classes (reference type)**
 - Wird für die meisten Objekte verwendet
- **Structs (value type)**
 - Für Daten-Objekte (Point, Complex, etc.).
- **Interfaces**

Klassen (class)

- Einfachvererbung (single inheritance)
- Implementierung von beliebig vielen Interfaces
- Elemente einer Klasse
 - Konstanten, Felder, Methoden, Operatoren, Konstruktoren, Destruktoren
 - Properties, Indexer, Events
 - Verschachtelte Typen
 - Statische Elemente (static)
- Zugriffsschutz
 - `public`, `protected`, `internal`, `private`

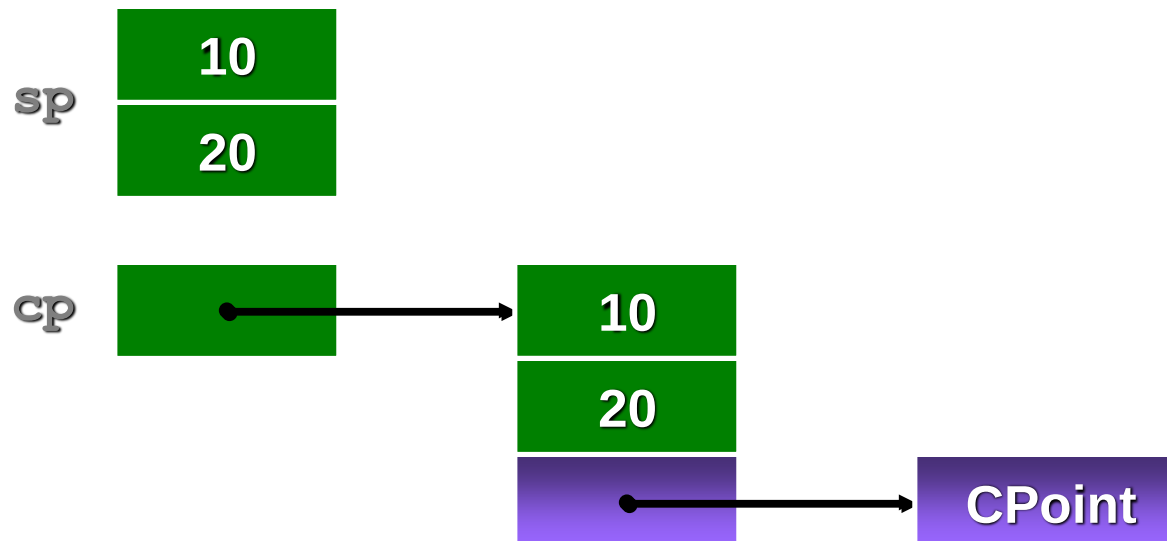
Strukturen (struct)

- Sind immer “value types“
- Ideal für “kleine” Objekte
 - Keine Allokierung auf dem Heap
 - Weniger Arbeit für den Garbage Collector
 - Effizientere Speicherbenutzung
 - Benutzer können “primitive” Typen selbst erzeugen
 - Syntax und Semantik sehr eingängig
 - Operator Overloading, Conversion Operators
- .NET Framework nutzt diese auch
 - `int, float, double, ...` sind alle structs

Classes und Structs

```
class CPoint { int x, y; ... }  
struct SPoint { int x, y; ... }
```

```
CPoint cp = new CPoint(10, 20);  
SPoint sp = new SPoint(10, 20);
```



- Enthalten Methoden, Properties, Indexer, und Events

```
interface IDataBound
{
    void Bind(IDataBinder binder);
}

class TextBox: Control, IDataBound
{
    void IDataBound.Bind(IDataBinder binder) {...}
}
```

- Methoden sind standardmäßig NICHT virtual
- Eine Methode kann nur mit `override` überschrieben werden

```
class B {  
    public virtual void foo() {}  
}  
  
class D : B {  
    public override void foo() {}  
}
```

- Methoden sind standardmäßig NICHT virtual
- Überschreiben einer nicht-virtual Methode
→ Compilerfehler!
Außer man verwendet `new` („Verdecken“)

```
class N : D {  
    public new void foo() {}  
}  
  
N n = new N();  
n.foo();           // call N's foo  
(D)n.foo();       // call D's foo  
(B)n.foo();       // call D's foo
```

- **in-Parameter**: Entspricht einer Übergabe „by value“. Es wird der Wert an die Methode übergeben

```
static void Foo(int p) {++p;}  
static void Main()  
{  
    int x = 8;  
    Foo(x);    // Kopie von x wird übergeben  
    Console.WriteLine(x); // x = 8  
}
```

- **ref-Parameter:** Man übergibt die Referenz an die Methode

```
static void Foo(ref int p) {++p;}
static void Main()
{
    int x = 8;
    Foo(ref x);      // Referenz von x wird übergeben
    Console.WriteLine(x); // x = 9
}
```

- **out-Parameter:** Erhält einen Wert von Methode zurück

```
static void Foo(out int p) {p = 3;}  
static void Main()  
{  
    int x;  
    Foo(out x); // x ist bei Übergabe nicht initialisiert  
    Console.WriteLine(x); // x = 3  
}
```



```
public class Button: Control
{
    private string caption;

    public string Caption {
        get {
            return caption;
        }
        set {
            caption = value;
            Repaint();
        }
    }
}
```

- Mischung aus Feldern und Methoden („smart fields“)

```
Button b = new Button();
b.Caption = "OK";
String s = b.Caption;
```



- **Konsistente Art, um Dokumentation aus dem Code zu erzeugen**
- **/// - Kommentare werden exportiert**
- **Dokumentation wird vom Compiler durch /doc: extrahiert werden**
- **Ein „kleines“ Schema ist eingebaut**

```
class XmlElement
{
    /// <summary>
    ///     Returns the attribute with the given name and
    ///     namespace</summary>
    /// <param name="name">
    ///     The name of the attribute</param>
    /// <param name="ns">
    ///     The namespace of the attribute, or null if
    ///     the attribute has no namespace</param>
    /// <return>
    ///     The attribute value, or null if the attribute
    ///     does not exist</return>
    /// <seealso cref="GetAttr(string)"/>
    ///
    public string GetAttr(string name, string ns) {
        ...}
}
```

- **C# unterstützt**
 - eingebauten Typ: `String`
 - benutzerdefinierte Referenztypen
 - große Auswahl an Collection-Klassen
 - Referenz- und Ausgabeparameter (`ref`, `out`)
- **99% der Pointer werden nicht mehr benötigt**
- **Dennoch sind Pointer verfügbar, wenn Programmcode mit `unsafe` markiert ist**

```
class FileStream: Stream
{
    int handle;

    public unsafe int Read(byte[] buffer, int index, int count)
    {
        int n = 0;
        fixed (byte* p = buffer)
        {
            ReadFile(handle, p + index, count, &n, null);
        }
        return n;
    }

    [DllImport("kernel32.dll", SetLastError=true)]
    static extern unsafe bool ReadFile(int hFile,
        void* lpBuffer, int nBytesToRead,
        int* nBytesRead, Overlapped* lpOverlapped);
}
```



1. Einführung

2. Cloud Computing

3. Middleware Layer



2.1 Grundlagen des Cloud Computing

2.2 Microsoft Azure

2.3 Infrastructure as a Service (IaaS)

2.4 Container as a Service (CaaS)

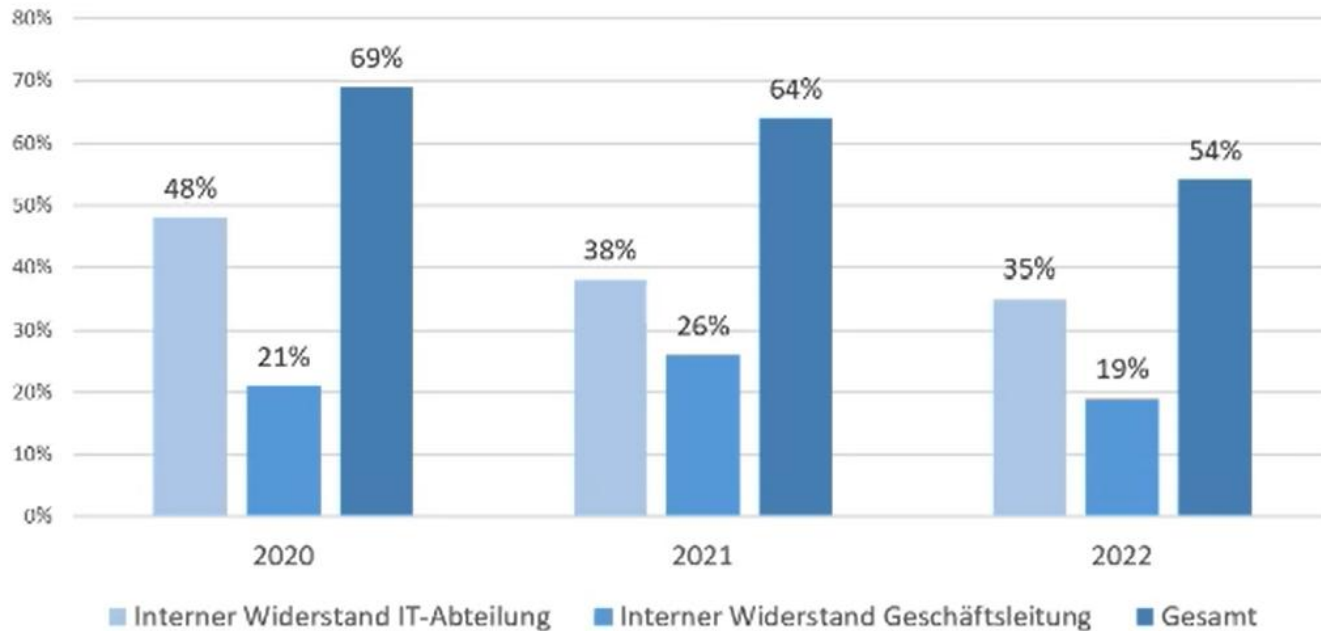
2.5 Platform as a Service (PaaS)

2.6 IoT Hub

2.7 Microsoft Entra ID



- **Der Widerstand bröckelt ...**



Quelle: Cloud Computing Marktbarometer Deutschland

Cloud Computing in a Nutshell

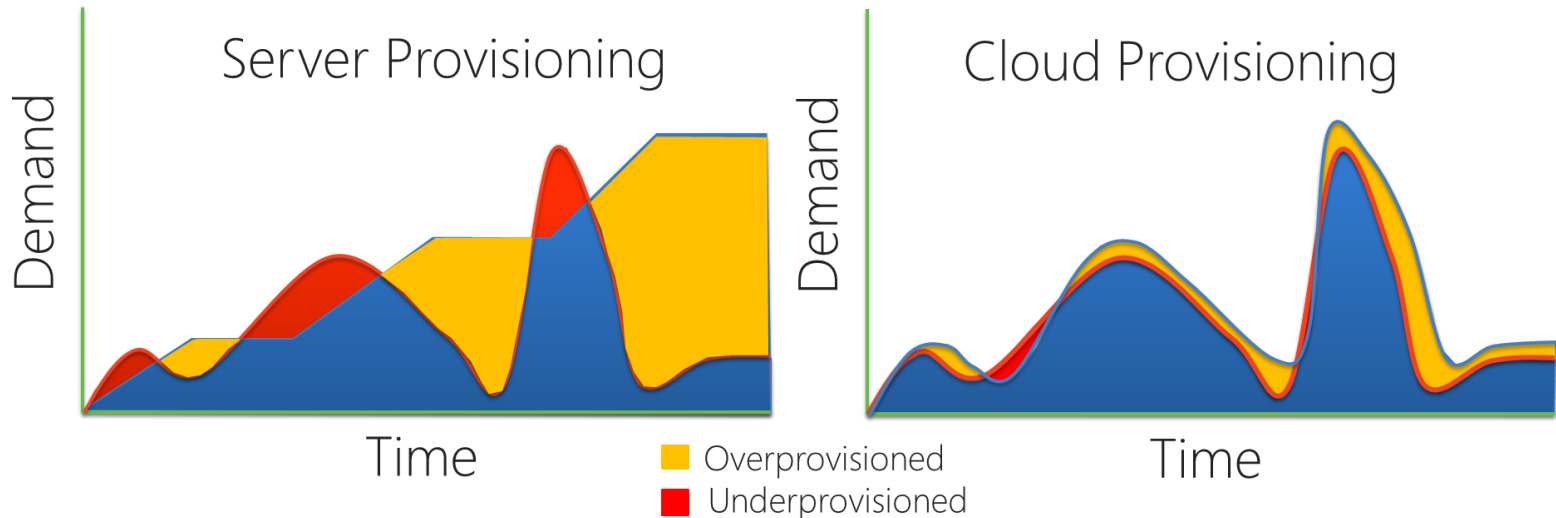


“Cloud Computing is the transformation of computer hardware, software and networks into a Utility just like your Electric Company, Water Company, or Gas Company.”



What is a “Cloud”?

- **NIST definition:** *(5 Aspects des Cloud Computing)*
 - **On-demand self-service**
 - **Broad network access**
 - **Multi-tenant (resource pooling)**
 - **Scalable compute** and storage resources („elasticity“)
 - **Measured service**



Defining the Cloud



*Unterschied
von Mitarbeitern
im Unternehmen*

Deployment Models

Private Cloud

Public Cloud

Hybrid Cloud

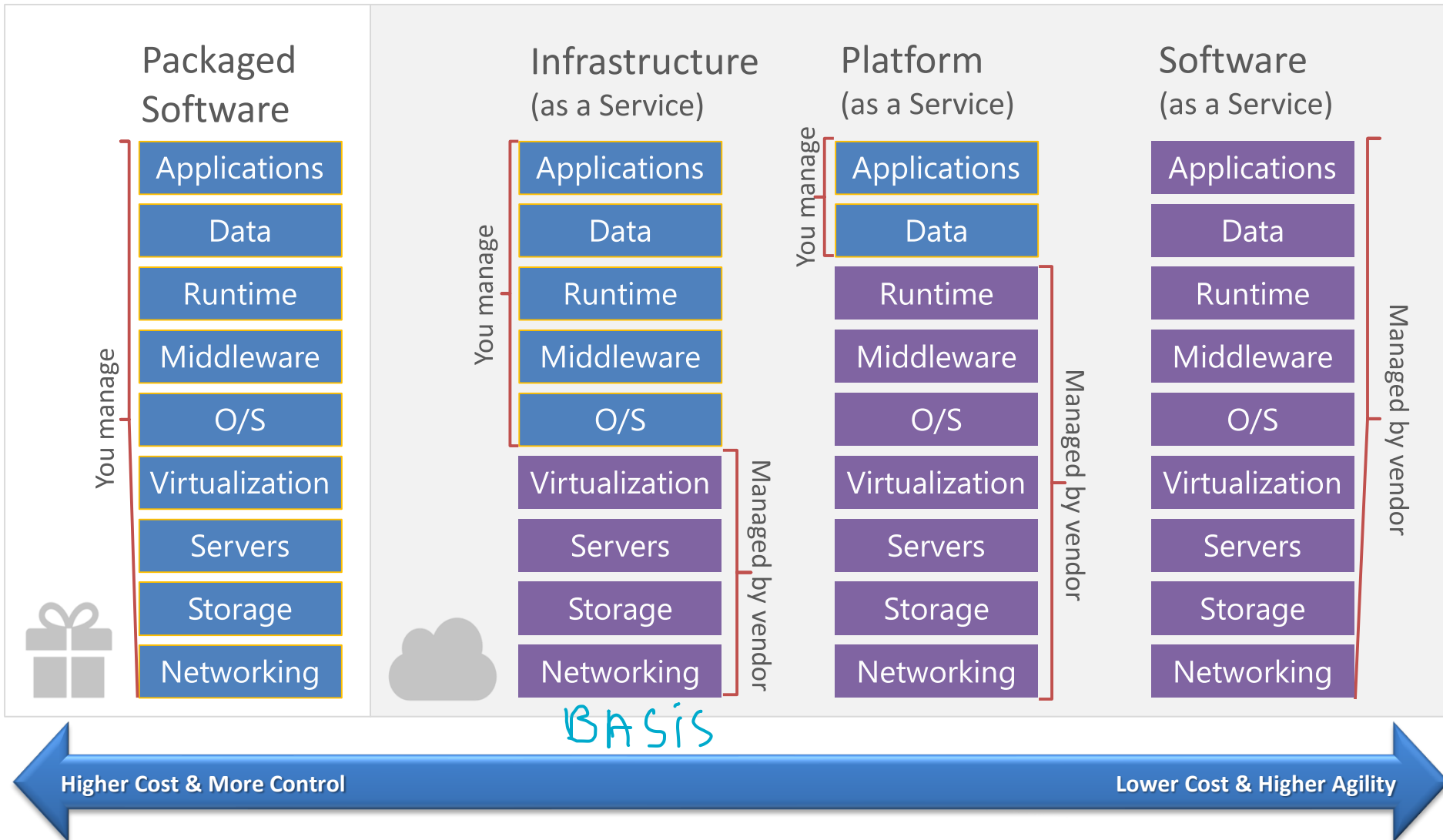
Service Models

Infrastructure as a Service
(IaaS)

Platform as a Service
PaaS

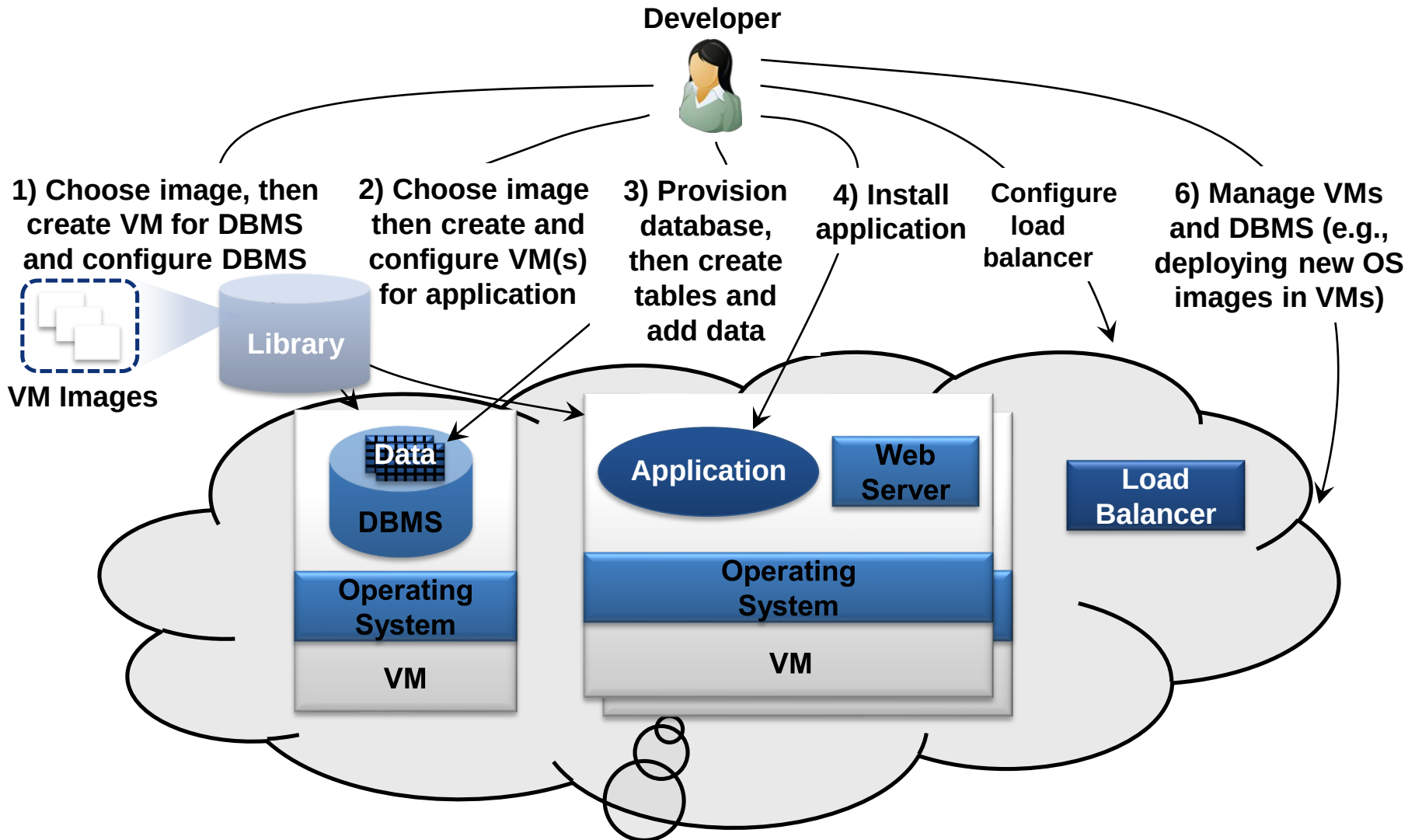
Software as a Service
SaaS

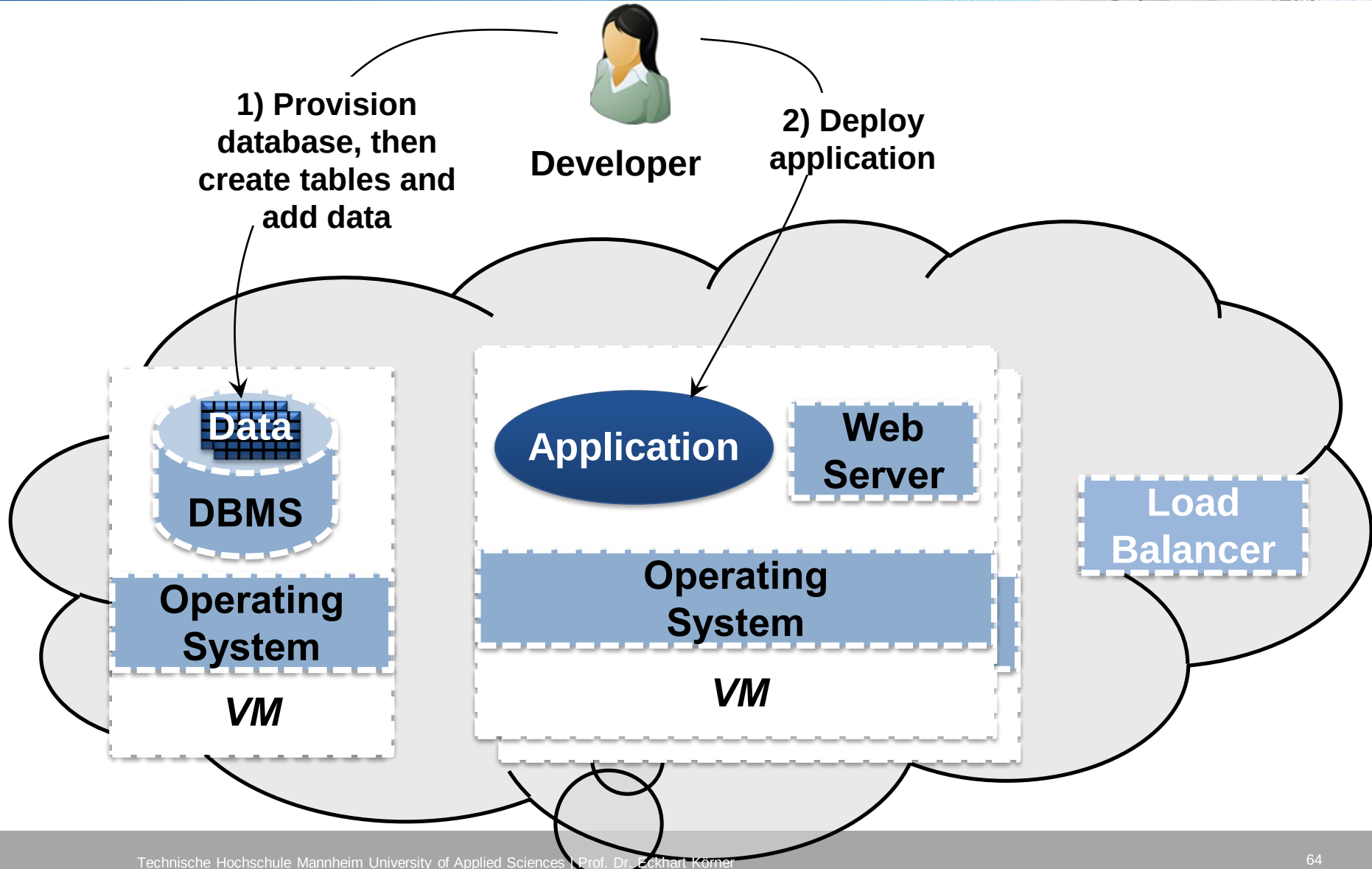
Cloud Computing Taxonomy





- **Infrastructure as a Service (IaaS):**
basic compute, network and storage resources
 - On-demand servers
 - Amazon EC2, VMWare vCloud
- **Platform as a Service (PaaS):** cloud application infrastructure
 - On-demand application hosting environment
 - E.g. SAP Cloud Platform, Google App Engine, Salesforce.com, Azure App Service
- **Software as a Service (SaaS):** cloud applications
 - On-demand applications
 - E.g. Office 365, GMail, Dropbox







2.1 Grundlagen des Cloud Computing

2.2 Microsoft Azure

2.3 Infrastructure as a Service (IaaS)

2.4 Container as a Service (CaaS)

2.5 Platform as a Service (PaaS)

2.6 IoT Hub

2.7 Microsoft Entra ID

Agenda

- Azure Overview
- The Azure Service Model
- Azure Storage
- High Availability and Azure Services
- Updating Azure Services
- Pricing



Azure Overview

Microsoft Azure

- Microsoft Azure is an OS for the data center
 - Handles resource management, provisioning, and monitoring
 - Manages application lifecycle
 - Allows developers to concentrate on business logic
- Provides common building blocks for distributed applications
 - Reliable queuing, simple structured storage, SQL storage
 - Application services like access control, caching, and connectivity
 - ...

Leading the journey to the Cloud

Microsoft Azure

DEVELOPER + APP PLATFORM	Visual Studio Family + Azure App Service
DATA + ANALYTICS	Azure AI
INTERNET OF THINGS	Azure IoT
CLOUD INFRASTRUCTURE	Azure + Azure Stack + Operations Management Suite

Microsoft Azure

CLOUD INFRASTRUCTURE

Compute



Virtual Machines



Containers



Remote App



Batch

Storage



Blob Storage



Azure Files



Premium Storage



Entra ID



Multi-Factor Auth



Automation



Key Vault

Security + Management



Express Route



Virtual Network



Load Balancer



DNS



Traffic Manager



VPN Gateway



Application Gateway

Networking



Entra ID Connect Health



Entra Privileged Identity Mgmt



Backup



Operational Insights



Import/Export



StorSimple



Site Recovery

Hybrid Operations

Microsoft Azure

DEVELOPER + APP PLATFORM

App Platform Services

-  App Service
-  Notification Hubs
-  API Management
-  Service Fabric
-  Cloud Services

Integration

-  Storage Queues
-  BizTalk Services
-  Hybrid Connections
-  Service Bus

Development Tools

-  Visual Studio
-  DevOps Services
-  Application Insights
-  Azure SDK

Media & CDN

-  Content Delivery Network
-  Media Services

CLOUD INFRASTRUCTURE

Azure + Azure Stack + Operations Management Suite

Microsoft Azure

DATA + ANALYTICS

Data



SQL Database



SQL Data Warehouse



Cosmos DB



Data Lake



Redis Cache



Tables



Search

Analytics



HDInsight



Data Factory



Machine Learning



Mobile Engagement

INTERNET OF THINGS

Azure IoT

CLOUD INFRASTRUCTURE

Azure + Azure Stack + Operations Management Suite

Microsoft Azure

INTERNET OF THINGS



IoT Hub



Stream
Analytics



Logic Apps

CLOUD INFRASTRUCTURE

Azure + Azure Stack + Operations Management Suite



The Microsoft Azure App Service

Microsoft Azure Application Philosophy: Design for Failure

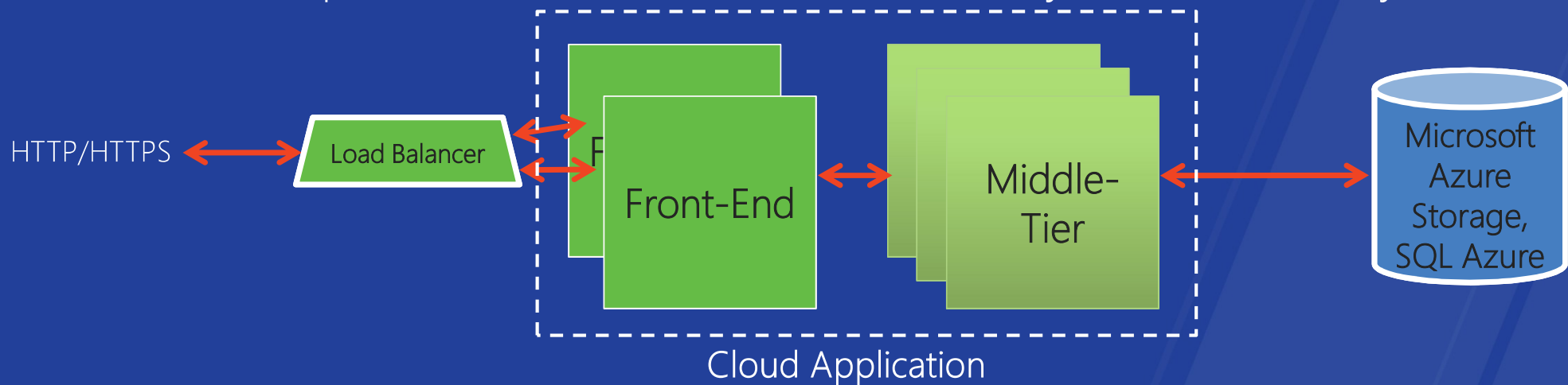
- Scale out for capacity
- Scale out for redundancy
- Asynchronous communication
- Short time outs with retries
- Idempotent operations
- Stateless with durable external storage

Let's Make a Cloud Application!

- Marketing wants to catch branding violations
 - Take as input an uploaded PowerPoint file and scan it for “branding violations” (use of “Microsoft Azure” and not “Windows Azure”)
- Requirements:
 - High availability
 - Apache/ASP.NET core MVC on LINUX
 - Scalable violation scanning workers

Multi-Tier Cloud Application

- A cloud application is typically made up of different components
 - Front end: e.g. load-balanced stateless web servers
 - Middle worker tier: e.g. order processing, encoding
 - Backend storage: e.g. SQL tables or files
 - Multiple instances of each for scalability and availability



Azure App Service Plan (Horizontal Scaling)

- App Service plans define
 - Region (West Europe, East US, etc.)
 - Scale count (one, two, three instances, etc.)
 - Stock Keeping Unit - SKU (Free, Shared, Basic, Standard, Premium)
- Availability is guaranteed to be at least 99,95% for Basic, Standard and Premium SKU
 - Achieved with update and fault domains
 - Requires at least two instances in each tier

Front-End

B3 Basic	
4	Kern
7	GB RAM
	10 GB Storage
	Benutzerdefinierte Do...
	SSL-Support Einschließlich SNI SSL
	Bis zu 3 Instanz(en) Manuelle Skalierung
188,22	
EUR/MONAT (GESCHÄTZT)	

Middle-Tier

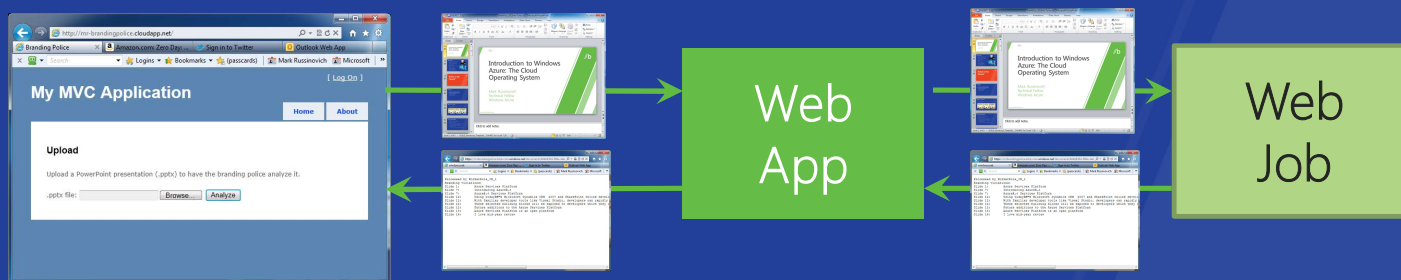
P3 Premium	
4	Kern
7	GB RAM
	250 GB Storage
	Benutzerdefinierte Do... Einschließlich SNI und IP-S...
	Bis zu 20 Instanz(en) * Verfügbarkeit vorbehalten
	20 Slots Web-App-Staging
	50 Mal pro Tag Sicherung
	Traffic Manager Geografische Verfügbarkeit
752,90	
EUR/MONAT (GESCHÄTZT)	



Microsoft Azure Storage

Branding Police Inter-Role Communication

- We have several types of communication between roles:
 - PowerPoint file sent from Web App to Web Job
 - Branding violations returned from Web Job to user
- Requirements:
 - Communication must be asynchronous
 - Must support concurrent violation scans for different users
 - Failure of any node must cause at worst a delay
- Let's look at Microsoft Azure storage and communications support...

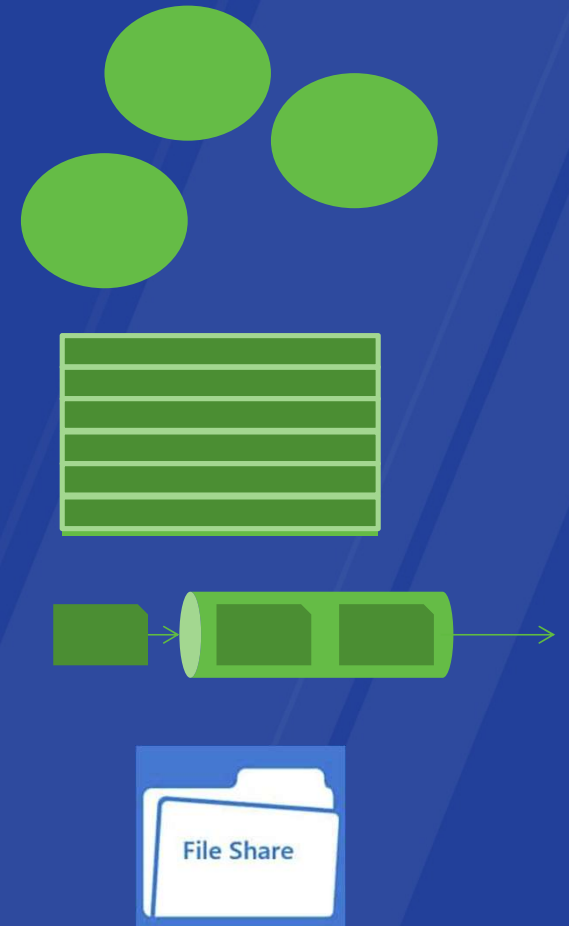


Microsoft Azure Storage Fundamentals

- Storage characteristics
 - Durable – replicated three times
 - Scalable (capacity and throughput)
 - Highly available
- Simple and familiar programming interfaces
 - REST (HTTP and HTTPS)
 - .NET accessible

Storage Objects

- Blobs
 - Provide a simple interface for storing named files along with metadata for the file
- Tables
 - Provide lightly structured storage with a set of entities that contain a set of properties
- Queues
 - Provide reliable storage and delivery of messages
- Files
 - File shares accessible via Server Message Block (SMB) or Network File System (NFS) protocol



Storage Account and Blob Containers

- Storage account
 - An account can have many blob containers
- Container
 - A container is a set of blobs
 - Sharing policies are set at the container level
 - Public READ or Private
 - Associate metadata with container
 - Metadata is <name, value> pairs
 - Up to 8KB per container
 - List the blobs in a container

Blob Storage Concepts

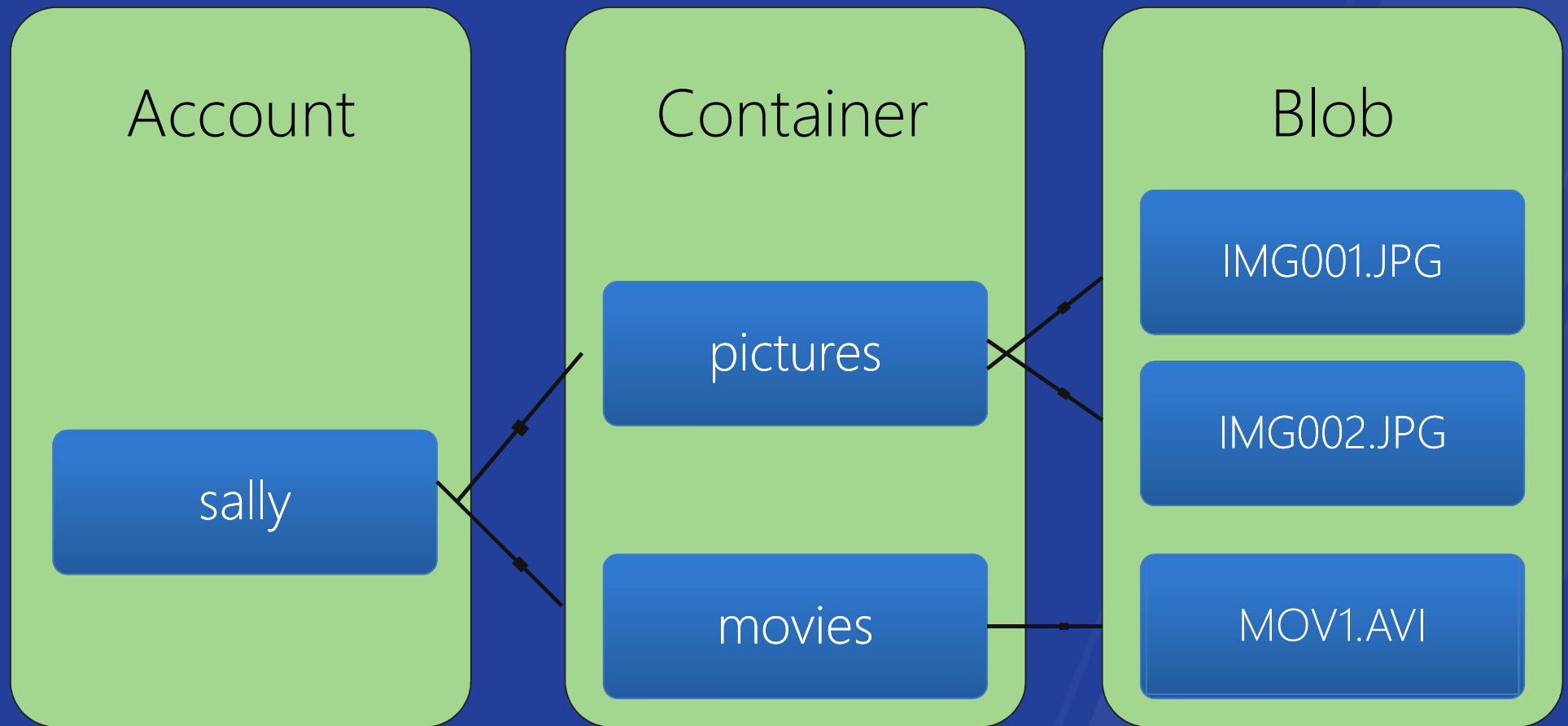
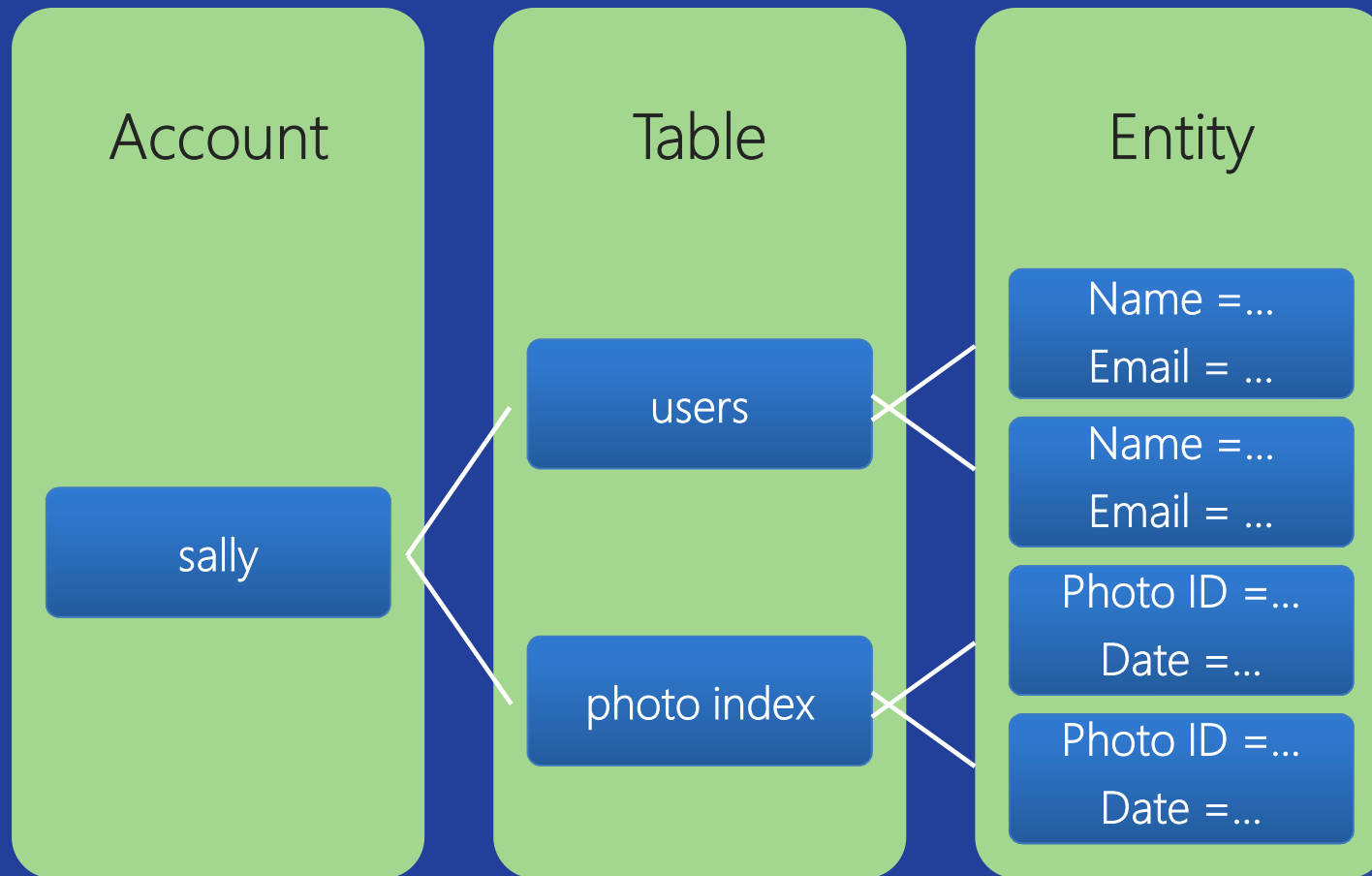


Table Data Model

- Table
 - A storage account can create many tables
 - .NET classes and LINQ
- A table is a set of entities (rows)
 - An entity is a set of properties (columns)
 - Billions of entities and TBs of data
- Two “key” properties that together are the unique ID of the entity in the table
 - PartitionKey – enables scalability
 - RowKey – uniquely identifies the entity within the partition

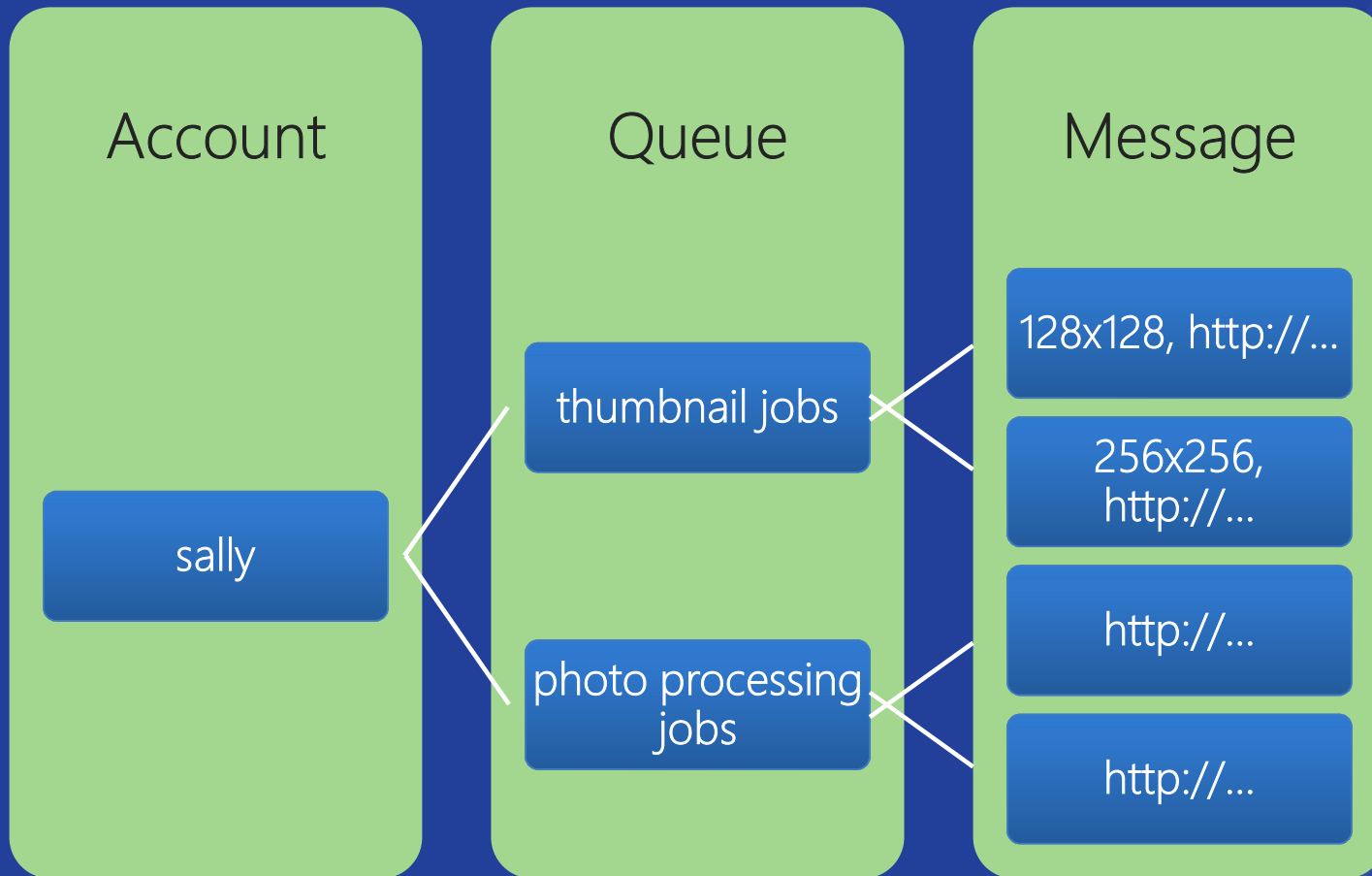
Table Storage Concepts



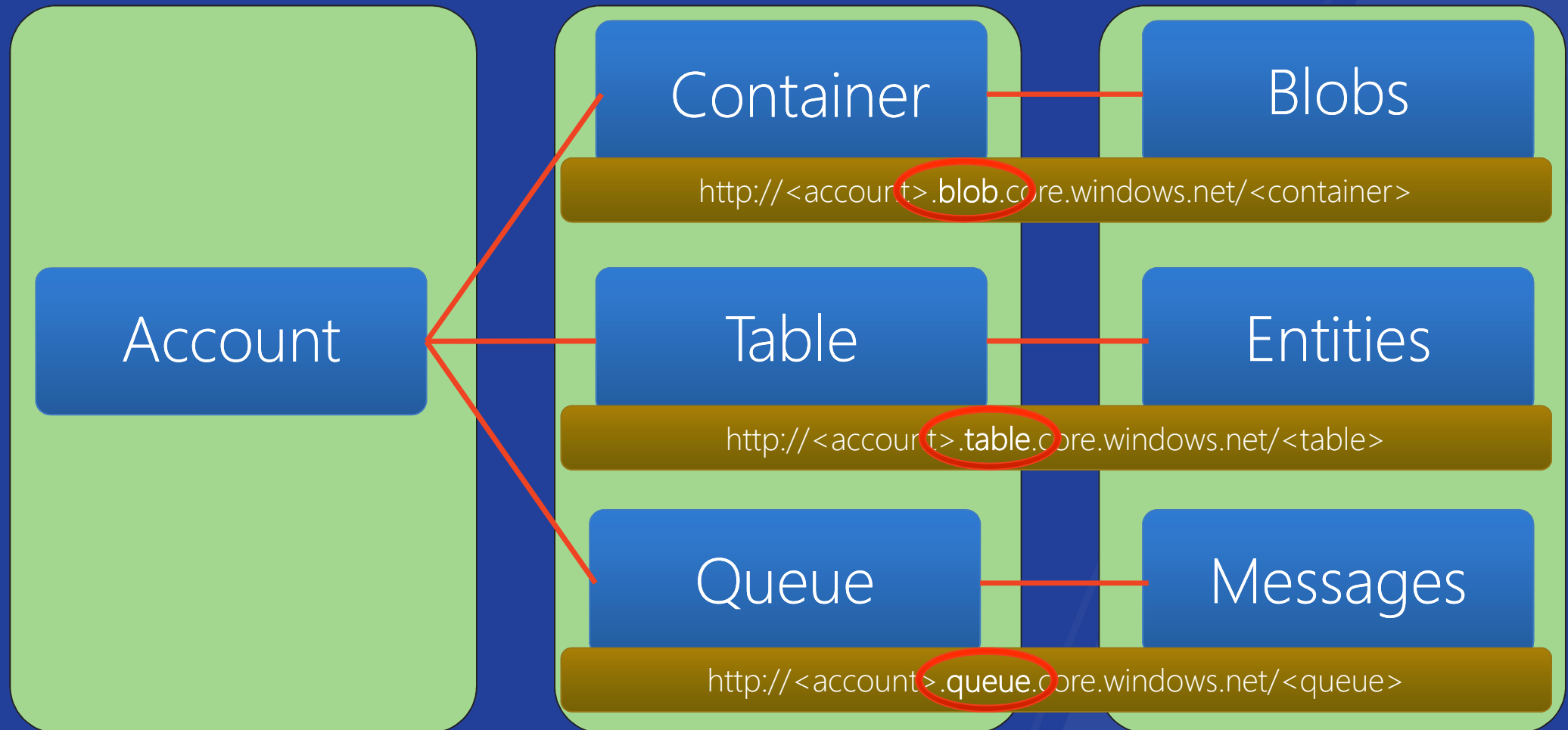
Microsoft Azure Queues

- Provide reliable message delivery
 - Simple, asynchronous work dispatch
 - Programming semantics ensure that a message can be processed at least once
- Queues are highly available, durable and performance efficient
 - Maximum size is 64K
 - FIFO in general, but not guaranteed
- Pulling an item from the queue doesn't delete it
 - It becomes invisible for a visibility timeout
 - Item must be deleted before timeout or else it becomes visible

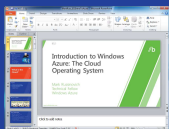
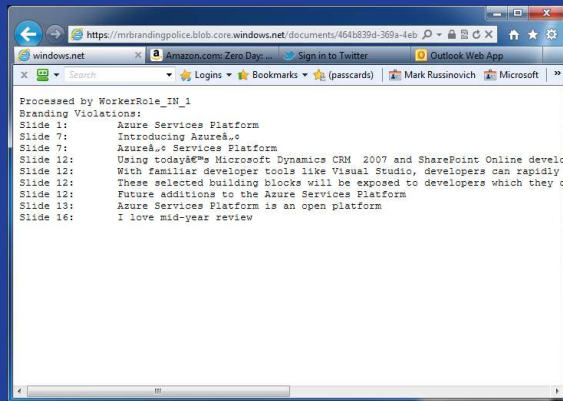
Queue Storage Concepts



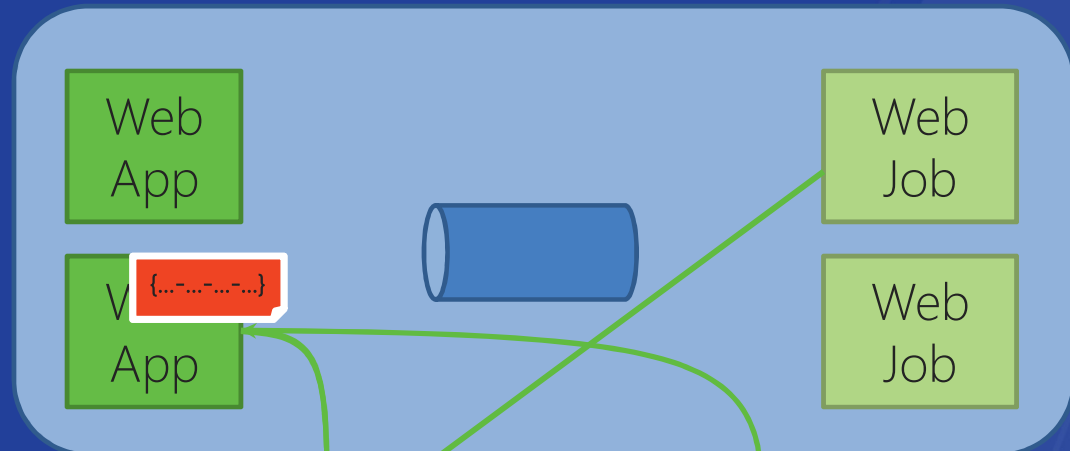
Microsoft Azure Data Storage Concepts



Branding Police



brandingpolice.cloudapp.net



{...-...-...-...}-report.txt

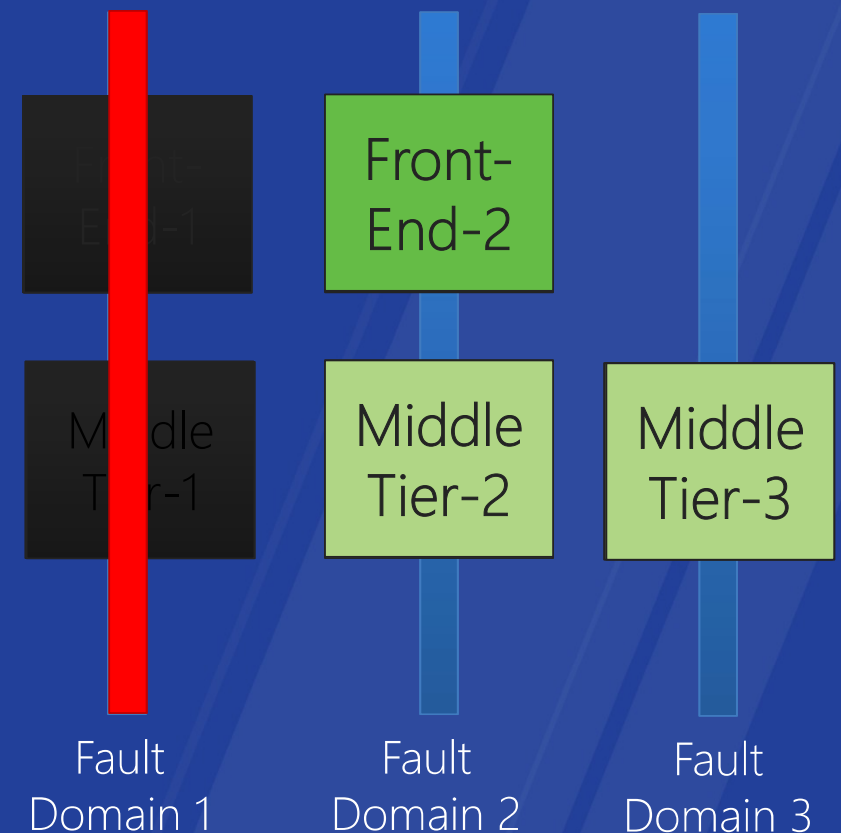
{...-...-...-...}



High Availability and Microsoft Azure Services

Availability: Fault Domains

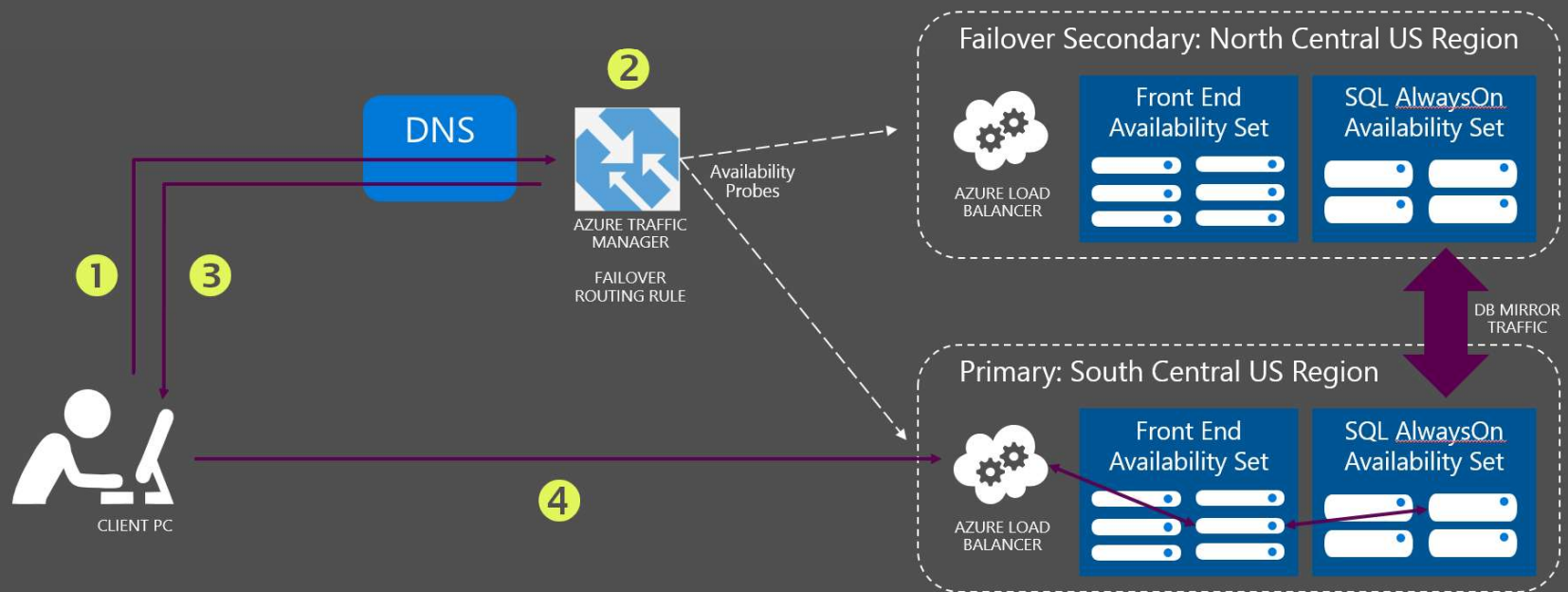
- Avoid single points of physical failures
- Unit of failure based on data center topology
 - E.g. top-of-rack switch on a rack of machines
- Microsoft Azure considers fault domains when allocating service roles
 - At least 3 fault domains per service
 - Will try and spread roles out across more
- Availability SLA – 99.95%



Specialised Load Balancer: Traffic Manager

Customer Responsibility: Building Cross-Region High Availability

Basic Topology



Updating Microsoft Azure Services

TH•M

The screenshot shows the VMware Support & Downloads page. The browser address bar displays 'http://www.vmware.com/support/'. The page features a navigation bar with links for Community, Forums, Technical Resources, Virtual Appliances, Store, and My Account. Below this is a secondary navigation bar with links for Cloud Computing, Virtualization, Solutions, Products, Services, Support & Downloads (highlighted), Partners, and Company. The main heading is 'Support & Downloads'. A search bar is located on the right. On the left, there is a 'Search VMware Knowledge Base' section with a warning icon and two entries: 'Scheduled Maintenance - September 9th' and 'Scheduled Maintenance - September 3rd'. The main content area contains two detailed announcements. The first announcement, 'Scheduled Maintenance - September 9th', states that VMware will be performing a system upgrade to several VMware Web applications on September 9th. Maintenance will begin on Friday, September 9th, at 6:00PM Pacific Time. All maintenance should be completed by Saturday, September 9th at 6:00AM Pacific Time. For more information, see the link below. The second announcement, 'Scheduled Maintenance - September 3rd', states that VMware will be performing a system upgrade to several VMware Web applications on September 3rd. Maintenance will be Saturday, September 3rd, 9:00PM - 11:00PM Pacific Time. During this time you will need to call VMware Support for assistance. See the link below for more information. A 'Learn More' link is provided. At the bottom, there are tabs for 'By Product' and 'By Task', with 'Datacenter & Cloud' selected under 'By Product'. A 'Top Support Resources' section is also visible, listing 'Product Documentation' and 'Documentation online at vmware.com'.

Scheduled Maintenance - September 9th
VMware will be performing a system upgrade to several VMware Web applications on September 9th. Maintenance will begin on Friday, September 9th, at 6:00PM Pacific Time. All maintenance should be completed by Saturday, September 9th at 6:00AM Pacific Time. For more information, see the link below.

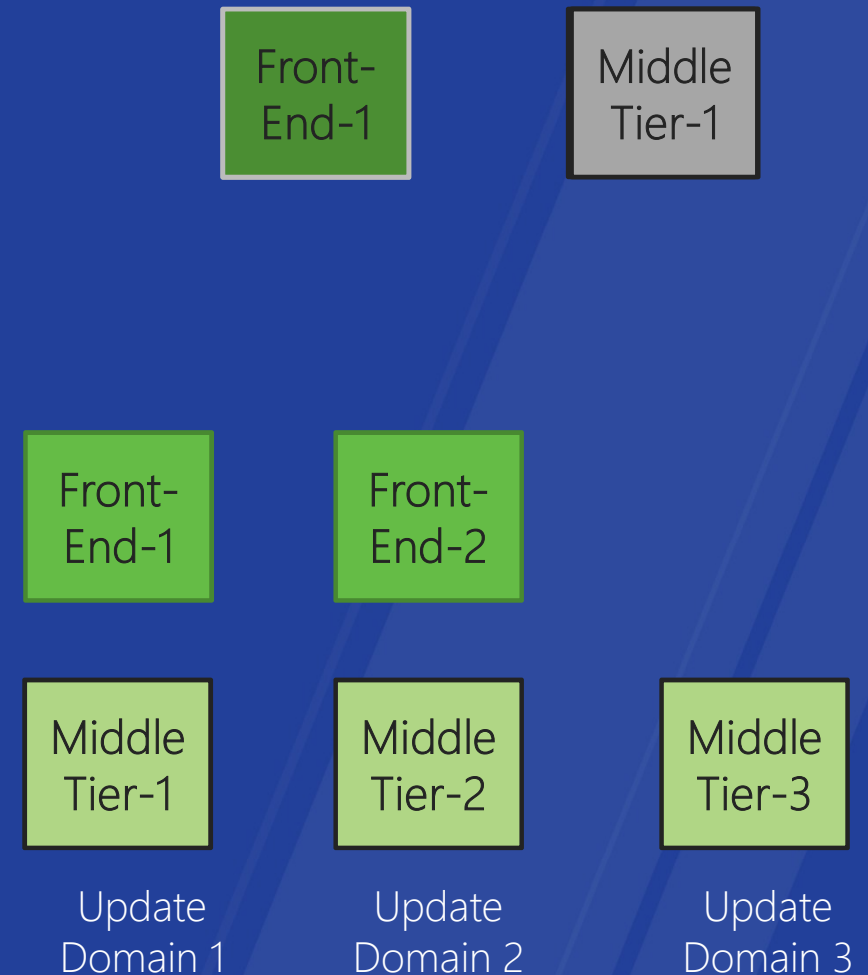
Scheduled Maintenance - September 3rd
VMware will be performing a system upgrade to several VMware Web applications on September 3rd. Maintenance will be Saturday, September 3rd, 9:00PM - 11:00PM Pacific Time. During this time you will need to call VMware Support for assistance. See the link below for more information.
[Learn More](#)

Update Types

- There are two update types:
 - In-place: updating a live service (IaaS)
 - Public/Private IP swap: for testing first (PaaS)
- In-place update:
 - Role instances updated one update domain at a time
 - Two modes: automatic and manual
- Public/Private IP address swap:
 - Two slots: staging and production
 - New version of service deployed to staging slot
 - External Public/Private IP mapping swapped with old
 - Note: slots retain IP address

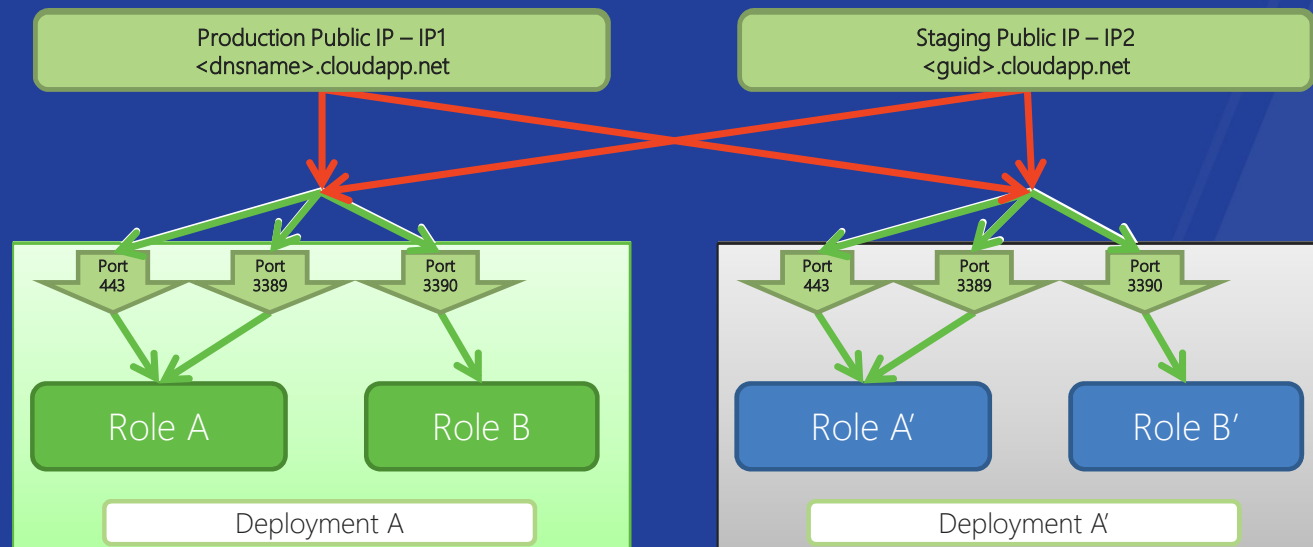
In-Place Update

- Purpose: Ensure service stays up while updating and Microsoft Azure OS updates
- System considers update domains when upgrading a service
 - 1/Update domains = percent of service that will be offline
 - Default is 5 and **max is 20**, override with `upgradeDomainCount` service definition property
- The Microsoft Azure SLA is based on at least two update domains



Public IP Swap Detail

- Note about Public IPs: You keep your assigned Public IP for the lifetime of a deployment
- Even for Public IP swap, “production” retains the production IP



Update Type Capabilities

Supported Changes	In-place	IP Swap	Delete and Redeploy
Operating system version	Yes	Yes	Yes
.NET trust level	Yes	Yes	Yes
Virtual machine size	Soon	Yes	Yes
Local storage settings	Yes (increase only)	Yes	Yes
Number of roles	Soon	Yes	Yes
Number of role instances	Yes	Yes	Yes
Number or type of endpoints	Soon	No	Yes
Names and values of configuration settings	Soon	Yes	Yes
Values (but not names) of configuration settings	Yes	Yes	Yes
Add new certificates	Soon	Yes	Yes
Change existing certificates	Yes	Yes	Yes
Deploy new code	Yes	Yes	Yes



Microsoft Azure Pricing

Pricing and Pricing Calculator

- <https://azure.microsoft.com/en-us/pricing/>
- <https://azure.microsoft.com/en-us/pricing/calculator/>

Conclusion

- The Cloud enables pay-as-you-go self-service provisioning of application resources
- Platform as a Service is all about reducing management and operations overhead
- Azure enables you to develop and deploy scalable, highly-available applications in minutes
- With Azure, you can deploy code using any language or runtime



2.1 Grundlagen des Cloud Computing

2.2 Microsoft Azure

2.3 Infrastructure as a Service (IaaS)

2.4 Container as a Service (CaaS)

2.5 Platform as a Service (PaaS)

2.6 IoT Hub

2.7 Microsoft Entra ID



- **Folien sind separat in Moodle verfügbar**

2. Cloud Computing



2.1 Grundlagen des Cloud Computing

2.2 Microsoft Azure

2.3 Infrastructure as a Service (IaaS)

2.4 Container as a Service (CaaS)

2.4.1 Docker

2.4.2 Kubernetes

2.5 Platform as a Service (PaaS)

2.6 IoT Hub

2.7 Microsoft Entra ID



- **What is Docker?**
 - Docker Container vs. Virtual Machine
 - Run Platforms
- **Images and Containers**
- **Volumes**
- **Networking**
- **Docker Registry**
- **Docker Use Cases**

What is Docker?

Docker is an open-source project that automates the deployment of applications inside software containers, by providing an additional layer of abstraction and automation of operating system–level virtualization on Linux and Windows.

[Source: en.wikipedia.org]

Problem with Distributed Applications



Static website

nginx 1.5 + modsecurity + openssl + bootstrap 2



Background workers

Python 3.0 + celery + pyredis + libcurl + ffmpeg + libopencv + nodejs + phantomjs



User DB

postgresql + pgv8 + v8



Queue

Redis + redis-sentinel



Analytics DB

hadoop + hive + thrift + OpenJDK

Multiplicity of stacks

Do services and apps interact appropriately?



Web frontend

Ruby + Rails + sass + Unicorn



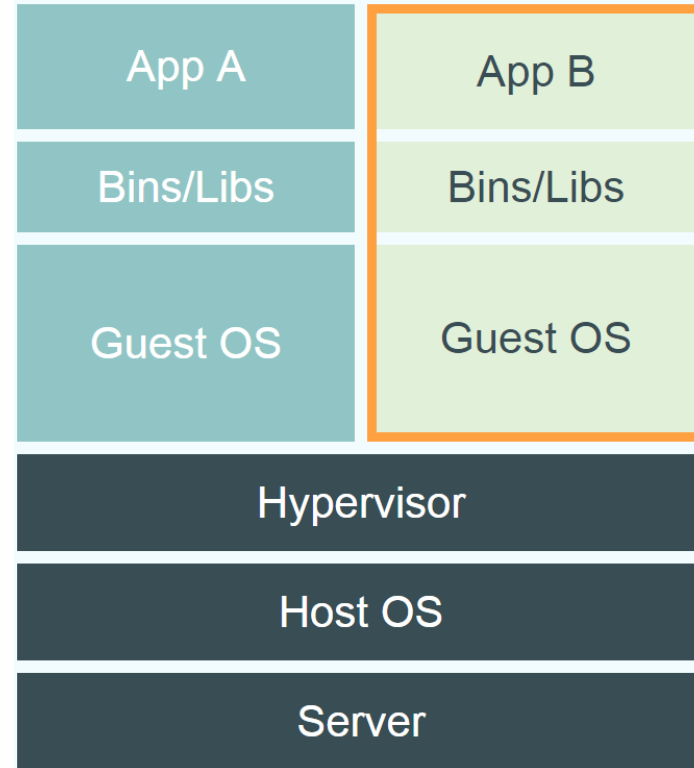
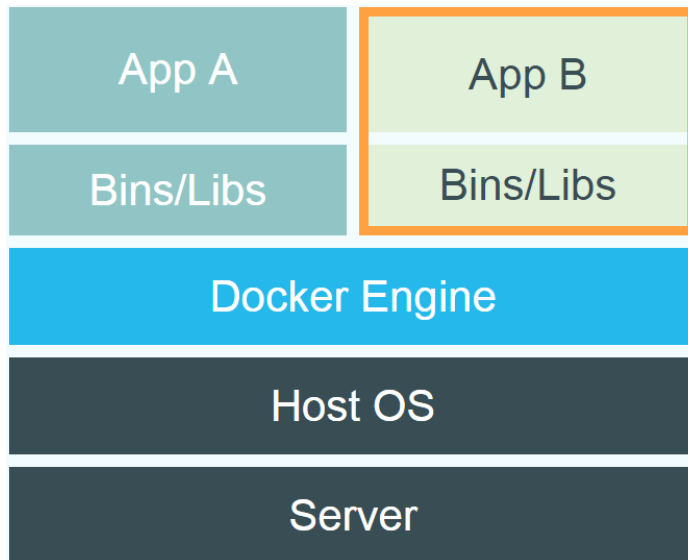
API endpoint

Python 2.7 + Flask + pyredis + celery + pycopg + postgresql-client



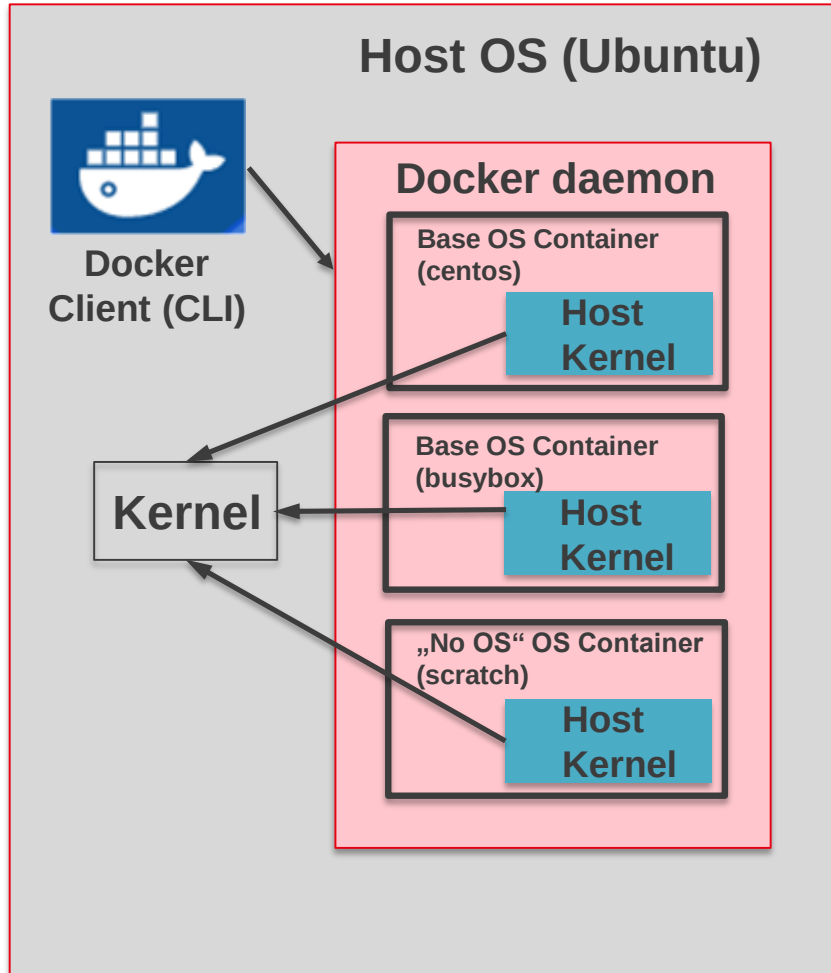
- Provide a uniformed wrapper around a software package: «*Build, Ship and Run Any App, Anywhere*» [www.docker.com]
 - Similar to shipping containers: The container is always the same, regardless of the contents and thus fits on all trucks, cranes, ships, ...

Docker vs. Virtual Machine

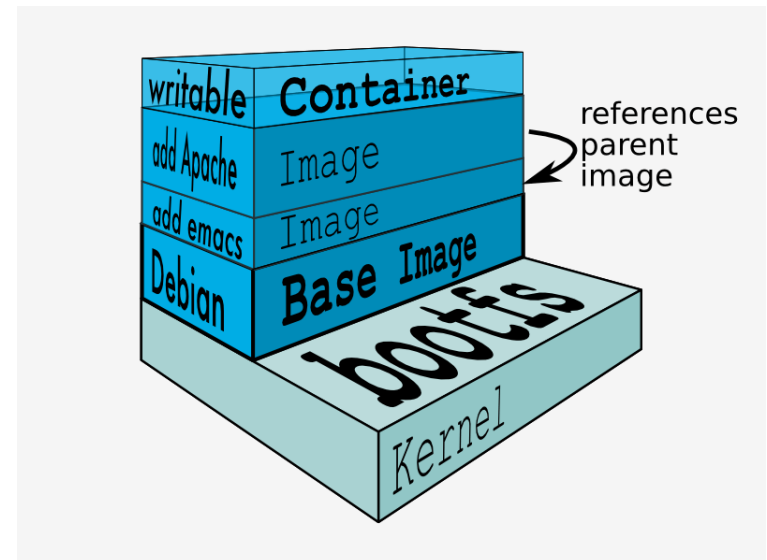


Source: <https://www.docker.com/resources/what-container>

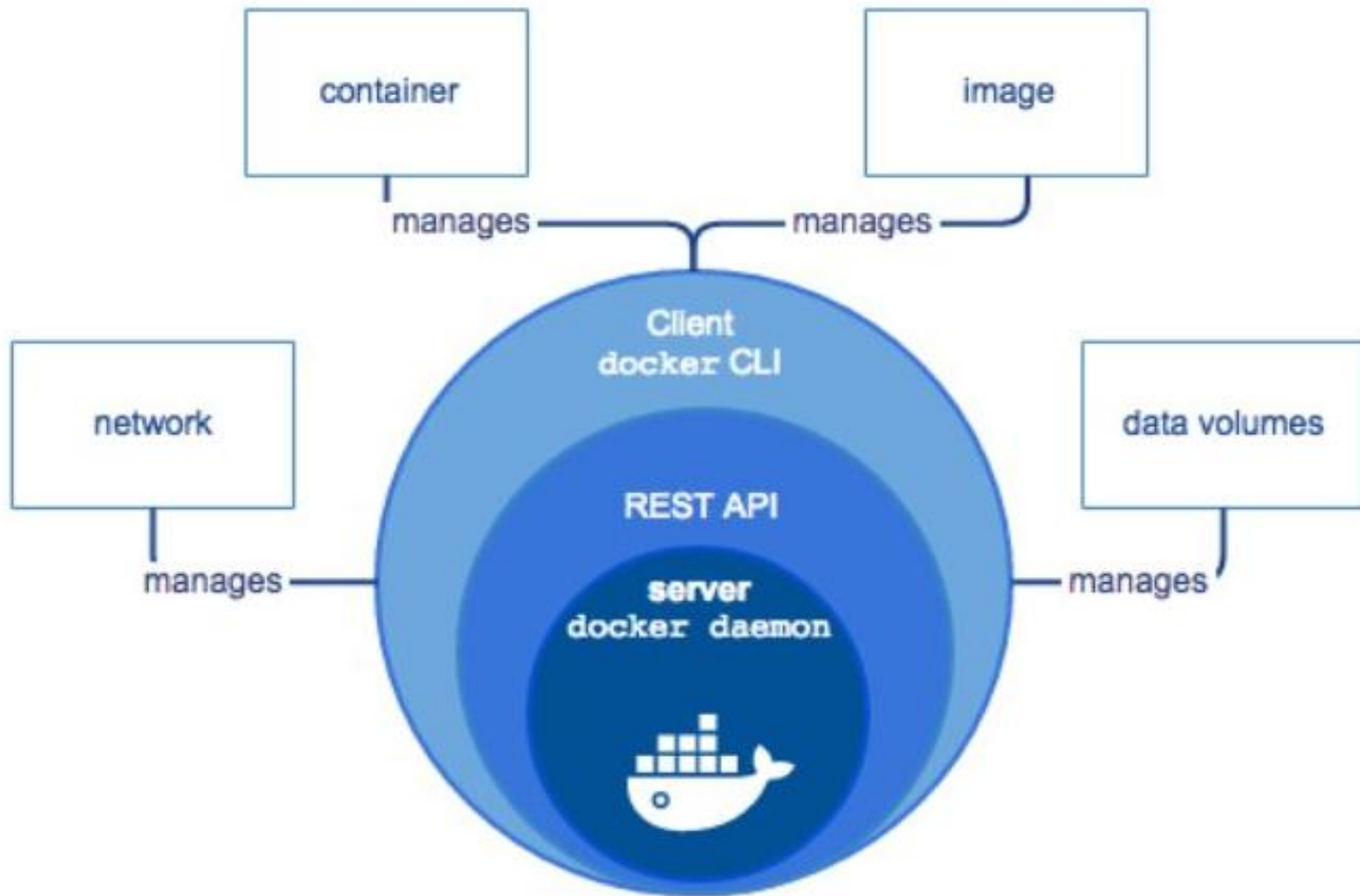
Docker Technology



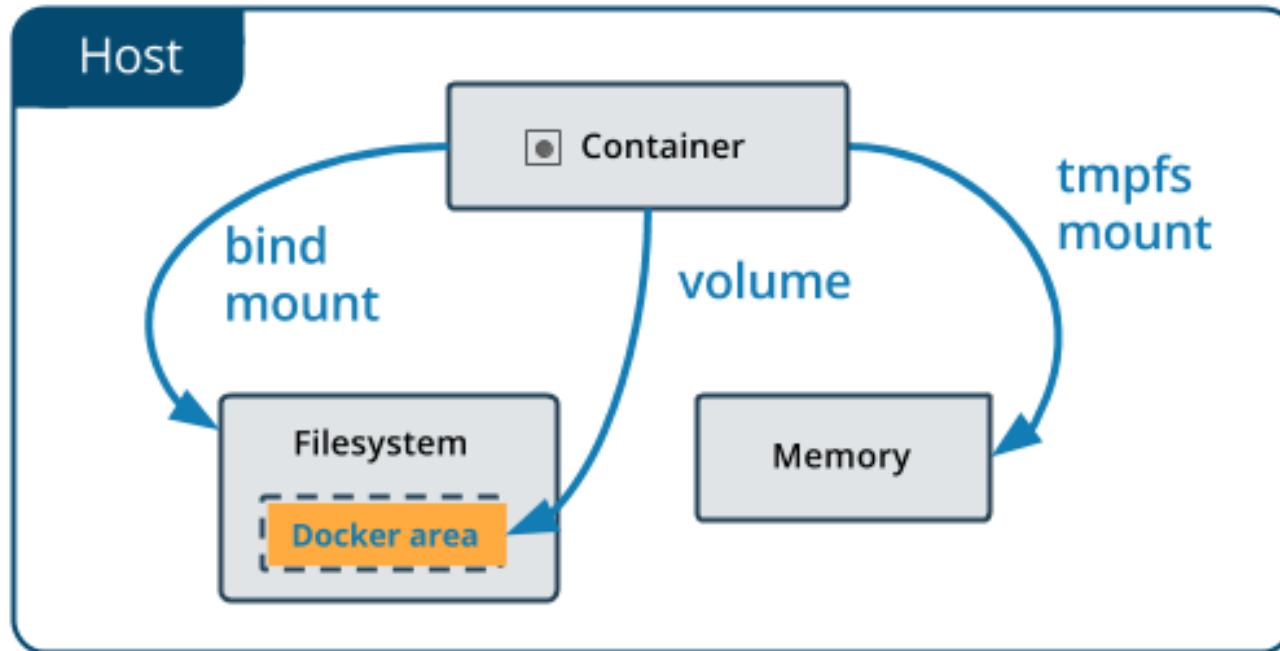
Docker Image



Docker Engine (1/3)

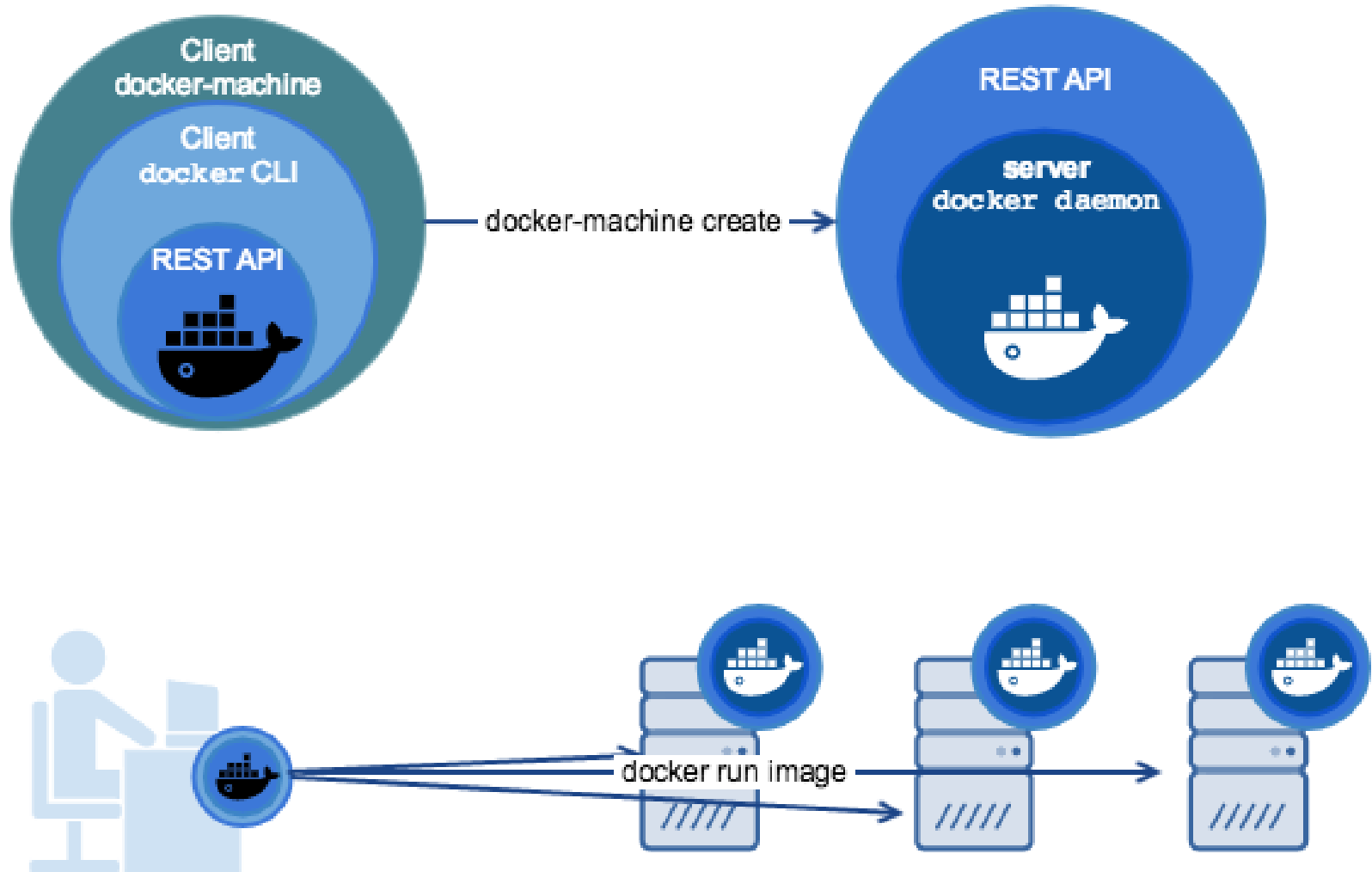


Docker Engine (2/3)



- **tmpfs mount** for non-persistent data
- A **volume** is managed by docker (writable layer)
- A bind mount depends on the directory structure of the host

Docker Engine (3/3)





- **Various Linux distributions (Ubuntu, Fedora, AlmaLinux, RHEL, openSUSE, ...)**
- **Windows Containers on Windows Nano Server**
- **Linux Containers on Windows Subsystem for Linux 2 (WSL2)**
- **Cloud (Amazon, Google, Microsoft, Rackspace)**

„Hello World“

Simple Command for an Ad-Hoc Container

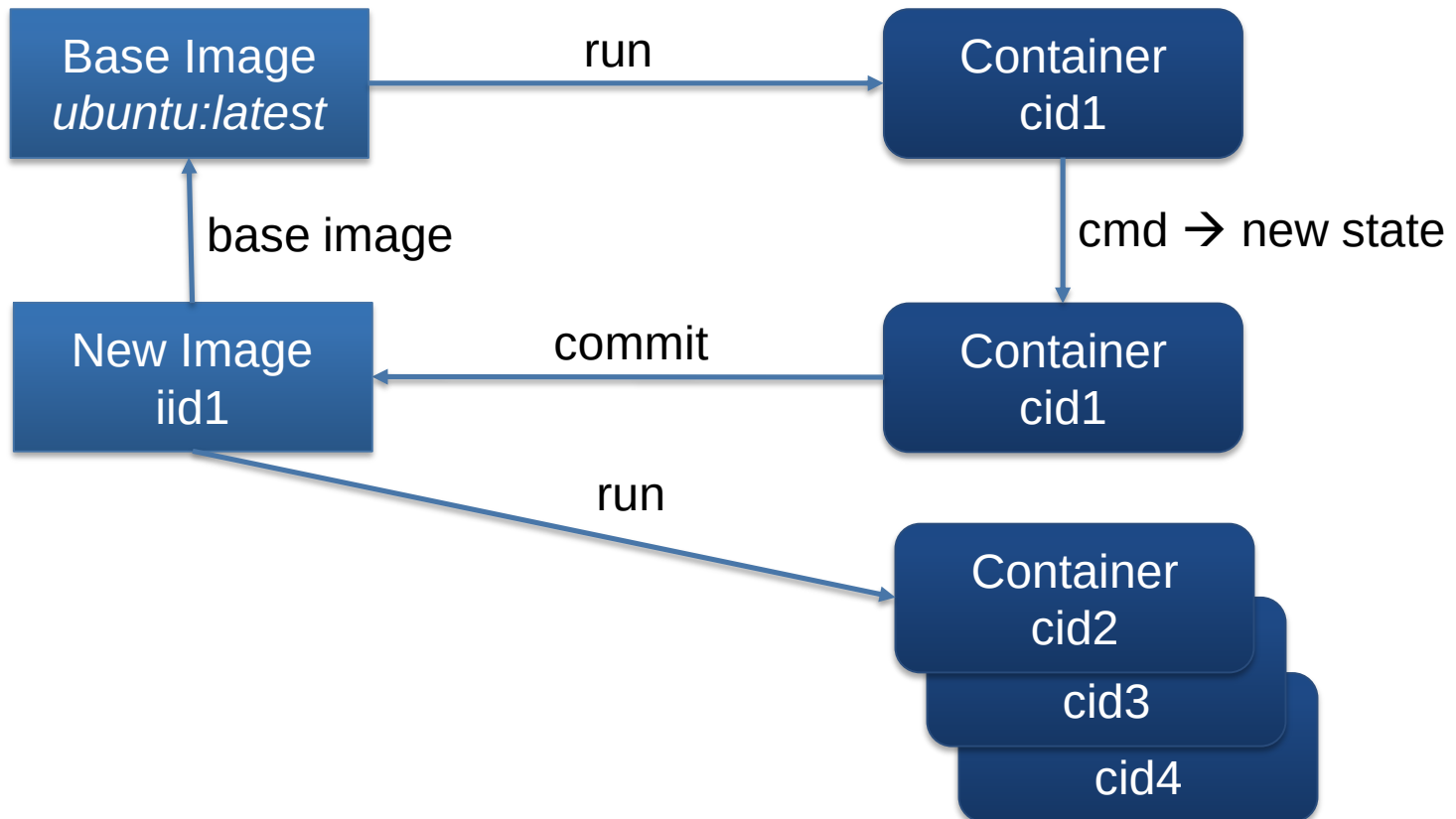
```
> docker run ubuntu echo Hello World
```

Docker Image

- **Persisted snapshot that can be run**
 - > **`docker images`** : List all local images
 - > **`docker run`** : Create a container from an image and execute a command in it
 - > **`docker tag`** : Tag an image
 - > **`docker pull`** : Download image from repository
 - > **`docker rmi`** : Delete a local image
 - This will also remove intermediate images if no longer used

- **Runnable instance of an image**
 - > **`docker start`** : Start a stopped container
 - > **`docker stop`** : Stop a running container
 - > **`docker pause`** : Pause all processes within a container
 - > **`docker rm`** : Delete a container
 - > **`docker commit`** : Create an image from a container

Image vs. Container



- Create images automatically using a build script: «Dockerfile»
- Can be versioned in a version control system like Git or SVN, along with all dependencies
- Docker Hub can automatically build images based on dockerfiles on Github

- **Dockerfile Example:**

```
FROM ubuntu
ENV DOCK_MESSAGE Hello My World
ADD dir /files
CMD ["bash", "someScript"]
```

> `docker build [DockerFileDir]`



- **Outgoing Connections**
 - Every container has unrestricted access to the Internet
- **Incoming Connections**
 - Ports must be explicitly opened («published»)
 - > `docker run -t -p 8080:5000 ubuntu nc -l 5000`
 - Map container port 5000 to host port 8080
 - Check on host: `nc localhost 8080`
 - Mapping can be restricted to specific host interface, e.g.
`-p 127.0.0.1:8080:5000`



- **Show all running containers (incl. stopped)**
 - > `docker ps -a`
- **Real-time resource usage statistics**
 - > `docker stats`
- **Disk space used by Docker daemon**
 - > `docker system df`
- **Low level information on Docker objects**
 - > `docker inspect [containerID]`

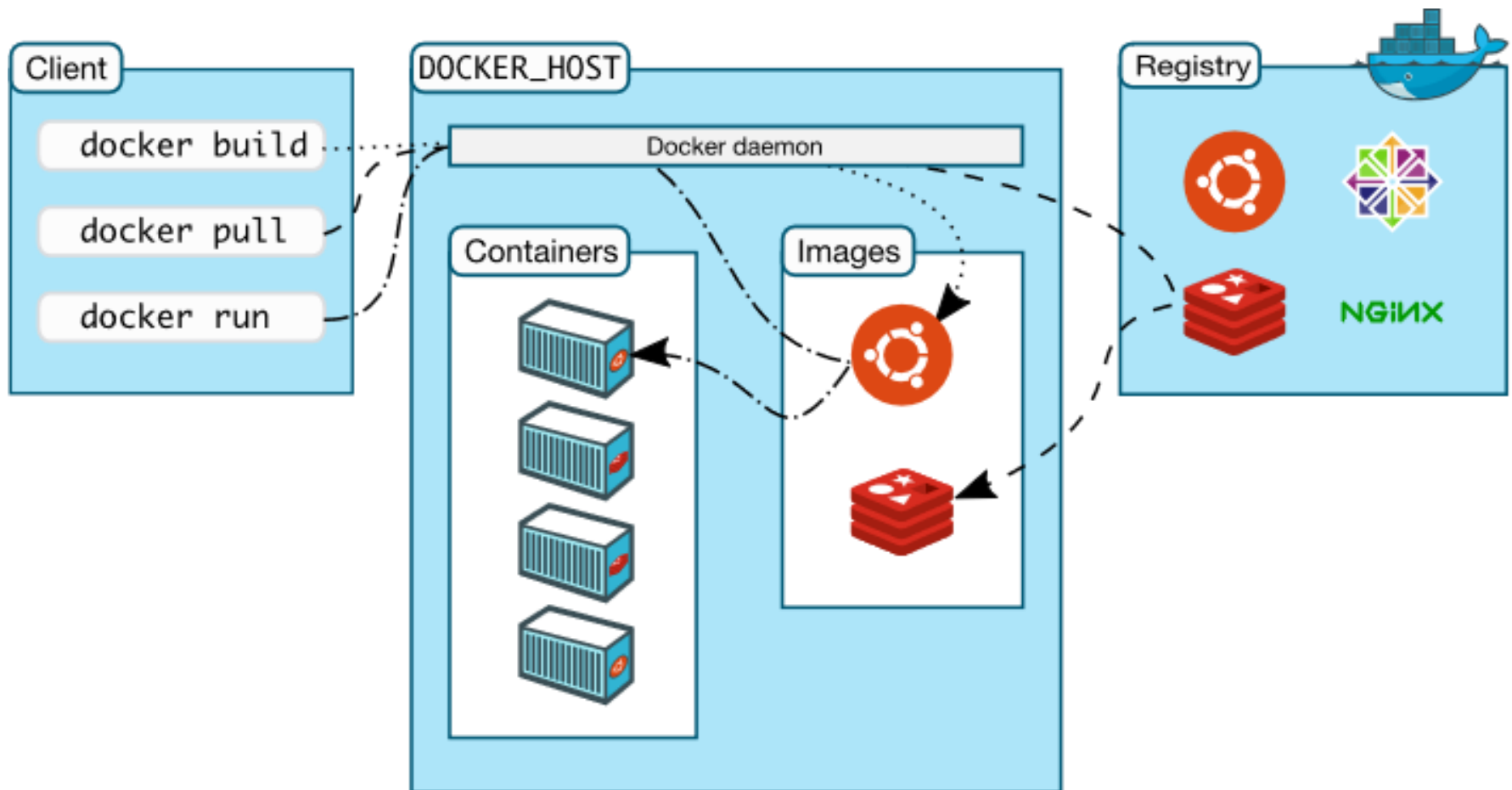


- **Docker Images: Docker Hub**
- **Docker Compose**
 - tool for running multi-container applications
 - define multiple services, link them together and run them
- **Google Kubernetes Engine (K8s)**
 - Platform for automating deployment, scaling, and operations of application containers across clusters of hosts
 - Available in Azure Kubernetes Service (AKS), Amazon Elastic Kubernetes Service (EKS)



- **Public repository of Docker images**
 - <https://hub.docker.com/>
 - > `docker search [term]`
- **Automated: Has been automatically built from Dockerfile**
 - Source for build is available on GitHub

Distributed Microservices Architecture





Datei `docker-compose.yaml`

- Container configurations are written in a single YAML file
- Applications are easier to build and scale
- Composition of individual applications is called a "Compose Project"
- Compose project is started by
> `docker compose up -d`

```
services:
  nginx:
    image: nginx:latest
    ports:
      - 9999:80
    links:
      - db
    volumes:
      - /var/www/html:/usr/share/nginx/html
    restart: always
  db:
    image: postgres:latest
    environment:
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres
    ports:
      - 5432:5432
    restart: always
```



- **Development Environment**
- **Environments for Integration Tests**
- **Quick evaluation of software**
- **Microservices**
- **Multi-Tenancy**
- **Unified execution environment**
dev → test → prod (local, VM, cloud, ...)

2. Cloud Computing

2.1 Grundlagen des Cloud Computing

2.2 Microsoft Azure

2.3 Infrastructure as a Service (IaaS)

2.4 Container as a Service (CaaS)

2.4.1 Docker

2.4.2 Kubernetes

2.5 Platform as a Service (PaaS)

2.6 IoT Hub

2.7 Microsoft Entra ID

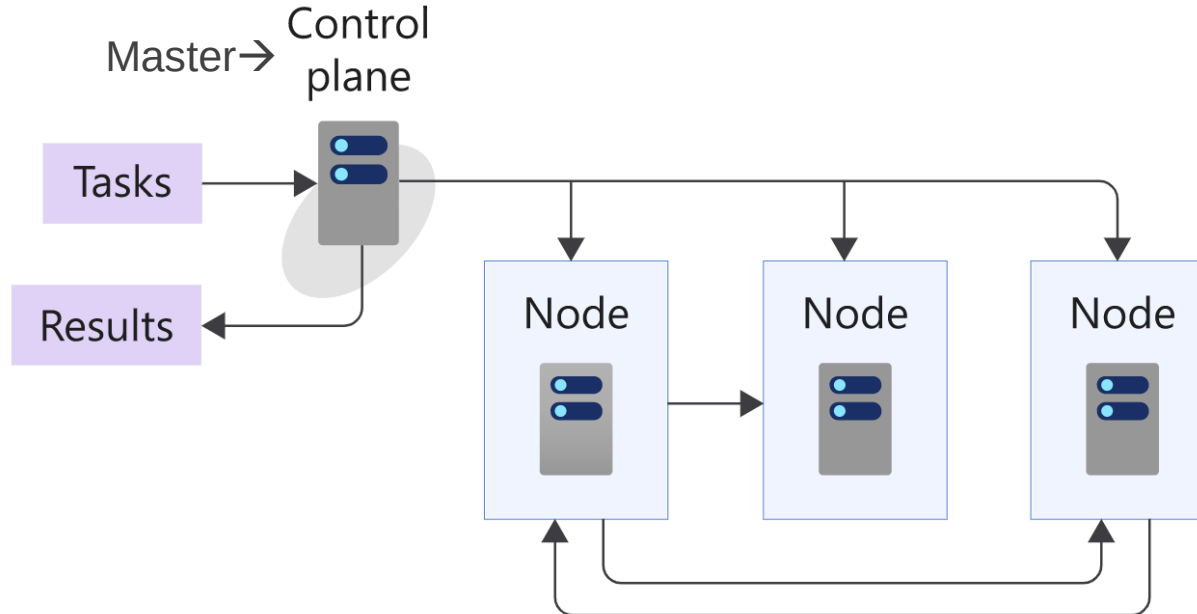
- **What is Kubernetes?**
- **System Architecture**
- **Deployment**
- **Where to go from here**

What is Kubernetes?

- Kubernetes (K8s) is Greek for "helmsman", your guide through unknown waters, nice but not true :-)
- Designed and announced by Google in mid 2014, written in Go and open source. Maintained by the Cloud Native Computing Foundation (CNCF) since 2015.
- Kubernetes is a platform and container orchestration tool for automating deployment, scaling, updating and operations of application containers.
- Kubernetes supports Docker, Containerd, CRI-O, Kata and Virtlet



- A Kubernetes cluster is a set of computers configured to work together. The computers in the cluster are called nodes
- Nodes are controlled by computers in the control plane that schedule tasks on the nodes



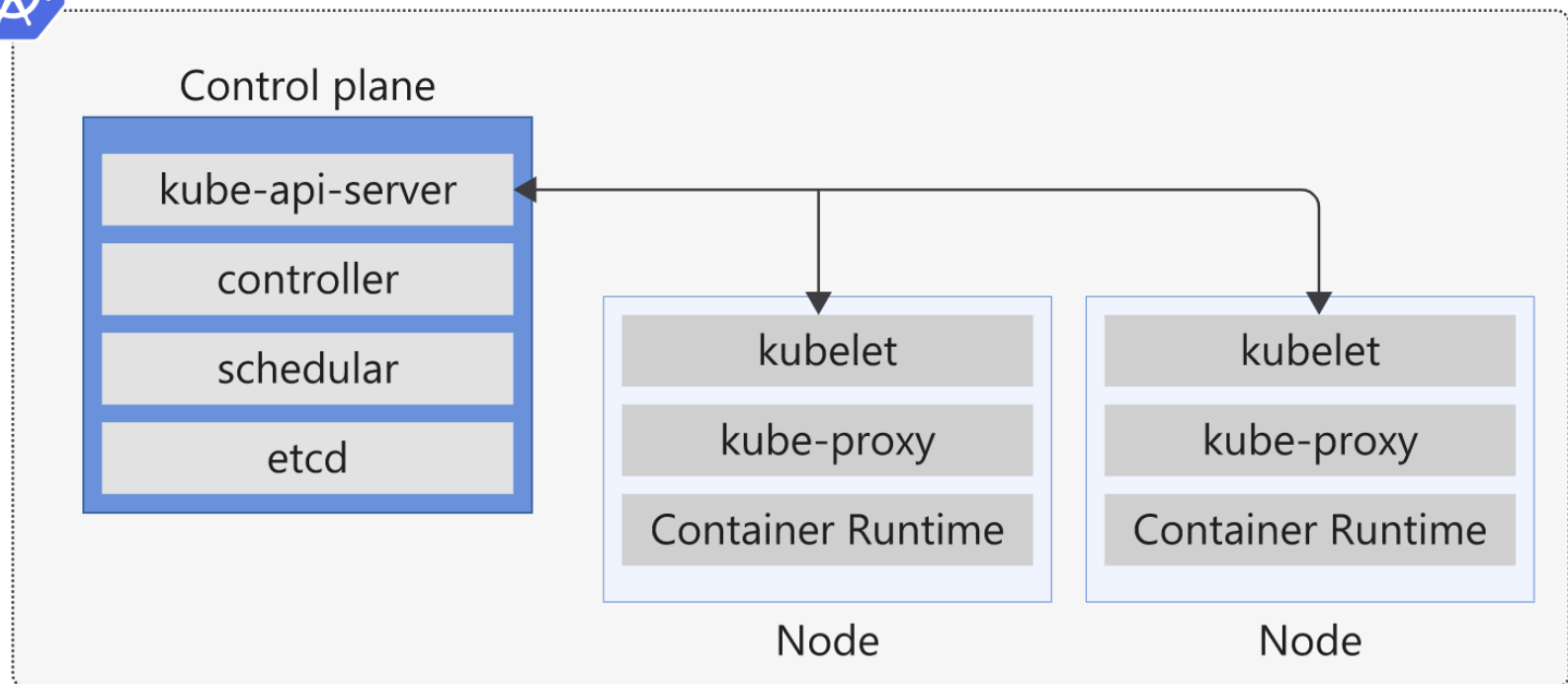
Kubernetes Architecture



- A Kubernetes cluster contains at least one master node in the control planes and one or more worker nodes
- Both the control planes and node instances can be physical devices, virtual machines, or instances in the cloud.



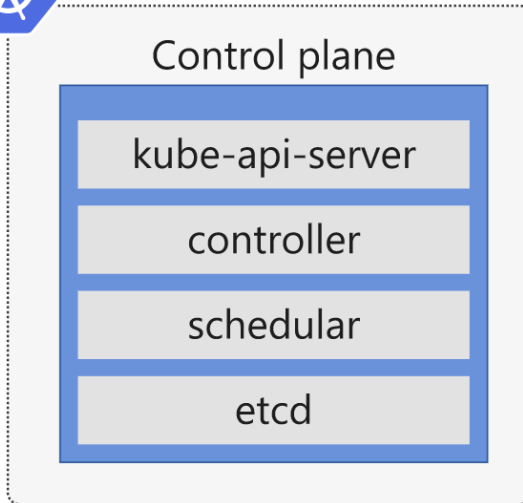
Kubernetes Cluster



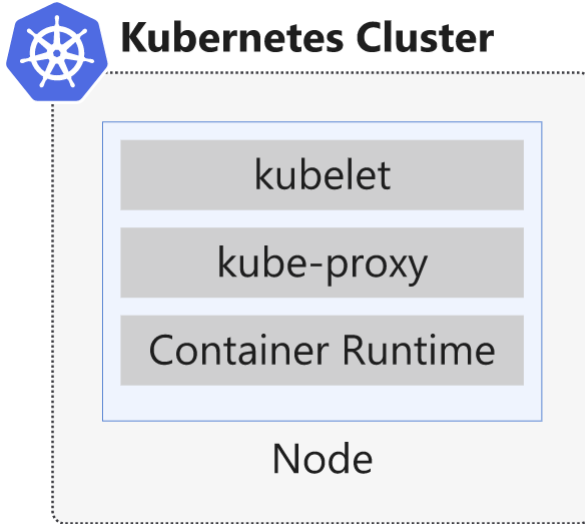
Control Plane



Kubernetes Cluster

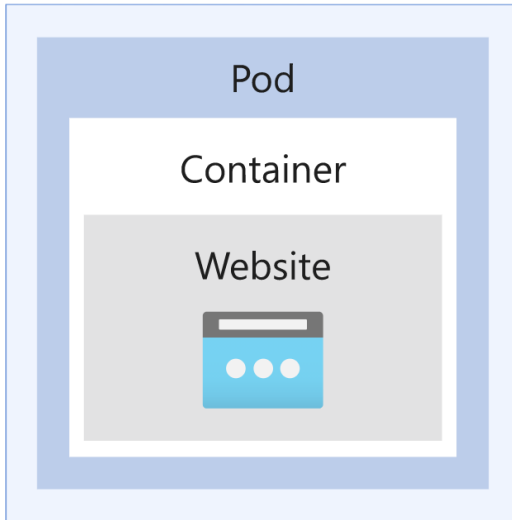


- **kube-api-server** is the front end to the control plane. As a user, you can use it by applications like **kubectl**
 - The key-value database called **etcd** is used to save the configuration of a Kubernetes cluster.
-
- **Controllers** monitor nodes, containers, and endpoints. A controller will take action to ensure the wanted state in the cluster
 - The **scheduler** assigns newly created containers to nodes



- The `kubelet` monitors the node and makes sure that the containers scheduled on each node run as expected
- The `kube-proxy` component is responsible for local cluster networking and makes sure that each node has a unique IP address
- The container runtime is the underlying software that runs containers on a Kubernetes cluster. The runtime is responsible for fetching, starting, and stopping container images. The default container runtime in Azure Kubernetes Service is `containerd`.

Kubernetes Node



- **The workloads in Kubernetes are containerized applications.**
- **Unlike in a Docker environment, containers do not run directly on Kubernetes. They are packaged into a Kubernetes object called a pod. A pod is the smallest object that can be created in Kubernetes.**
- **A pod typically runs either a single container or a group of containers of different applications**
- **Pod templates are used to define the information about the pods that run in a cluster.**

- Kubernetes deployment is an abstraction layer for the pods
- Pods are the basic units that get deployed in the cluster
- A deployment configuration can be of YAML (preferred) or JSON format.
- YAML, which stands for Yet Another Markup Language, is a human-readable text-based format for specifying configuration-type information
- YAML files can be passed to Kubernetes using the `kubectl` CLI, e.g. `kubectl apply -f whoami.yaml`

YAML Lists and Maps

- Maps associate name-value pairs, eg:

```
apiVersion: v1
```

```
kind: Pod
```

- Lists can have virtually any number of items in a list, which is defined as items that start with a dash (-) indented from the parent, eg:

```
containers:
```

```
- name: front-end
```

```
  image: nginx
```

```
- name: rss-reader
```

```
  image: nickchase/rss-php-nginx:v1
```

Example of YAML Deployment



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fitness-app-deployment
spec:
  selector:
    matchLabels:
      app: myfitnessapp-pod
  replicas: 2
  template:
    metadata:
      labels:
        app: myfitnessapp-pod
    spec:
      containers:
        - name: myfitnessapp-container
          image: hsf.it.azurecr.io/fitnessapp:west
          ports:
            - containerPort: 80
```

Kubernetes API version

Deployment name

Desired state

Selector label for pod spec

Number of replicas

Selector label for pod mgmt

Container name

Container image

Internal port

für Updates:
spec.
replicas 3
Strategy.
type. Rolling
Update

YAML

Service Definition



```
apiVersion: v1
kind: Service
metadata:
  name: fitness-service
spec:
  selector:
    app: myfitnessapp-pod
  ports:
    - port: 8080
      targetPort: 80
  type: LoadBalancers
```

- To expose a deployment with one or many Pods externally a Kubernetes Service endpoint needs to be mapped to the deployment using labels and selectors.
- The set of pods that constitute a service are defined by a label selector, here `myfitnessapp-pod`
- The internal port 80 is mapped externally to 8080
- The replicated pods are load balanced

- **Kubernetes is the industry-leading container orchestrator**
- **Kubernetes provides auto-placement, auto-restart, auto-replication and auto-scaling of containers**
- **With Kubernetes it is possible to manage containers across public, private, hybrid and multi-cloud**
- **Kubernetes APIs are continuously extended**
- **Kubernetes has grown very complex over the years**



- Tutorial “Learn Kubernetes Basics” using Minikube:
<https://kubernetes.io/docs/tutorials/kubernetes-basics/>
- „Lightweight Kubernetes“ K3s von SUSE, optimiert für Edge- und IoT-Umgebungen: <https://docs.k3s.io/>
- Canonical MicroK8s - Another Mini-Kubernetes:
<https://learn.microsoft.com/en-us/training/modules/intro-to-kubernetes/5-exercise-kubernetes-functionality>
- Deploy a containerized application on Azure Kubernetes Service (AKS): <https://learn.microsoft.com/en-us/training/modules/aks-deploy-container-app/>



2.1 Grundlagen des Cloud Computing

2.2 Microsoft Azure

2.3 Infrastructure as a Service (IaaS)

2.4 Container as a Service (CaaS)

2.5 Platform as a Service (PaaS)

2.6 IoT Hub

2.7 Microsoft Entra ID



- **Folien sind separat in Moodle verfügbar**



2.1 Grundlagen des Cloud Computing

2.2 Microsoft Azure

2.3 Infrastructure as a Service (IaaS)

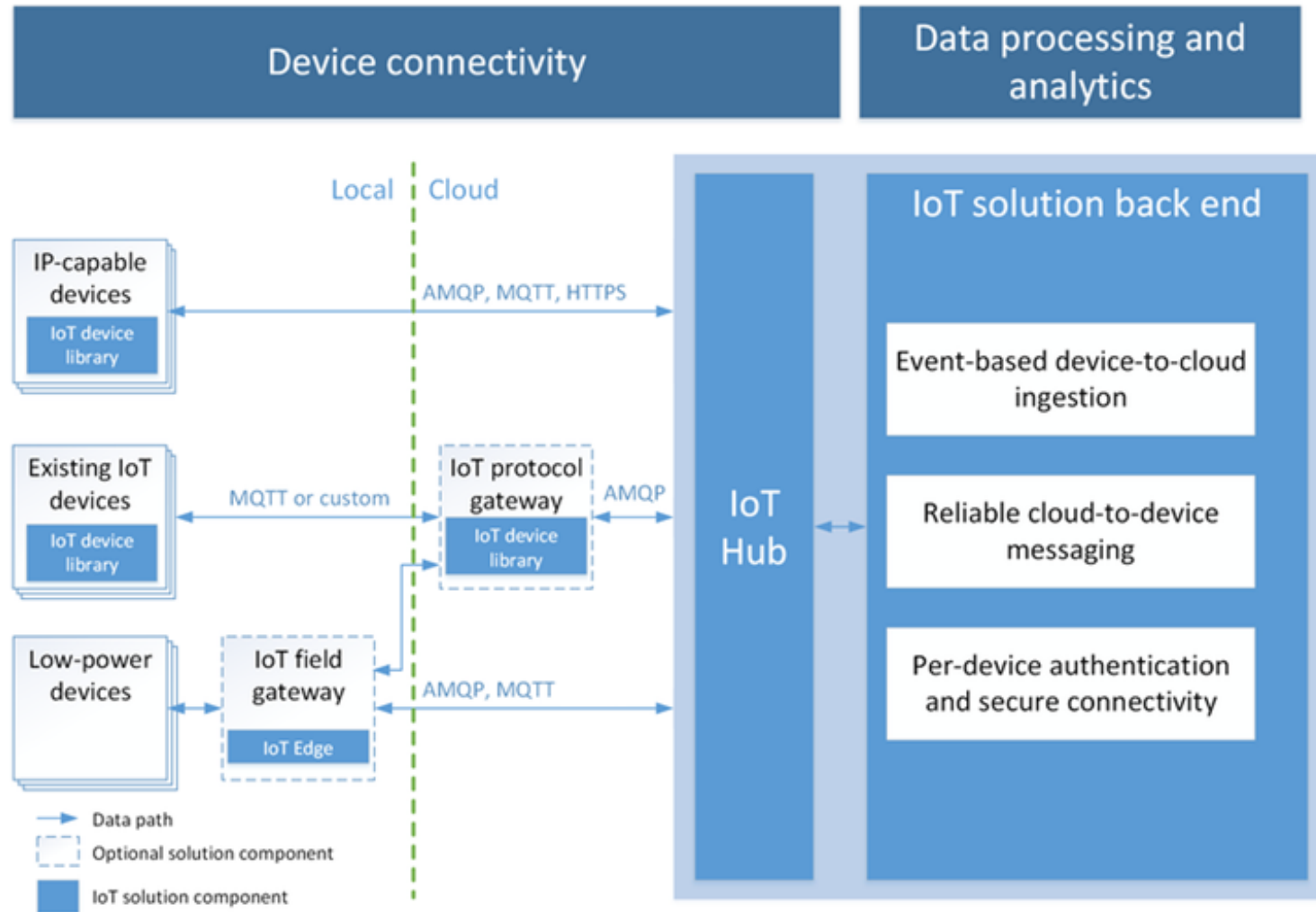
2.4 Container as a Service (CaaS)

2.5 Platform as a Service (PaaS)

2.6 IoT Hub

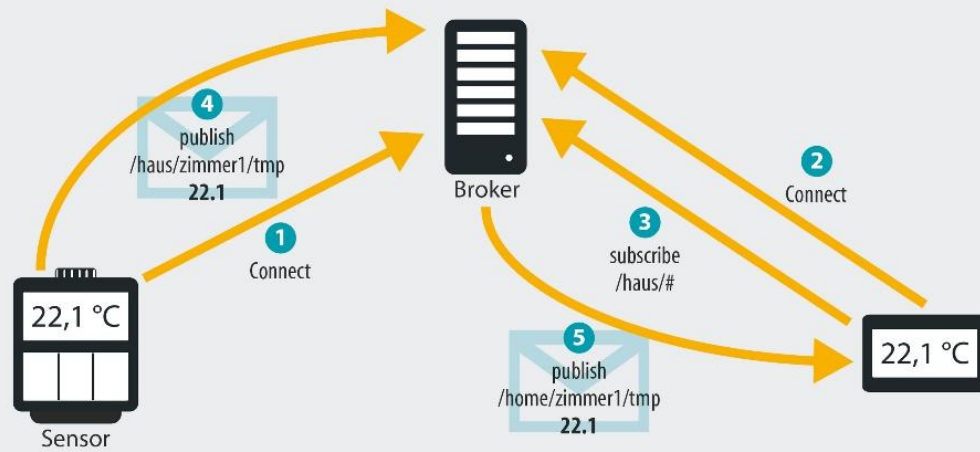
2.7 Microsoft Entra ID

Azure IoT Hub Overview



Das Protokoll MQTT

Die gesamte Kommunikation über MQTT erfolgt über den Broker. Die Endgeräte sehen sich untereinander nicht.



- **Azure IoT Hub erscheint für ein Device wie ein MQTT Broker**

- **Clients senden dem Server (“Broker”) nach Verbindungsaufbau Nachrichten mit einem Topic, welches die Nachricht hierarchisch einstuft**
- **Clients können diese Topics abonnieren, wobei der Server die empfangenen Nachrichten an die entsprechenden Abonnenten weiterleitet**



IoT Hub Pricing



Region:

Germany Central

Currency:

Euro (€)

Pricing

– IoT Hub

Connect your IoT devices and build your IoT solution to turn data into business intelligence.

Standard tier

EDITION TYPE	PRICE PER IOT HUB UNIT (PER MONTH)	TOTAL NUMBER OF MESSAGES/DAY PER IOT HUB UNIT	MESSAGE METER SIZE
Free	Free	8,000	0.5 KB
S1	€25.299	400,000	4 KB
S2	€252.99	6,000,000	4 KB
S3	€2,529.90	300,000,000	4 KB



2.1 Grundlagen des Cloud Computing

2.2 Microsoft Azure

2.3 Infrastructure as a Service (IaaS)

2.4 Container as a Service (CaaS)

2.5 Platform as a Service (PaaS)

2.6 IoT Hub

2.7 Microsoft Entra ID

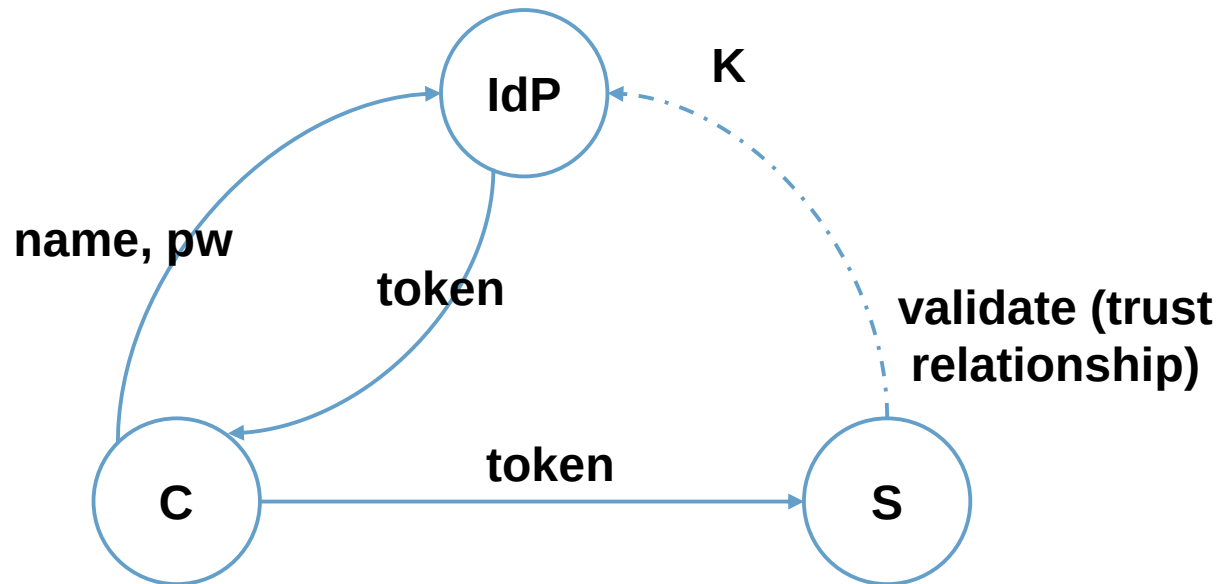


Security Principals

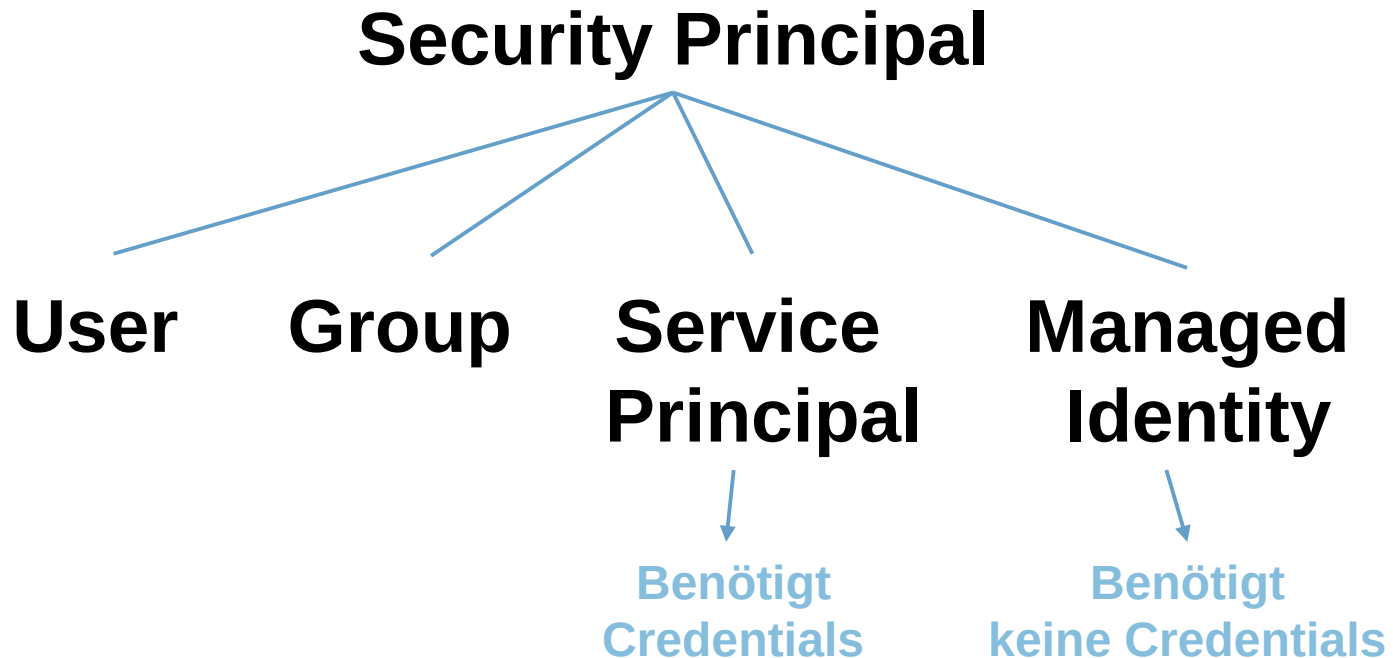
Demo im Azure Portal

Role-based Access Control (RBAC) Identity and Access Management (IAM)

Basic Concepts of (modern) Authentication



IdP: Identity Provider
C: Client, S: Server
pw: password
K: signing key oder public key



**Service Principal und Managed Identity
repräsentieren Azure Resources in Entra ID**



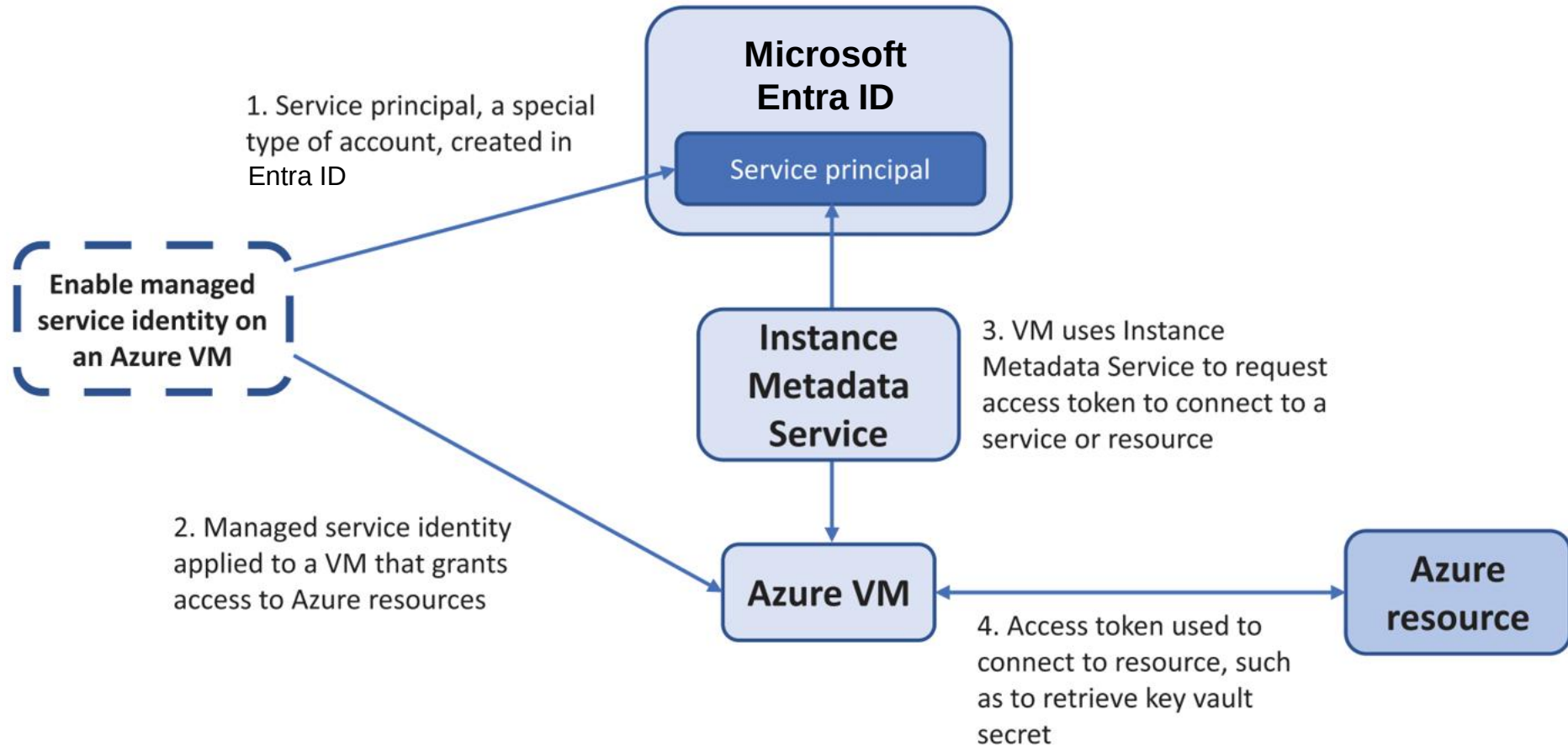
Managed Identity



App Service

Ziel: Verhindern, dass Entwickler Credentials (z.B. im Connection String für SQL DB) im Code oder in Konfigurationsdateien schreiben müssen bzw. mit Credentials umgehen müssen

Managed Identity Token Authentication



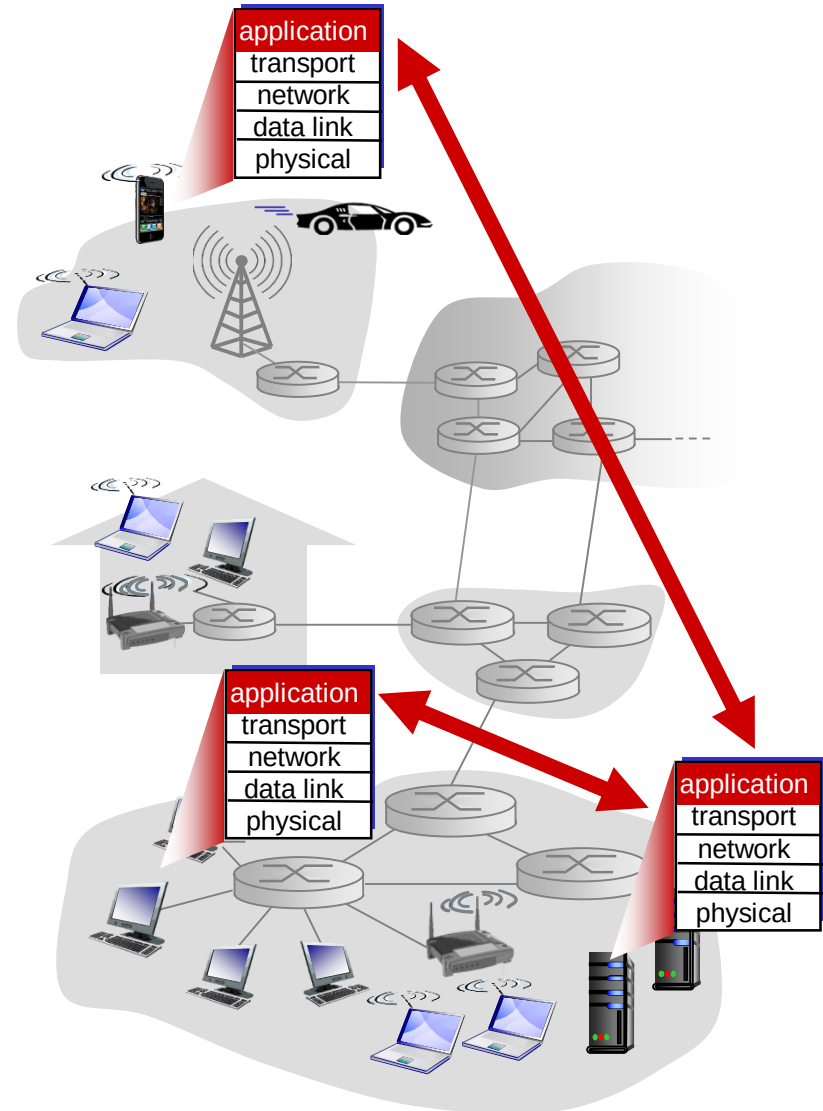


1. Einführung

2. Cloud Computing

3. Middleware Layer

- **Anwendung: kommunizierende, verteilte Prozesse**
 - Prozesse laufen auf Netzwerk Hosts im “user space”
 - Austausch von Nachrichten, um Anwendung zu realisieren
 - z.B. Smart Home, Autonomous Driving, Smart Grid
- **Anwendungsprotokoll:**
 - ein Teil der Anwendung
 - definiert PDUs und lokale Prozeduren
 - nutzt Dienste der darunterliegenden Schichten





API: Application Programming Interface

- Definition eines Interfaces zwischen Anwendungen und den Schichten des Protokollstapels

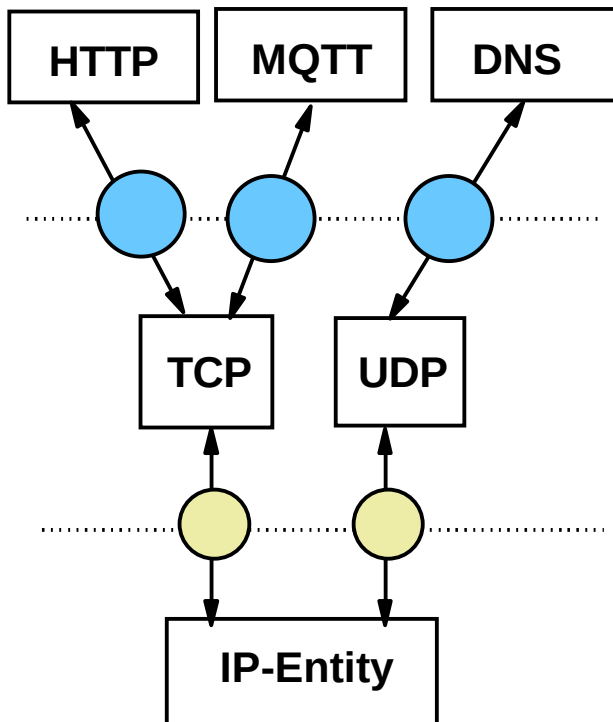
Beispiele:

- Namespace `System.Net` in der .NET FCL: API für den clientseitigen Zugriff auf das HTTP-Protokoll
- **ASP.NET** (Active Server Pages .NET): API für das Erstellen von Web – Anwendungen mit dem Microsoft Internet Information Server (IIS) oder dem Apache-Server (mit ASP.NET Core)
- **Socket API**: Generisches API für den Zugriff auf die Transportschicht oder die Netzwerkschicht



Application	Application layer protocol	Transport layer protocol
DNS	dns [RFC 7858]	UDP, TCP
remote terminal	ssh [RFC 4251]	TCP
WWW, IoT	http [RFC 9110]	TCP
streaming multimedia	rtp [RFC 3550] http [RFC 9110]	UDP, TCP
e-mail	smtp [RFC 5321]	TCP
IoT	MQTT5, AMQP [OASIS] CoAP [RFC 7252]	TCP

Frage: Wie “identifiziert” ein Prozess eindeutig einen anderen Prozess, mit dem er kommunizieren möchte?



Port Number im TCP / UDP Header

⇒ **MQTT / HTTP / DNS ...**

z.B. MQTT 1883/TCP

HTTPS 443/TCP

DNS 53/UDP

“well-known ports”

Protocol Field im IP Header

⇒ **UDP / TCP / ...**

IP-Adresse

z.B. 9.7.54.124

„Vollqualifizierte“ (fully qualified) Adresse einer Endanwendung ist
Port Number + Protocol Number + IP-Adresse



Syntax:

<Protokoll>:// <Systemadresse> : <Port>

Beispiele:

tcp://141.19.96.200:4000

https://www.swt.th-mannheim.de



23.06.

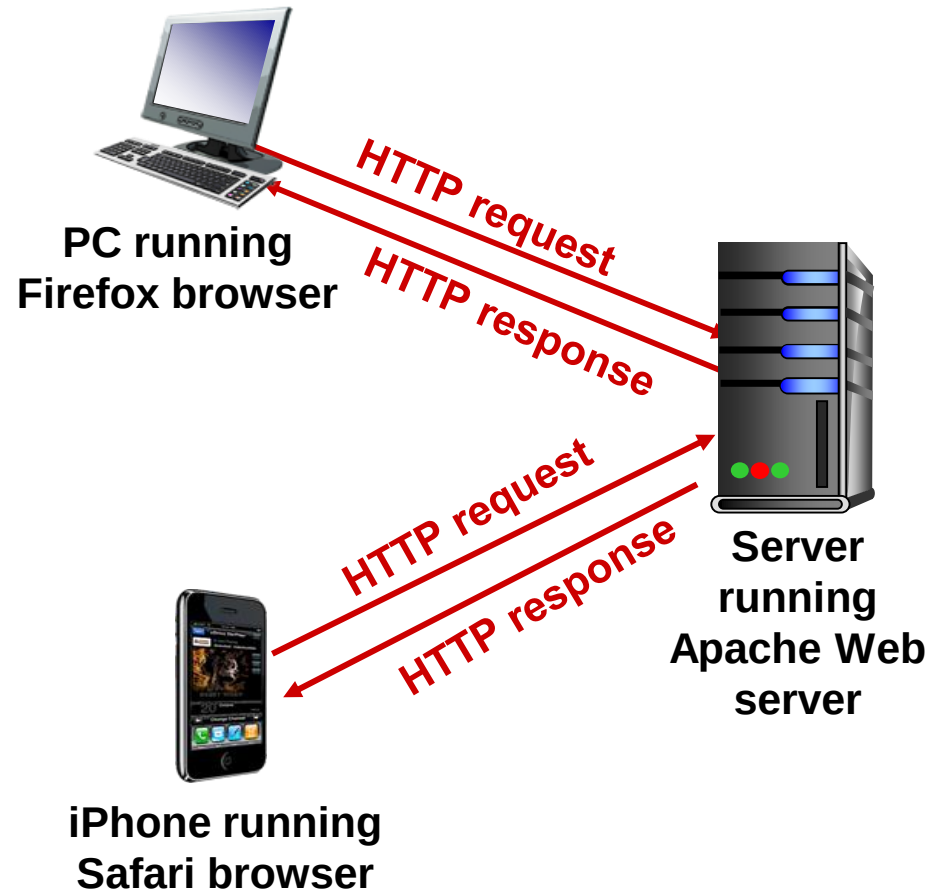
3.1 Hypertext Transfer Protocol (HTTP)

3.2 Domain Name System (DNS)

3.3 Distributed Hash Table (DHT)

HTTP: HyperText Transfer Protocol

- “objektorientiertes” Anwendungsprotokoll des WWW gemäß Client/Server-Modell
 - **Client:** Browser sendet eine Anforderung
 - **Server:** Webserver sendet Webobjekte als Antwort auf Anforderungen
- HTTP1.1: RFC 9110-9112 (2022)
- HTTP2.0: RFC 9113 (2022)
- HTTP3.0: RFC 9114 (2022)



Ursprung: HTTP 1.0 (1/2)



Nutzer gibt URL <http://www.swt.th-mannheim.de> ein (umfasst Text und Referenzen auf 5 jpeg Bilder)

1a. HTTP-Client initiiert TCP-Verbindung zum HTTP Server- Prozess auf `www.swt.th-mannheim.de`. Port 80 ist Standard für http Server.

2. HTTP-Client sendet *HTTP Request PDU* mit URL in den TCP Socket

1b. HTTP-Server www.swt.th-mannheim.de erwartet TCP-Anforderung auf Port 80, “akzeptiert” Verbindung, bestätigt es dem Client.

3. HTTP-Server empfängt *Request PDU* und konstruiert *Response PDU* mit dem angeforderten Objekt (`index.htm`), sendet PDU in den Socket

Zeit
↓

Ursprung: HTTP 1.0 (2/2)



4. HTTP-Server schließt TCP-Verbindung.

5. HTTP Client empfängt *Response PDU* mit der **HTML**-Datei, Browser interpretiert **HTML**. Beim Parsen der **HTML-Datei** werden 5 referenzierte **jpeg**-Objekte gefunden

6. Schritte 1-5 werden für jedes der 5 **jpeg**-Objekte wiederholt

Zeit
↓

Verbindungsmanagement

- **Bei HTTP1.0 - nicht-persistente Verbindungen:** In jeder TCP-Verbindung wird nur ein Objekt übertragen. Browser erzeugen mehrere TCP-Verbindungen *parallel*.
- **Neu in HTTP1.1 - persistente Verbindungen:** Mehrere Objekte werden innerhalb einer einzigen TCP-Verbindung transferiert.



- **Es müssen viele TCP-Verbindungen auf- und abgebaut werden**
 - Netzwerkoverhead und Latenzzeiten!
- **Firewalls werden enorm belastet, da sie jede TCP-Verbindung prüfen müssen**
- **Riesiger Overhead bei Verschlüsselung (https), da die TLS Session an jede neue Verbindung übergeben werden muss**
- **TCP-Verbindungen starten wegen Staukontrolle immer langsam (Slow Start)**
 - Besonders ungünstig bei kleinen Webobjekten

Abschätzung der Antwortzeit bei HTTP

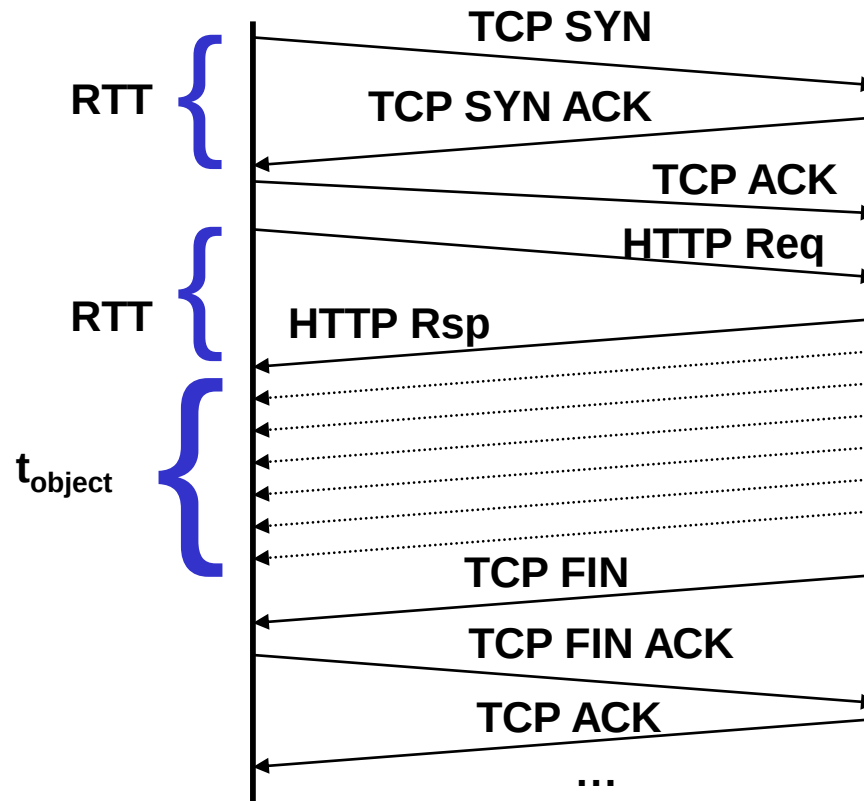


Gesucht: t_{download} : Zeit zum Empfang einer Webseite mit n Objekten gleicher Größe

Geg.: RTT: Round Trip Time für eine kleine Nachricht

t_{object} : Zeit zum Senden eines Objekts

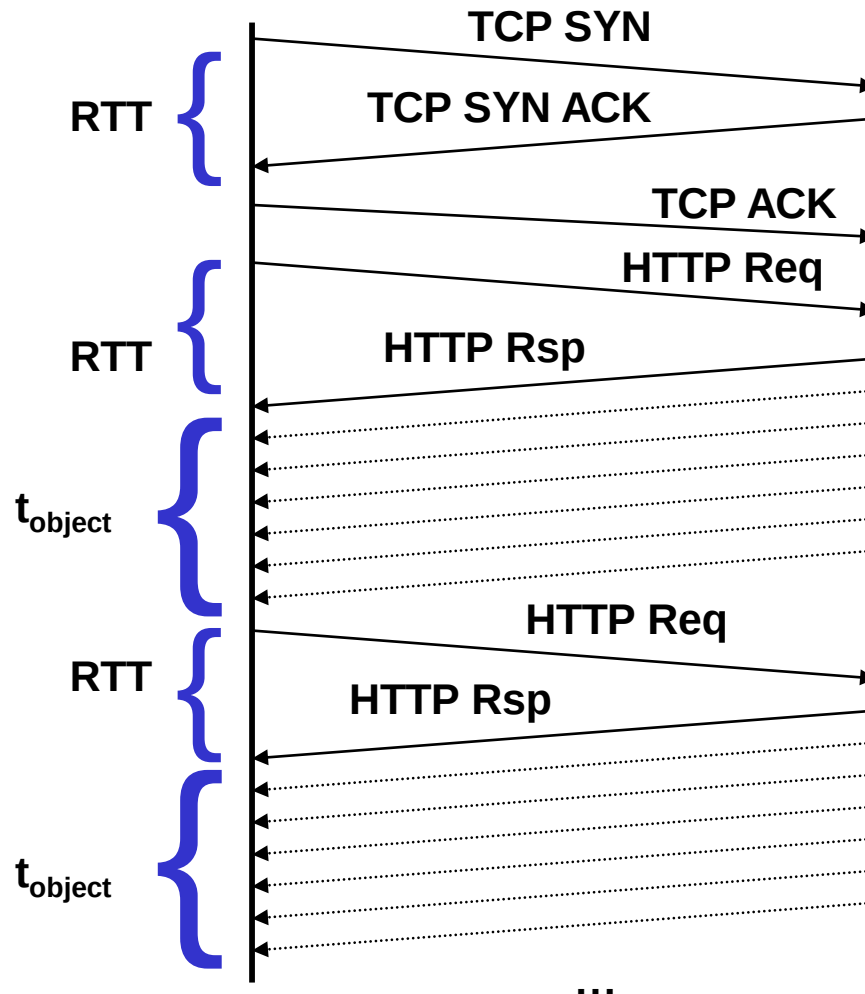
Nicht-persistente Verbindung



$$t_{\text{download}} = n (2 \text{ RTT} + t_{\text{object}})$$

Abschätzung der Antwortzeit bei HTTP

Persistente Verbindung (ohne Pipelining)



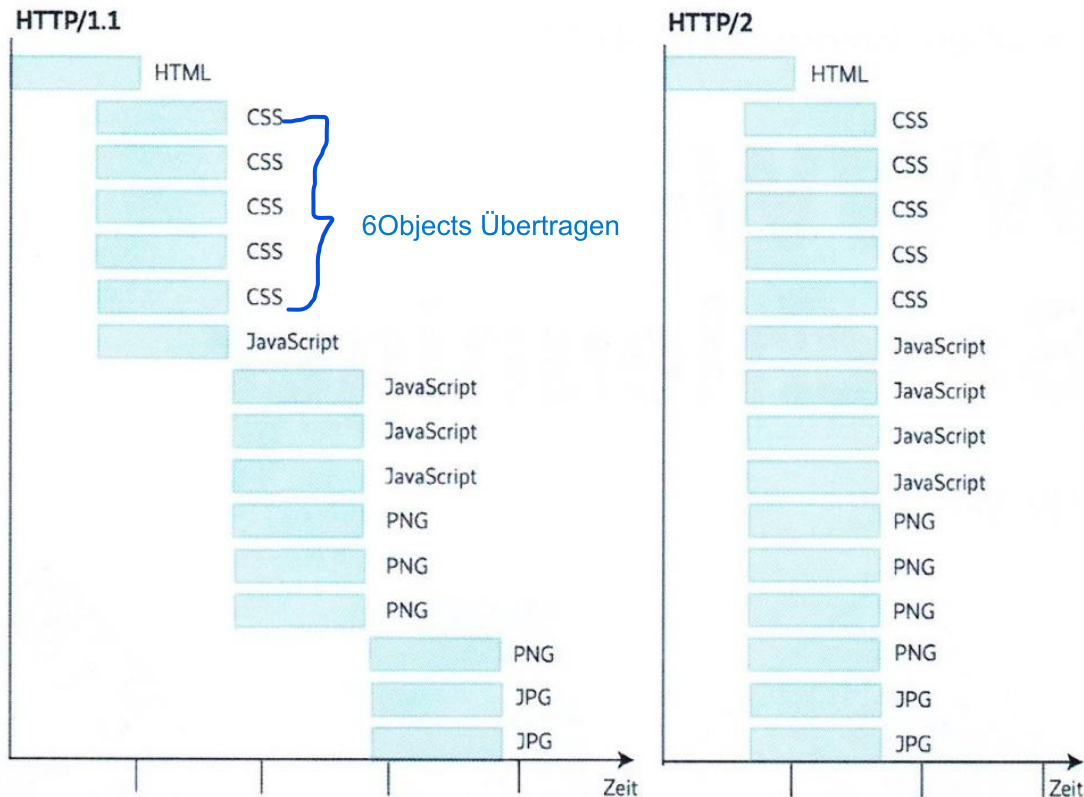
$$t_{\text{download}} =$$

$$\text{RTT} + n (\text{RTT} + t_{\text{object}})$$



- Es gibt nur eine einzige persistente TCP-Verbindung mit Pipelining zum Server!
- Über die Verbindung werden beliebig viele Webobjekte verschränkt („interleaved“) übertragen, jede Objektübertragung enthält eine Stream ID
- Sichere Verbindungen (https) bevorzugt, aber nicht zwingend
- Komprimierung der http-Header mit gzip
- Server Push, d.h. Objekte einer Webseite können gesendet werden, bevor sie angefordert wurden. Beispielsweise CSS- oder JavaScript-Dateien einer Webseite
Achtung: Server Push hat sich bisher als ineffizient erwiesen, da viele Dateien bereits im Browser Cache liegen
- Priorisierung von Streams, HTML und CSS zuerst

Performance bei HTTP/2

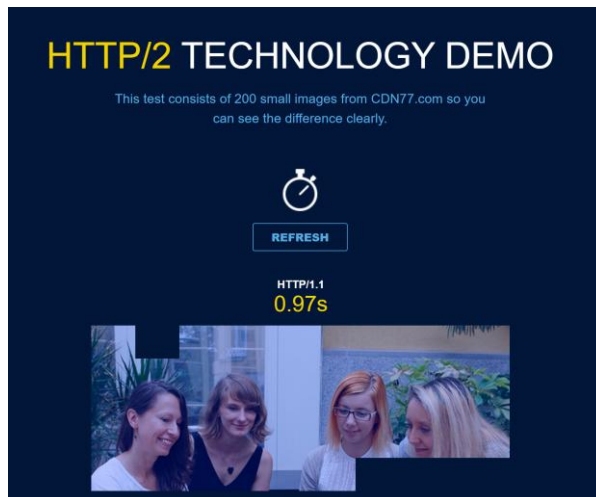


Das Wasserfalldiagramm zeigt, wie der Browser bei HTTP/1.1 nach dem Download der HTML-Seite sechs Requests parallel stellt. Die weiteren Requests werden so lange geblockt, bis diese beantwortet sind (Head of Line Blocking). Mit HTTP/2 kann der Browser alle Ressourcen gleichzeitig anfragen, sodass sich die Ladezeit verringert (Abb. 1).

HTTP2 Demos

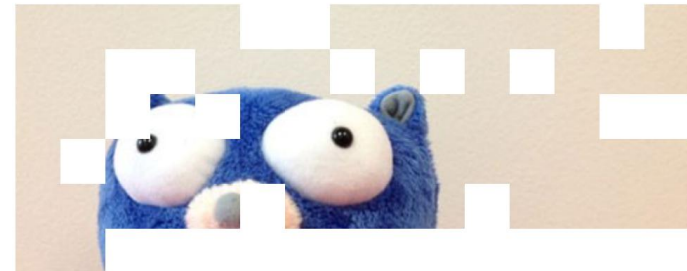
<http://www.http2demo.io/>

Achtung: Die Demo ist sehr vorteilhaft für HTTP2, da Hunderte kleine Kacheln übertragen werden.



A grid of 180 tiled images is below. Compare:

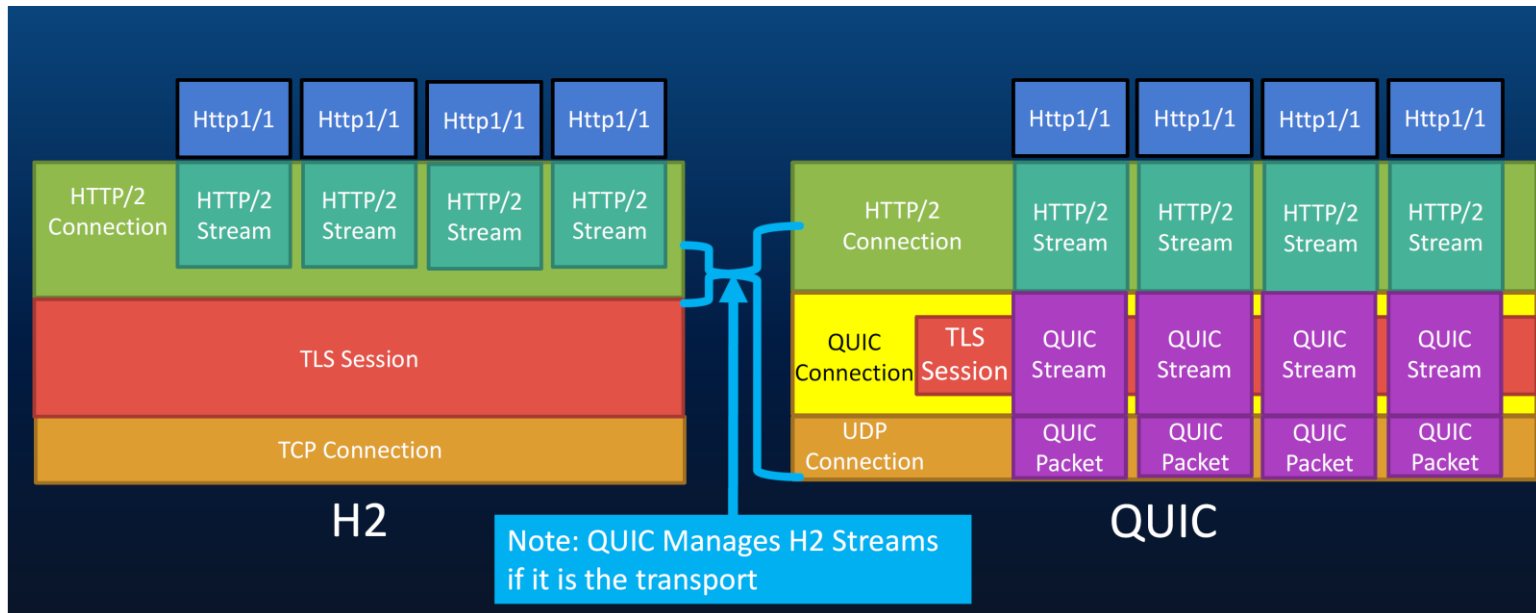
[\[HTTP/2, 0s latency\]](#) [\[HTTP/1, 0s latency\]](#)
[\[HTTP/2, 30ms latency\]](#) [\[HTTP/1, 30ms latency\]](#)
[\[HTTP/2, 200ms latency\]](#) [\[HTTP/1, 200ms latency\]](#)
[\[HTTP/2, 1s latency\]](#) [\[HTTP/1, 1s latency\]](#)



Server push support in Chrome entfernt:

<https://de.ubunlog.com/in-google-chrome-106-server-push-wird-nicht-mehr-unterst%C3%BCtzt/>

Evolution zu HTTP/3: HTTP/2 und QUIC



QUIC: QUICK UDP INTERNET CONNECTIONS

- TCP-like congestion control, loss recovery
- Integrates TLS (less protocol overhead)

Format der HTTP Request PDU

- Zwei HTTP PDU-Typen: **Request, Response**
- **HTTP Request PDU:**
 - erweiterter ASCII-Code ISO 8859-1 (human-readable format) wird als Zeichensatz verwendet

request line
(GET, POST,
HEAD commands)

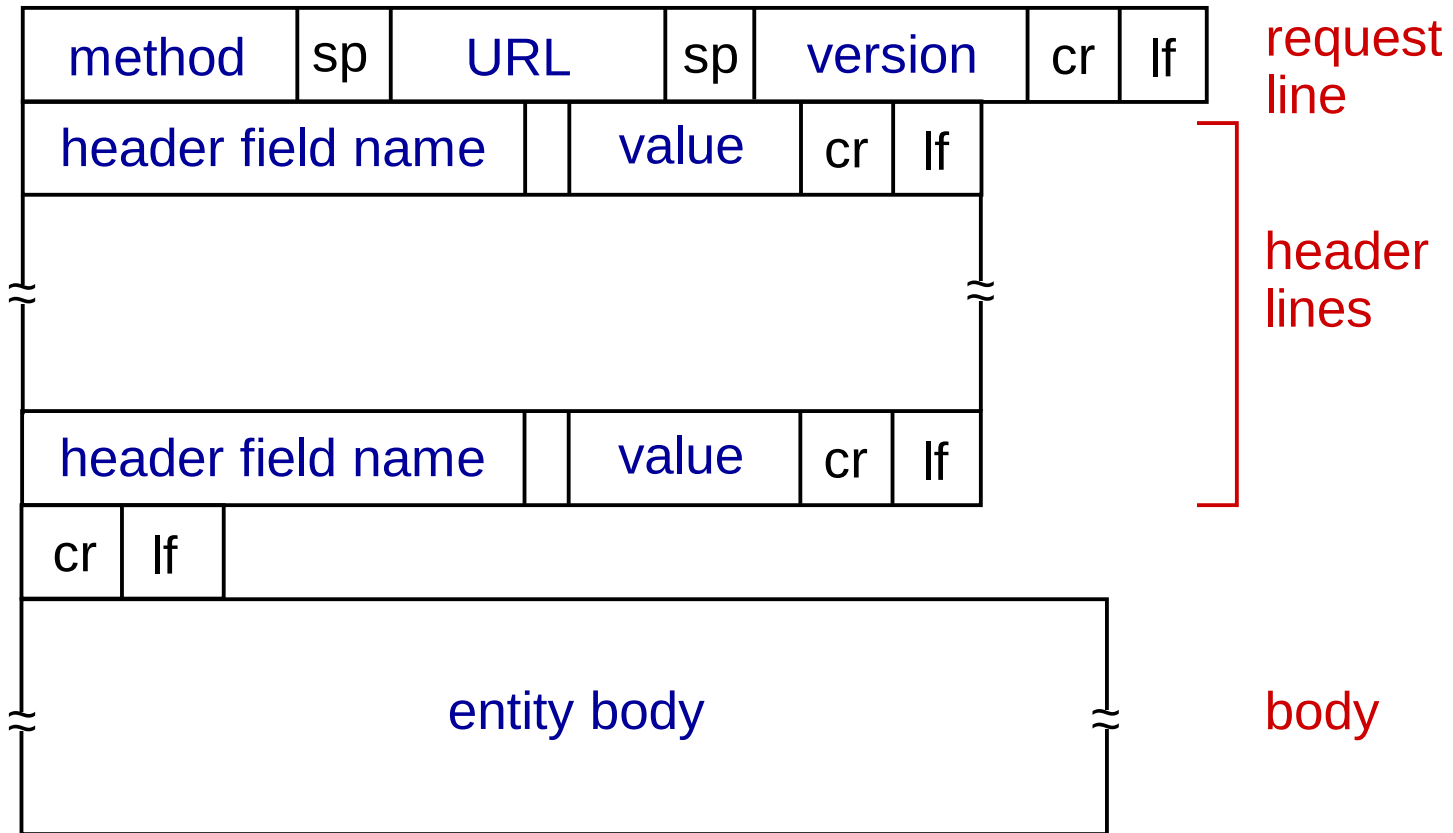
header
lines

carriage return,
line feed at start
of line indicates
end of header lines

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

carriage return character
line-feed character

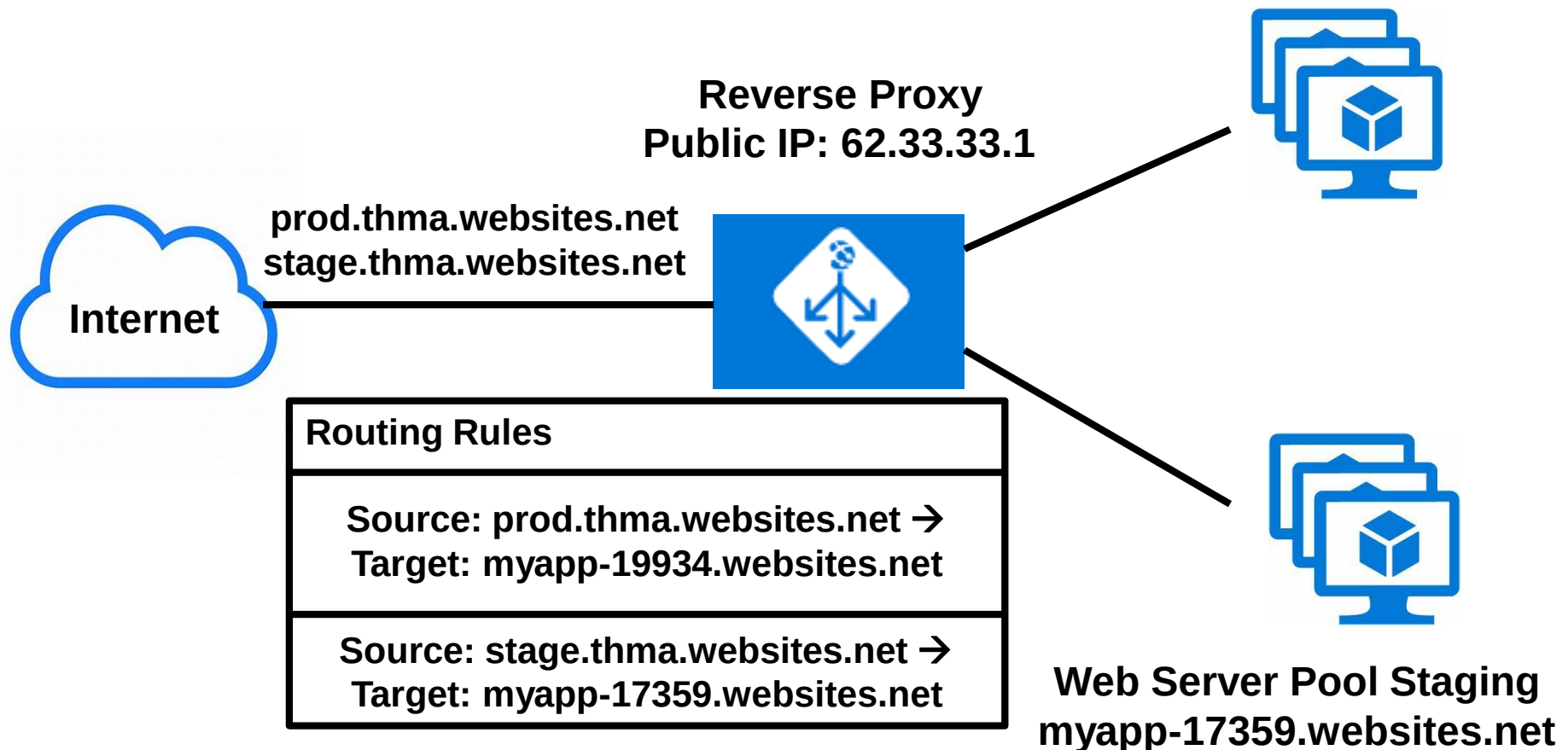
Format der HTTP Request PDU



Deployment Slot Swap für Web Apps

Typische Reverse Proxies : nginx, traefik, web Application GW

Web Server Pool Production
myapp-19934.websites.net





POST-Methode

- Eingaben werden an den Server im Entity Body des HTTP Requests übergeben
- Im IoT-Kontext werden die Daten sehr häufig in JSON übergeben

URL-Methode

- Benutzt die GET-Methode
- Eingaben werden im URL-Feld des HTTP Requests an den Server weitergegeben:

`www.somesite.com/animalsearch?q=monkey+banana&sclient=gws`



HTTP/1.0:

- GET
- POST
- HEAD
 - Fordert den Server auf, nur den Response Header ohne das angeforderte Objekt zu schicken

HTTP/1.1:

- GET, POST, HEAD
- PUT
 - Lädt die Datei im Entity Body in den Pfad, der im URL-Feld angegeben wird
- DELETE
 - Löscht die Datei, die im URL-Feld angegeben wird
- Unterstützt CRUD-Operationen
 - Create, Read, Update, Delete
 - Grundlage des Programmierparadigmas REST für Web Services

Beispiel: Erzeugung eines Buchs in einer Datenbank

```
POST /api/v1/books HTTP/1.1  
HOST: vt.cloudapp.net  
Content-Type: application/json
```

```
{  
  title: 'Behavior-Driven Development',  
  author: 'Viktor Farcic'  
}
```



- gRPC steht für “Google Remote Procedure Call”
- Wird in Google Cloud Platform (GCP) für Web Services eingesetzt
- Für Microservices in Kubernetes
- Aktuell nicht in Azure Microservices verfügbar
- In ASP.NET verwendbar:

<https://learn.microsoft.com/en-us/aspnet/core/tutorials/grpc/grpc-start>

- >verwendet binäres Serialisierungsformat "protocol buffers", effizienter als JSON
- >Protocol Buffers - Format entspricht tflite-Format in Miniprojekt 1, spei



- Publish-Subscribe Protokoll AMQP für Service Bus Endpoint (sb://) auf Ports 80 und 443:

```
GET /messages/events HTTP/1.1
Host: cec.servicebus.cloudapi.de
Upgrade: websocket
Connection: Upgrade
Origin: https://cecfunction.azurewebsites.net
Sec-WebSocket-Protocol: amqp
Sec-WebSocket-Version: 13
```

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Protocol: amqp
```

Format der HTTP Response PDU



Entity tags um IoT: -> Überprüfung auf neue Gerätekonfigurationen und Firmware Updates,
 Z.B "Deployment Manifest" in Reponse -> Etag: ""1.0.3"
 in Request -> if-None-matsch: "1.0.3"

Concurrency-Safe-Updates : Steuerung über Device Twins, Beispiel Windrad:
 {"id": "Windrad 22" ; "State": "running" ; "drehzahl: "12"}
 ->Etag: "abc"

status line
 (protocol
 status code
 status phrase)

header
 lines

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Oct 2019 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Sep 2019 17:00:02 GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-1\r\n
\r\n
data data data data data ...
```

data, e.g.,
 requested
 HTML file

Race Conditen: -> Edge Controller Command { "id": "Windrad"; "State": "emergency-Stop"}
 -> Cloud Controller Command: {"id": "Windrar 22"; "State": "Running"; "drehzahl:"6"} If-match:
 "abc"



Ein Status Code befindet sich in der ersten Zeile der Server->Client Response PDU.

Einige Beispiel-Codes:

200 OK

- request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (Location:)

404 Not Found

- requested document not found on this server

500 Internal Server Error

- processing error on the server



HTTP ist ein zustandsloses (“stateless”) Protokoll

- Der HTTP-Server verwaltet keine Informationen über frühere Client-Anforderungen

Anmerkung

Protokolle, die Zustände verwalten, sind komplex!

- derartige Protokolle werden als “stateful” bezeichnet
- wenn Client oder Server abstürzt, sind die Zustände in der Regel inkonsistent und müssen wieder abgeglichen werden.

HTTP: Authentifizierung

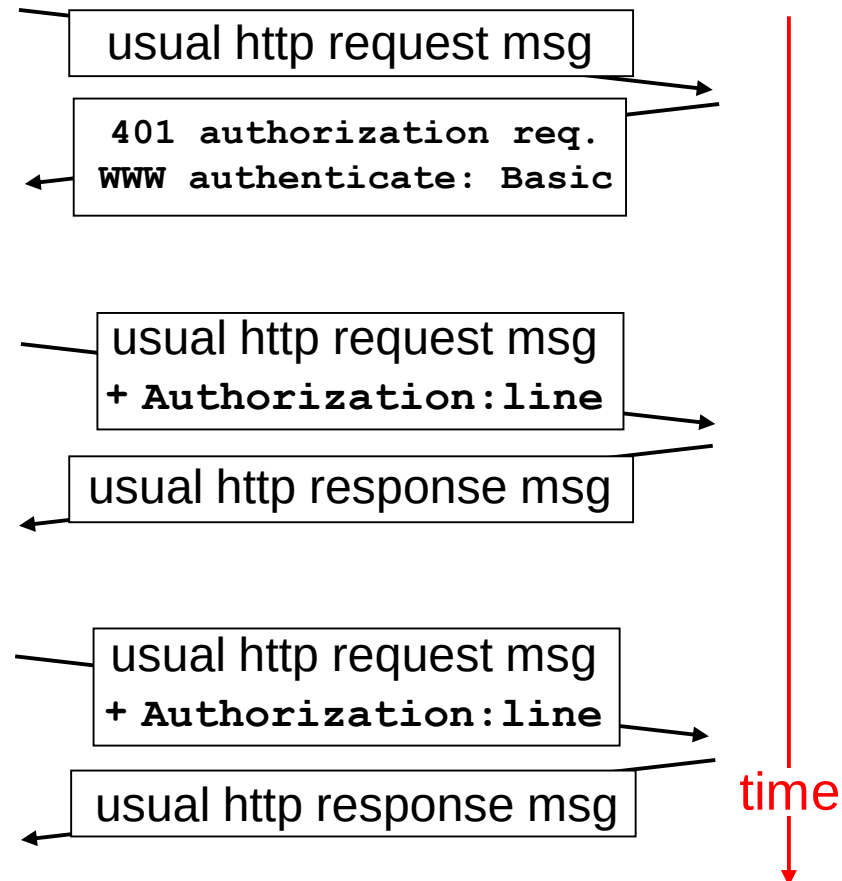
Ziel der Authentifizierung:

Zugangskontrolle zu den Server-Objekten

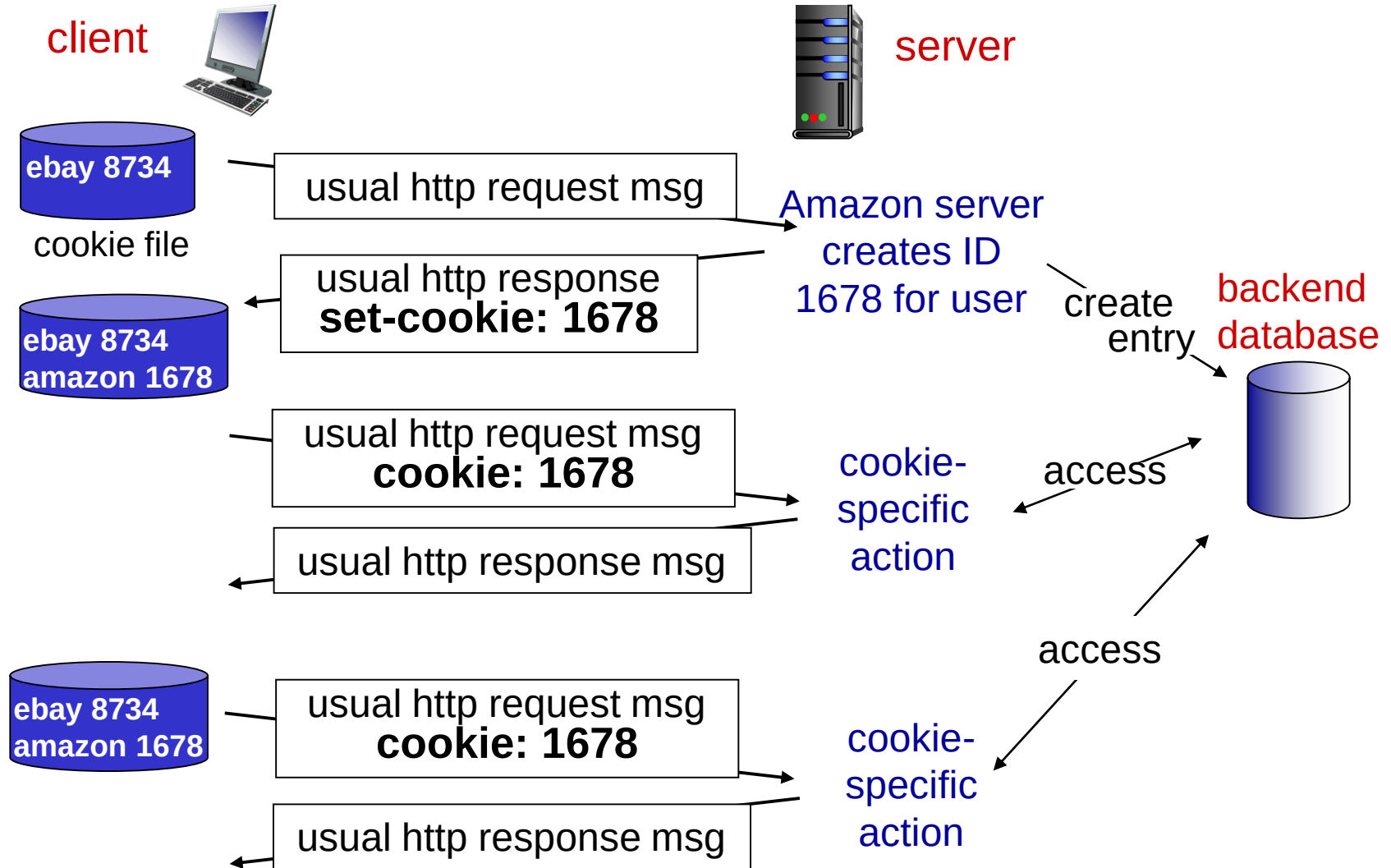
- **HTTP zustandslos** $\Rightarrow$ Client muss die Autorisierung in jedem Request nachweisen
- **Authentifizierung typisch** auf Basis von Name und Password
 - Client sendet Header-Zeile `Authorization:` im Header des Requests
 - wenn keine Autorisierung vorliegt, lehnt der Server den Zugang ab und sendet die Header-Zeile `WWW authenticate:` in der Response

client

server



HTTP: Cookies



HTTP: Bedingtes GET

- **Ziel:** Wenn Client eine aktuelle Version des Objekts im Cache gespeichert hält, soll es nicht gesendet werden
- Client spezifiziert Datum der gespeicherten Kopie im http Request
`If-modified-since: <date>`



HTTP request msg
`If-modified-since: <date>`

object
 not
 modified
 before
`<date>`

HTTP response
HTTP/1.0
304 Not Modified

- Die HTTP Response des Servers beinhaltet kein Objekt, wenn die Kopie im Cache aktuell ist:
HTTP/1.0 304 Not Modified

HTTP request msg
`If-modified-since: <date>`

object
 modified
 after
`<date>`

HTTP response
HTTP/1.0 200 OK
`<data>`



3.1 Hypertext Transfer Protocol (HTTP)

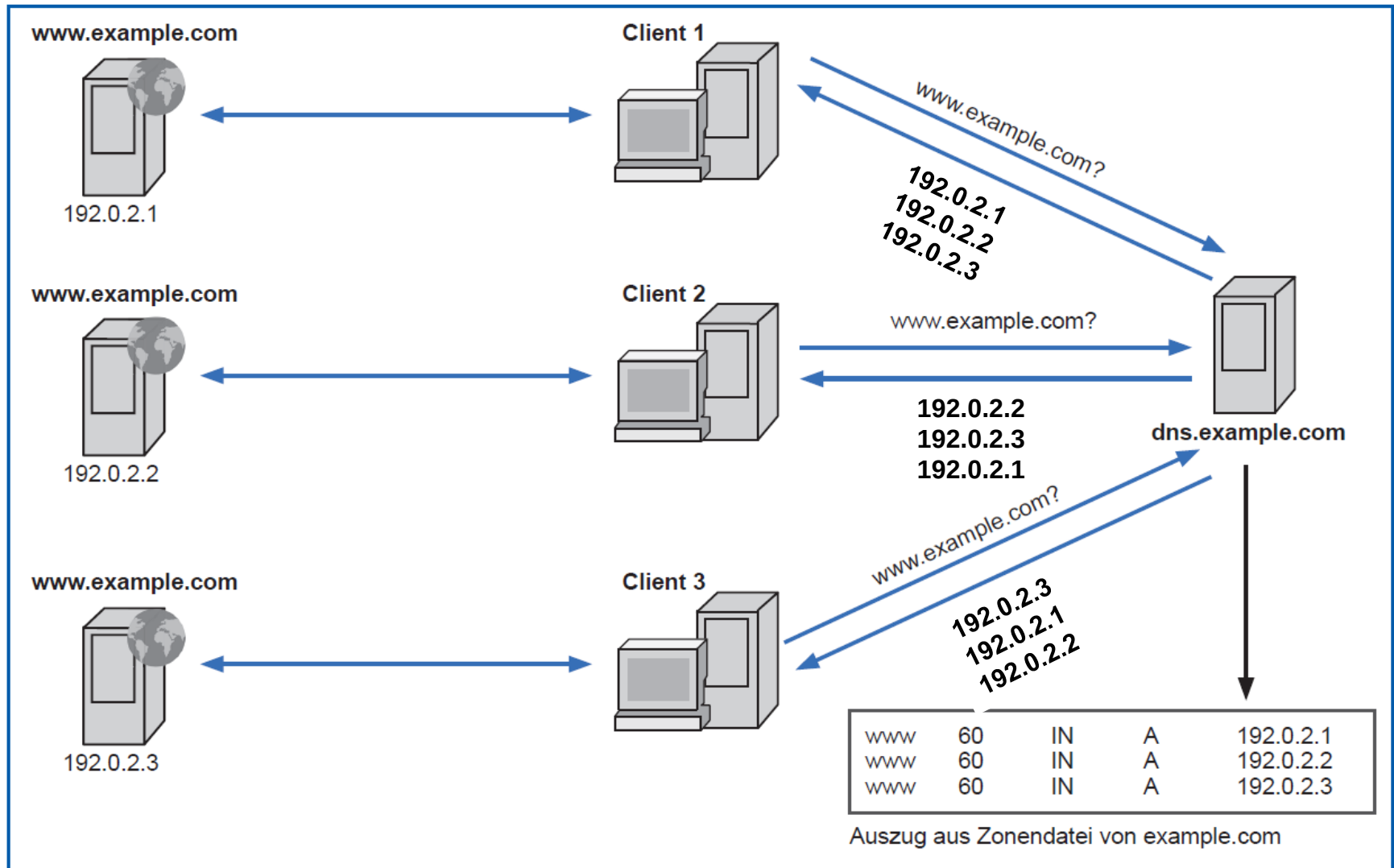
3.2 Domain Name System (DNS)

3.3 Distributed Hash Table (DHT)

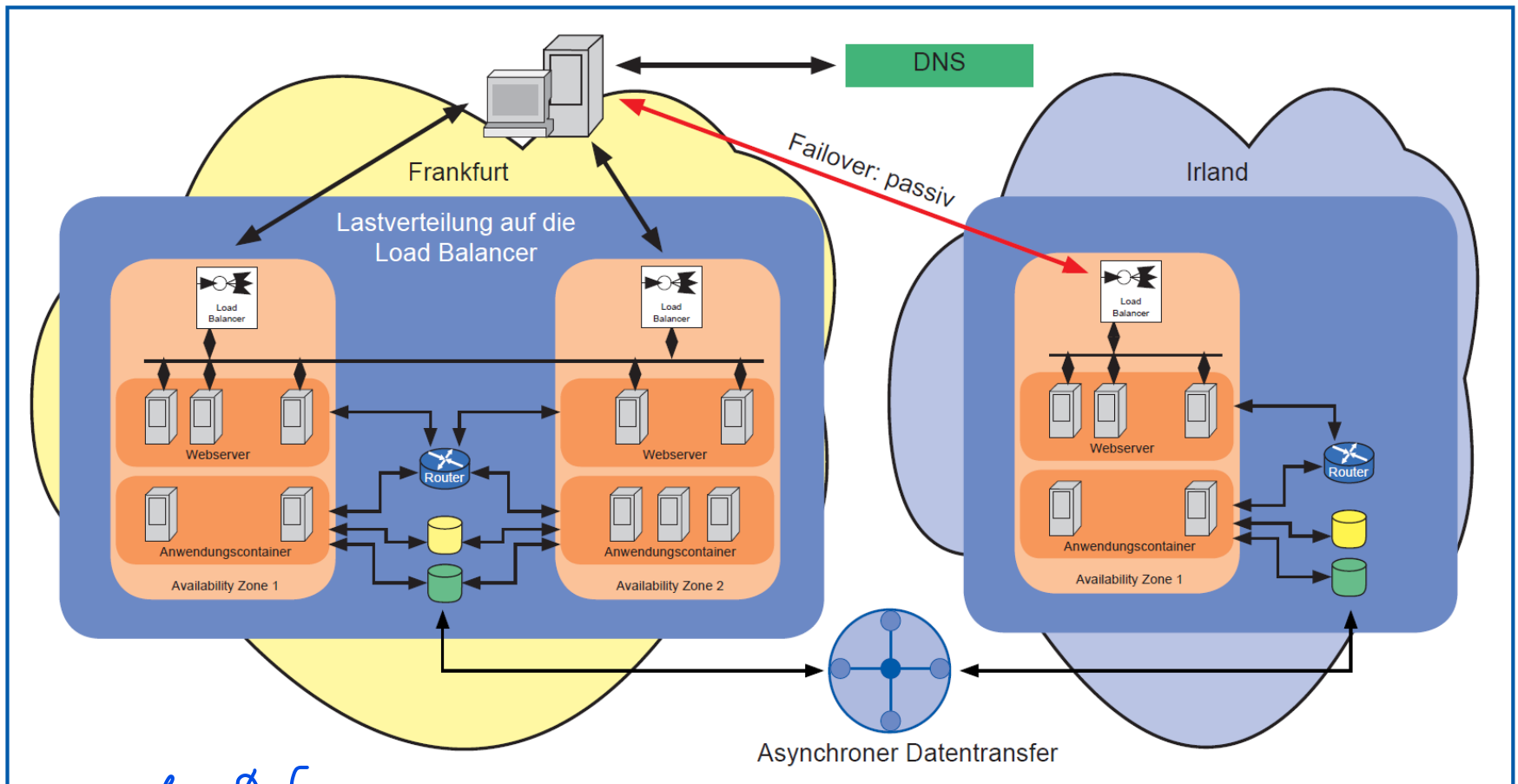
Aufgaben des DNS

- Abbildung Namen -> Adressen
- Host Aliasing, z.B. `www.swt.th-mannheim.de`
- Mail Server Aliasing
- DNS Blacklisting (z.B. <https://www.dnsbl.info> , Datenbank unseriöser IP-Adressen)
- Load Balancing - „round robin“ (rotiert IP-Adressen bei jeder Antwort) oder Geolocation DB für CDN
- Auffinden von Diensten mit Portnummer (z.B. VoIP/SIP)
- Absicherung von Opportunistic TLS

DNS-Anwendung Loadbalancing



DNS-Anwendung Geo Replication mit Failover





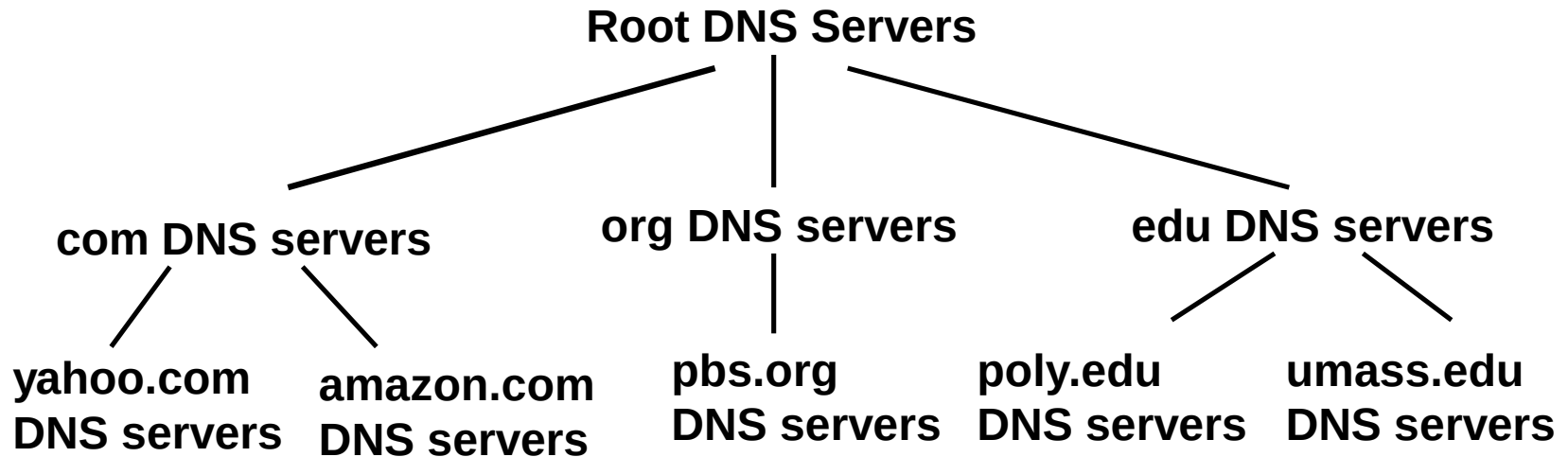
Identifikation von Internet Hosts und Routern

- IP-Adresse (32 bit) - benutzt zur Identifikation eines Interfaces
- “Fully Qualified Domain Name (FQDN)”, z.B.
`n-swt.swt.th-mannheim.de.` – identifiziert einen Knoten im hierarchischen DNS-Baum eindeutig

Frage: Abbildung zwischen IP-Adressen und Namen ?

Domain Name System (DNS)

- *Verteilte Datenbank*, realisiert durch eine Hierarchie vieler *Name Server* ⇒ kein Server besitzt alle Abbildungen von Namen auf IP-Adressen
- Hosts, Router und Name Server kommunizieren, um Namen *aufzulösen* (“name resolution”)
- Kernfunktionalität ist im *Application Layer Protocol* realisiert



Client benötigt IP-Adresse von www.amazon.com; 1. Ansatz:

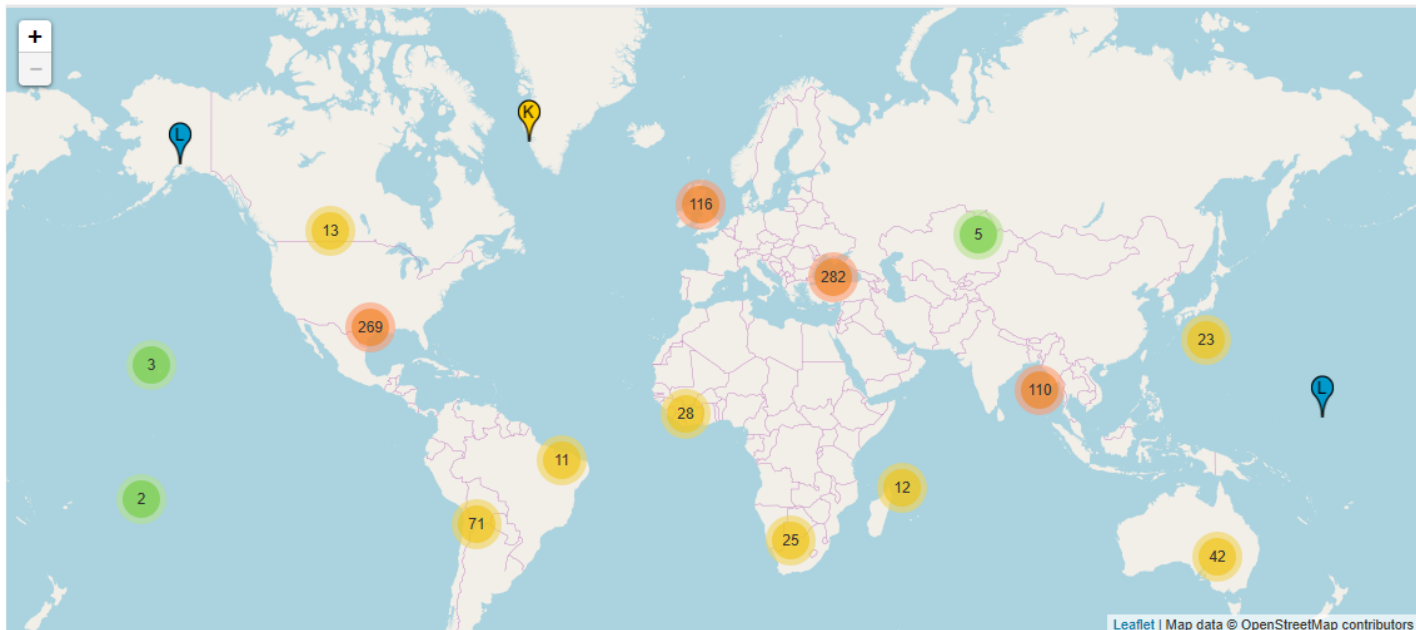
- Client befragt einen Root Server, um den **com** DNS Server zu finden
- Client befragt **com** DNS Server, um die Adresse des **amazon.com** DNS Servers zu bekommen
- Client befragt **amazon.com** DNS Server, um die IP-Adresse von **www.amazon.com** zu erhalten

Vier Rollen von Name Servern

- **Local Name Server („Default Name Server“)**
 - Jeder Netzbetreiber hat einen Local Name Server
 - Jeder DNS Query geht zuerst an den Local Name Server
- **Authoritative Name Server**
 - Verwaltet Namen und IP-Adressen für eine Organisation
 - Liefert verbindliche Abbildung von Namen auf Adressen
- **Top-level Domain (TLD) Name Server**
 - Verantwortlich für com, org, net, edu, etc, und alle top-level Country Domains wie z.B. uk, fr, ca, jp
 - Network Solutions betreibt Server für com TLD
 - Educause für edu TLD
- **Root Name Server**
 - Wird von Name Servern kontaktiert, die Hilfe bei der Namensauflösung benötigen

Root Name Server

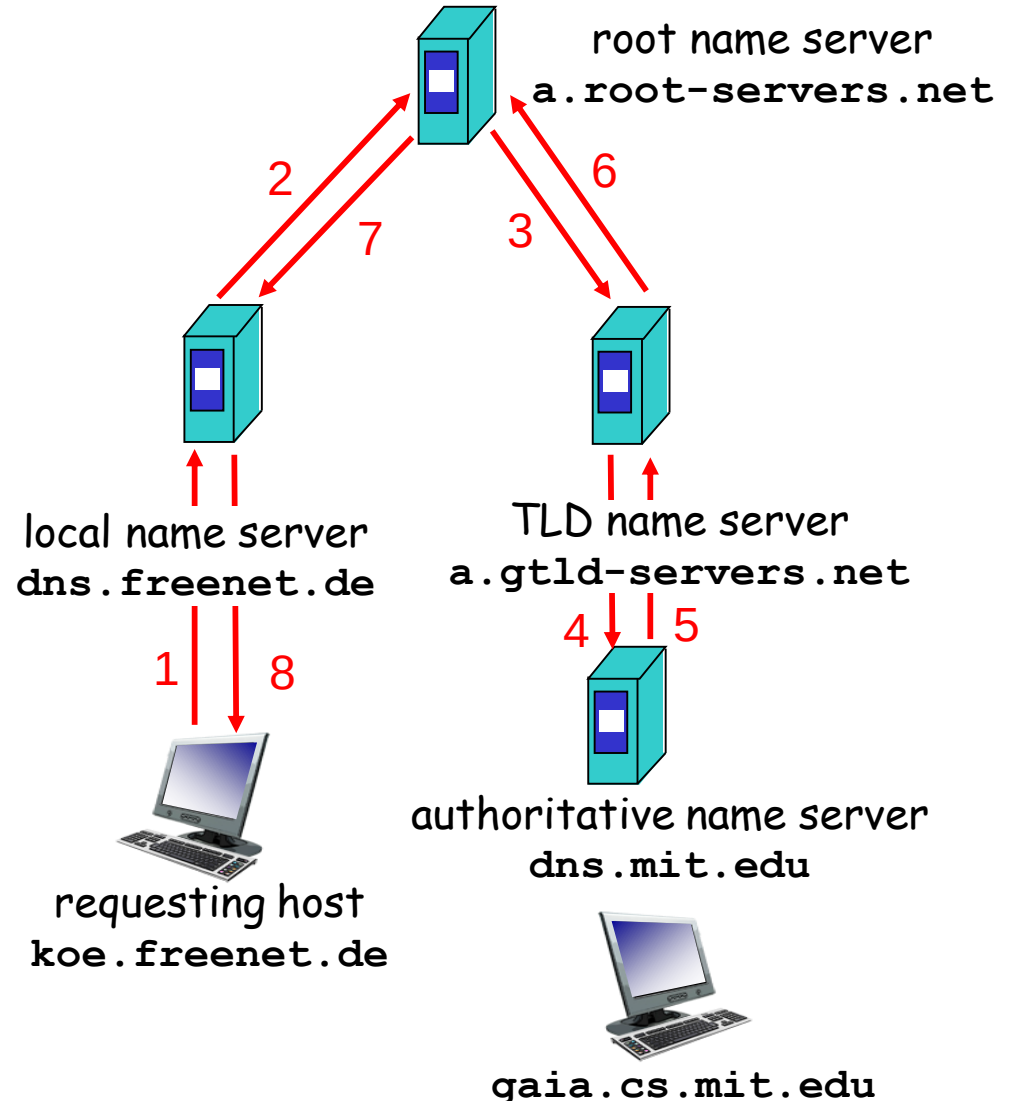
- Ein Root Name Server wird benötigt, wenn für eine Top Level Domain die IP-Adressen der zuständigen Name Server ermittelt werden müssen
- Im Jahr 2024 über 1800 Root Name Server Instanzen an ca. 400 Standorten weltweit (s. www.root-servers.org)
- Root-Nameserver sind nach dem Schema <Buchstabe>.root-servers.net benannt, jeder Buchstabe gehört zu einem anderen Root Server-Betreiber



Beispiel für DNS-Namensauflösung

Host `koe.freenet.de`
sucht IP-Adresse von
`gaia.cs.mit.edu`

1. Kontaktiert lokalen DNS server
2. Lokaler DNS Server kontaktiert Root Name Server
3. Root Name Server kontaktiert TLD Name Server
4. TLD Server kontaktiert Authoritative Name Server



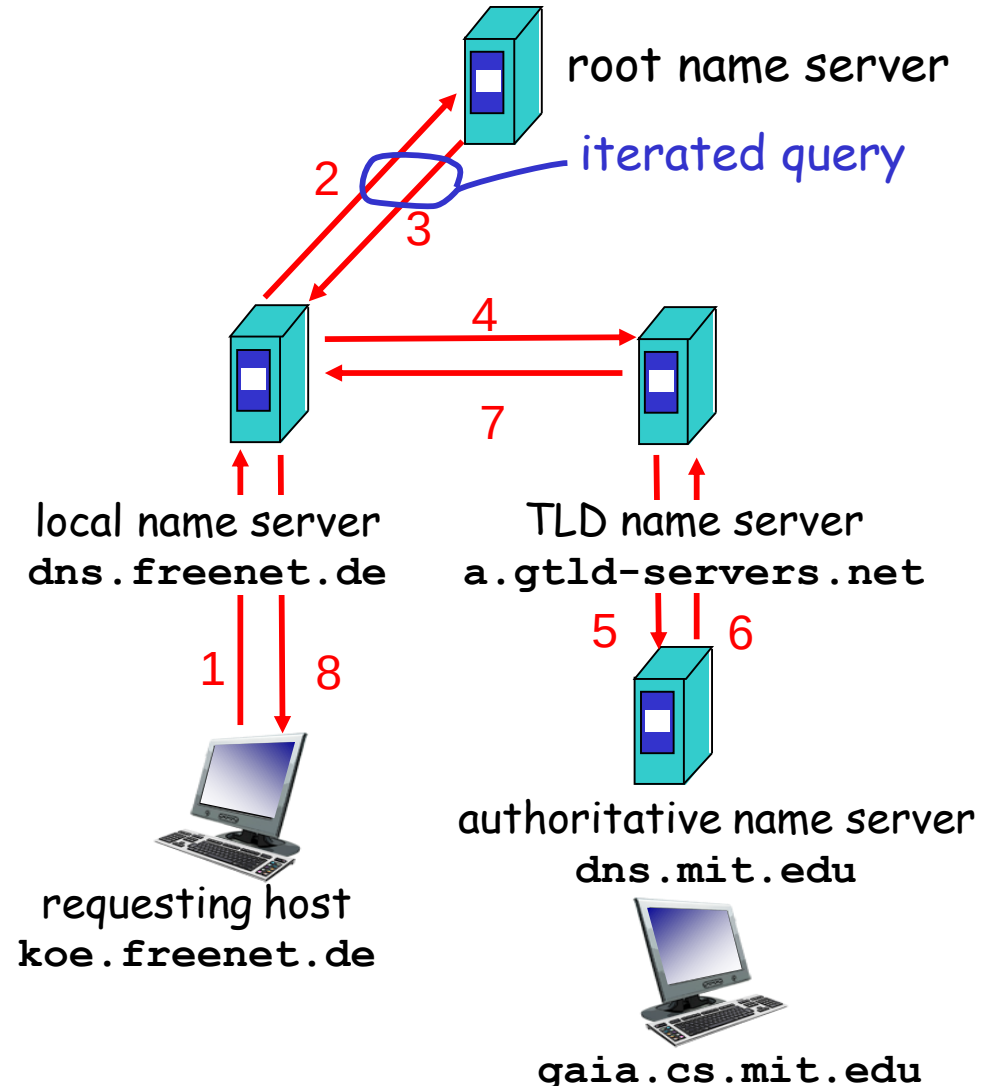
Rekursive vs. iterative Abfrage

Recursive Query:

- Last der Namensauflösung liegt auf dem kontaktierten Name Server
- Starke Last?

Iterated Query:

- Der kontaktierte Server antwortet mit dem Namen des zu kontaktierenden Servers
- “Ich kenne diesen Namen nicht, aber fragen Sie mal bei diesem Server nach”





- Wenn ein Name Server eine Namensauflösung mitgeteilt bekommt, speichert er die Abbildung in seinem lokalen **Cache**
 - Cache-Einträge werden nach Ablauf eines Timeout (TTL – Time To Live) gelöscht
- Es besteht auch die Möglichkeit, dass DNS-Server periodisch ihre gesamte Datenbank austauschen
- DNS Cache unter Windows:
`c:\>ipconfig /displaydns`
`c:\>ipconfig /flushdns`

Resource Records

- Die DNS-Datenbank besteht aus Resource Records (RR):

RR format: (**name**, **value**, **type**, **ttl**)

AAAA → IPv6

- Type=A
 - name** ist Hostname
 - value** ist IP Adresse
- Type=NS
 - name** ist Domäne (z.B. foo.com)
 - value** ist Name (z.B. dns.foo.com) eines Authoritative oder Intermediate Name Server für diese Domäne
- Type=CNAME
 - name** ist ein Alias Name für den “canonical” (den wirklichen) Namen
 - value** ist der canonical Name
- Type=MX
 - value** ist Hostname des Mailservers, der die Maildomäne mit dem Namen **name** bedient
- Tipp: <https://toolbox.googleapps.com/apps/dig>

- RFC9460 (Nov. 2023)
- Beispiel:

```
koe@rpi4:~ $ dig google.com https

; <<>> DiG 9.16.48-Raspbian <<>> google.com https
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 63678
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;google.com.                IN      HTTPS

;; ANSWER SECTION:
google.com.                15372   IN      HTTPS    1 . alpn="h2,h3"

;; Query time: 11 msec
;; SERVER: 192.168.0.84#53(192.168.0.84)
;; WHEN: Fri Jun 14 17:06:09 CEST 2024
;; MSG SIZE  rcvd: 64
```



- Bei DENIC:
 - (th-mannheim.de, nice.rz.th-mannheim.de, NS, 86400)
 - (nice.rz.th-mannheim.de, 141.19.1.75, A, 86400)

- Bei `nice.rz.th-Mannheim.de` (HS Mannheim):
 - (www.th-mannheim.de, tom.rz.th-mannheim.de, CNAME, 86400)
 - (tom.rz.th-mannheim.de, 141.19.1.171, A, 86400)
 - (th-mannheim.de, mailnode.rz.th-mannheim.de, MX, 86400)
 - (mailnode.rz.th-mannheim.de, 141.19.1.96, A, 86400)



Auszug Zonenfile

ipv6.swt.th-mannheim.de



```
; Database file ipv6.swt.th-mannheim.de.zone
;
$TTL      1D
@          IN      SOA  swt24.swt.th-mannheim.de.  admin.swt.th-mannheim.de. (
                                3          ; Serial
                                3H         ; refresh
                                15M        ; retry
                                1W         ; expiry
                                1D )      ; negative TTL

; Zone NS records
@          IN      NS   swt24.swt.th-mannheim.de.

; Zone records
swt24      IN      AAAA  2001:7A0:202:1:208:21FF:FE9A:0001
cisco     IN      AAAA  2001:7A0:202:1:208:21FF:FE9A:4301
sipgate   IN      AAAA  2001:7a0:202:1:2a0:24ff:fe5a:f040
          IN      A     141.19.97.35
www       IN      AAAA  2001:7a0:202:2:2a0:24ff:fe5a:f009

; SIP specific entries
; SRV records
_sip._udp  43200 IN SRV 0 0 5060 sipgate.ipv6.swt.th-mannheim.de.
_sip._tcp  43200 IN SRV 0 4 5060 sipgate.ipv6.swt.th-mannheim.de.
```

DNS Protokoll - PDUs

- **Query** und **Reply** Messages, beide mit demselben Format
- UDP als Transportprotokoll, well-known Port 53

Message Header:

- **Identification**: 16 bit ID für **Query**
- Die **Reply** auf einen **Query** besitzt dieselbe ID
- **Flags** markieren, ob:
 - **Query** oder **Reply**
 - Rekursion gewünscht
 - Rekursion verfügbar
 - **Reply** ist **Authoritative**

← 2 bytes → ← 2 bytes →

identification	flags
# questions	# answer RRs
# authority RRs	# additional RRs
questions (variable # of questions)	
answers (variable # of RRs)	
authority (variable # of RRs)	
additional info (variable # of RRs)	

DNS Protokoll - PDUs

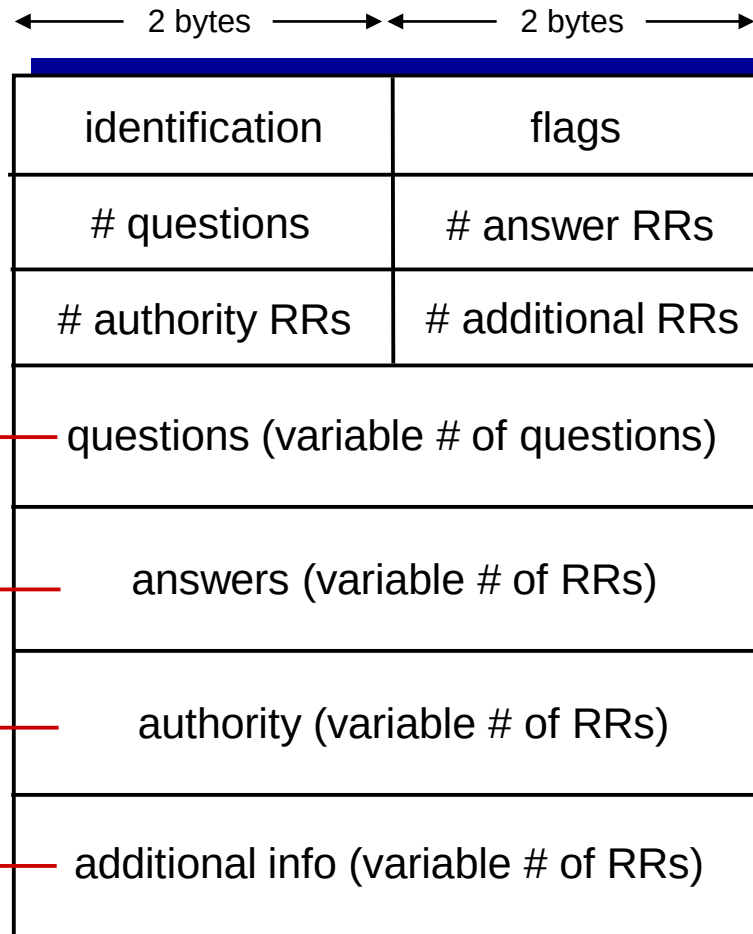


**name, type fields
for a query**

**RRs in response
to query**

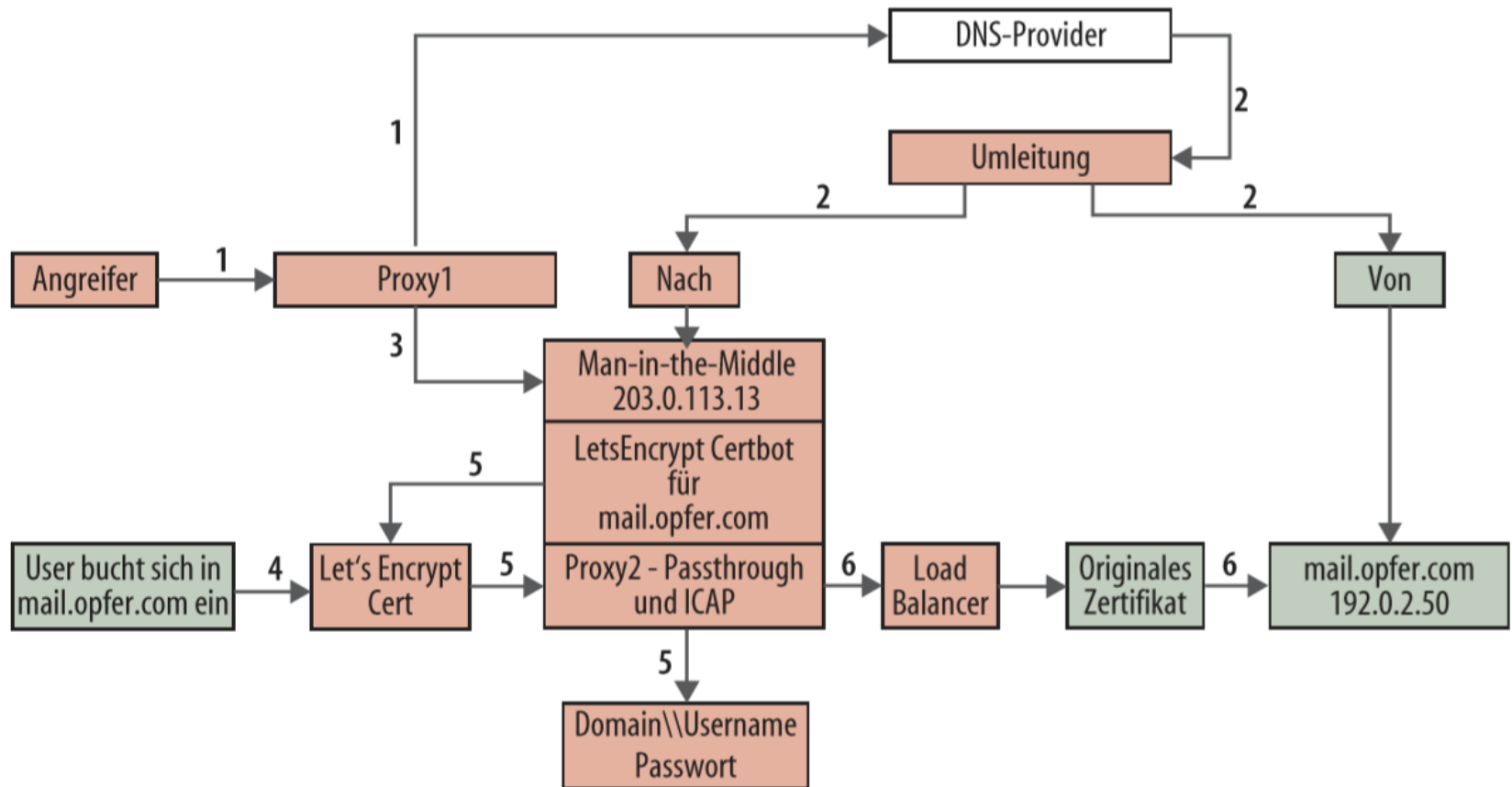
**records for
authoritative servers**

**additional “helpful”
info that may be used**



- **Signierung von Resource Records**
 - RFC 4033 bis 4035 - DNS Security (DNSSEC), March 2005
- **Transportverschlüsselung**
 - RFC 7858 - Specification for DNS over Transport Layer Security (TLS), May 2016
 - RFC 8484 - DNS Queries over HTTPS (DoH), October 2018
 - RFC 9250 - DNS over Dedicated QUIC Connections, May 2022

DNSpionage



Schnellstart: Erstellen einer Azure DNS-Zone und eines entsprechenden Eintrags mithilfe von Azure PowerShell:

<https://learn.microsoft.com/en-us/azure/dns/dns-getstarted-powershell>



3.1 Hypertext Transfer Protocol (HTTP)

3.2 Domain Name System (DNS)

3.3 Distributed Hash Table (DHT)



- Database (NoSQL, Big Data)
 - Apache Cassandra, <http://cassandra.apache.org/>
 - Amazon Dynamo DB (AWS)
 - Couchbase, <https://www.couchbase.com/>
- Naming
 - Ersetzt DNS im Tor Netzwerk/Deep Web (.onion –TLD, z.B. <https://3g2upl4pq6kufc4m.onion/> – DuckDuckGo Search Engine), Tor Hidden Service Directory (HSDir)
- Messenger
 - jami.net

Distributed Hash Table (DHT)



- DHT = distributed P2P database
- Database has **(key, value)** pairs:
 - key: social security number; value: human name
 - key: content name; value: IP address
- Peers **query** DB with key
 - DB returns values that match the key
- Peers can also **insert, update and delete** (key, value) pairs



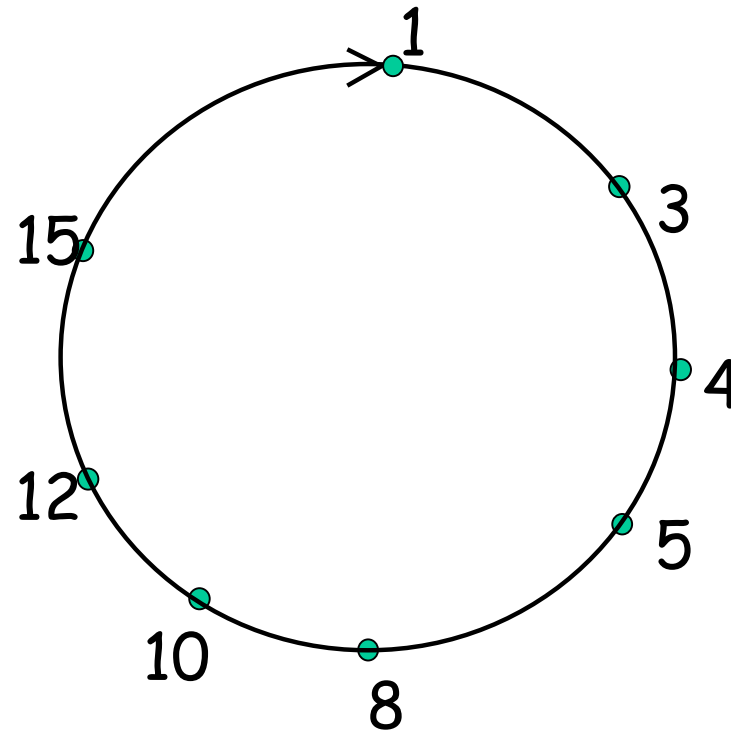
- Assign integer identifier to each peer called **Nodeld** in range $[0, 2^n - 1]$.
 - Each identifier can be represented by n bits
- Require each **key** to be an integer in **same range**.
- To get integer keys, hash original key.
 - Eg, key = $h(\text{"Led Zeppelin IV"})$
 - This is why they call it a distributed "hash" table
- Typisch $n=256$, also 2^{256} mögliche Keys

How to assign keys to peers?



- **Central issue:**
 - Assigning (key, value) pairs to peers.
- **Rule:** assign key to the peer that has the **closest ID**.
- **Convention in lecture:** closest is the **immediate successor** of the key.
- **Ex:** $n=4$ bit; peer NodeID: 1,3,4,5,8,10,12,14;
 - key = 13, then successor peer = 14
 - key = 15, then successor peer = 1
 - key = 10, then successor peer = 10

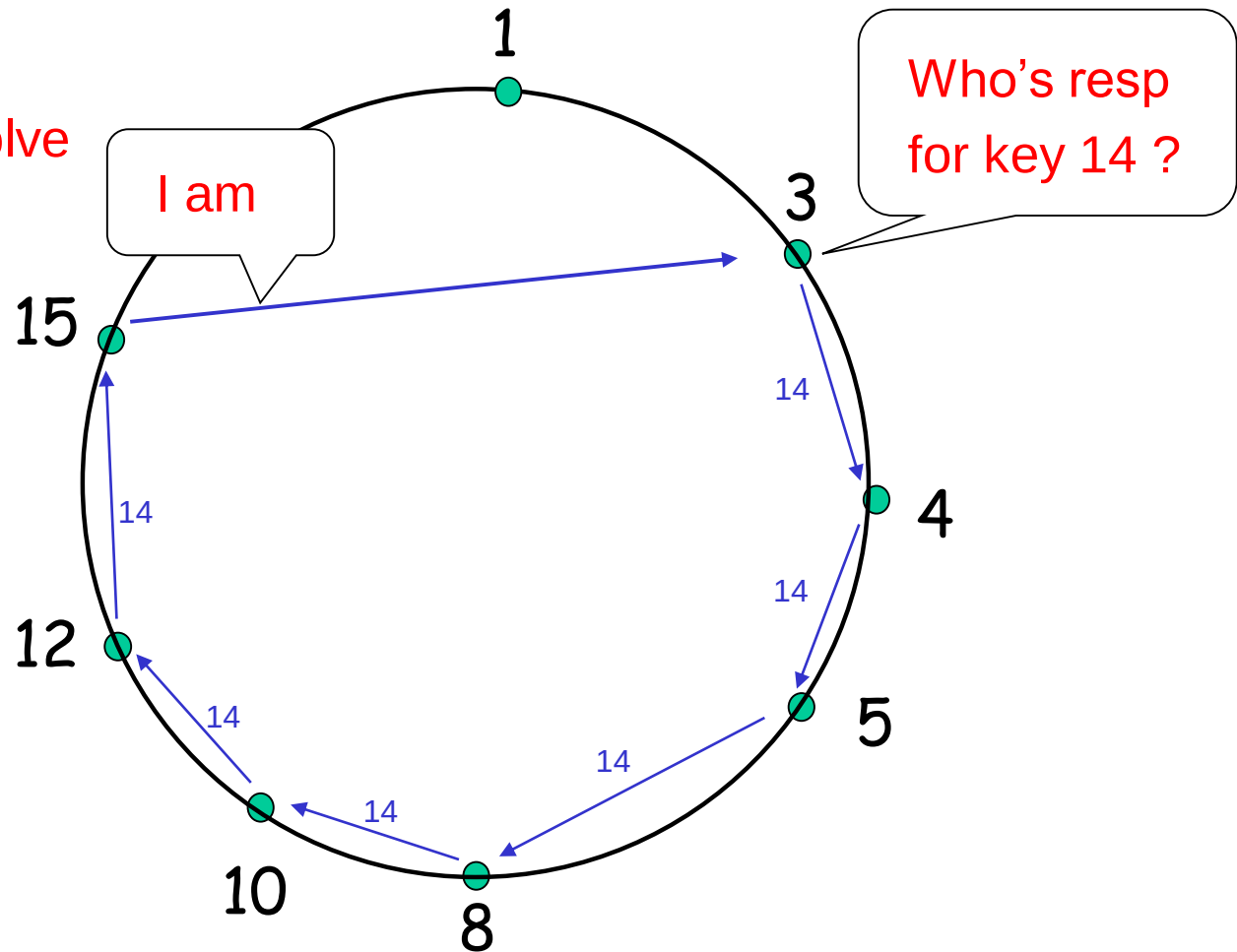
Circular DHT (1/2)



- Each peer only aware of immediate successor and predecessor.
- “Overlay network”

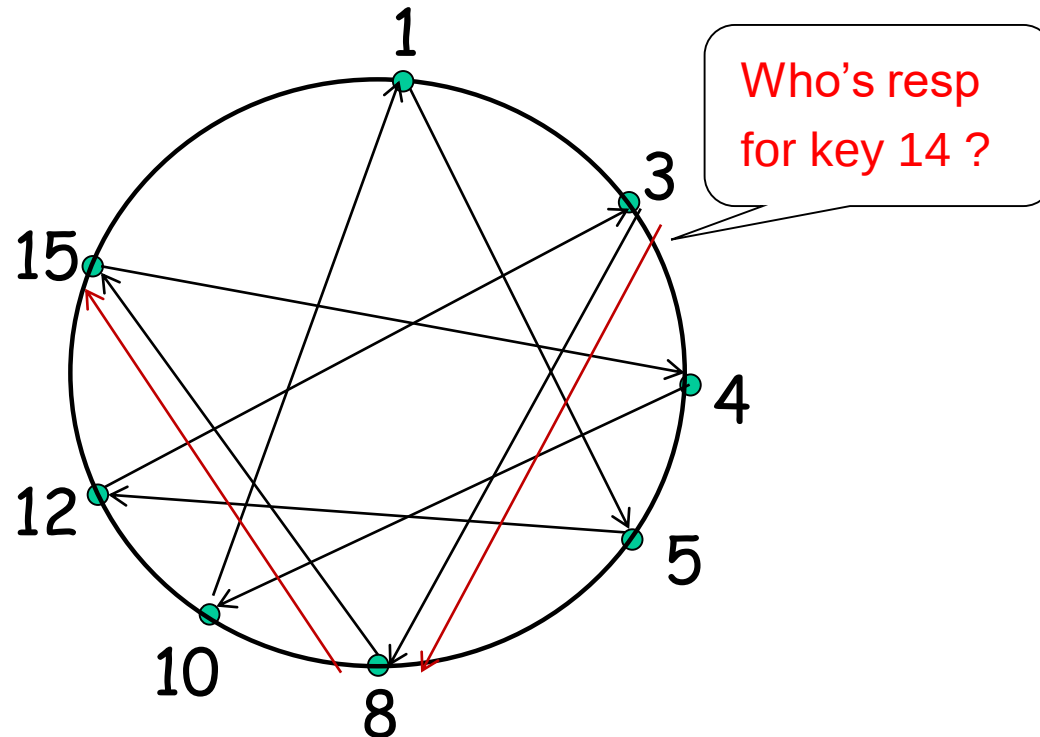
Circular DHT (2/2)

$O(N)$ messages
on average to resolve
query, when there
are N peers



Define closest
as closest
successor

Circular DHT with Shortcuts

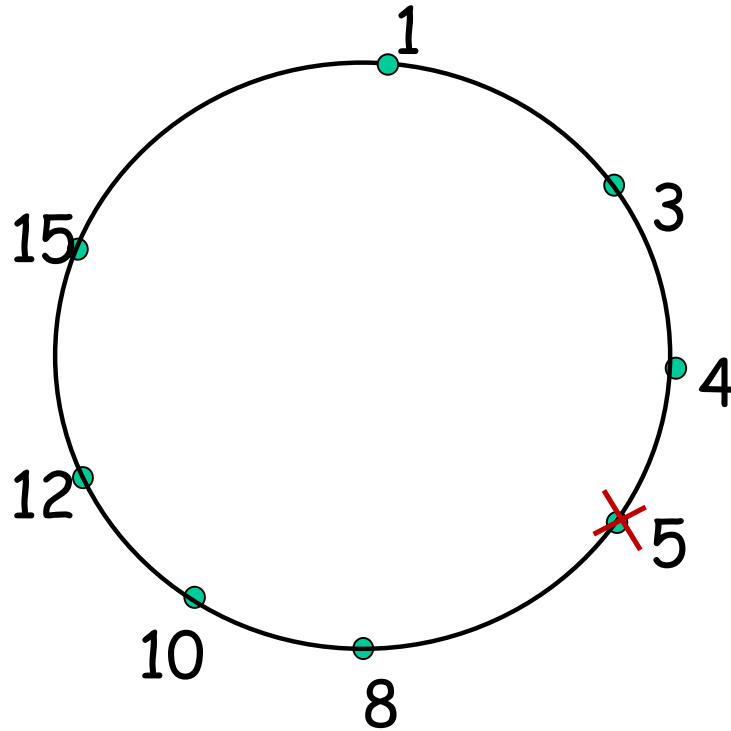


- Each peer keeps track of IP addresses of predecessor, successor, shortcuts.
- Reduced from 6 to 2 messages.
- Possible to design shortcuts so $O(\log N)$ neighbors, $O(\log N)$ messages in query

Zahlenbeispiel

- Anzahl Knoten sei $N = 10^6$
- Dann ist die Anzahl Shortcuts $\text{Id } 10^6 \approx 20$
- Mittlere Anzahl Hops bei Suche $\frac{1}{2} \text{Id } 10^6 \approx 10$

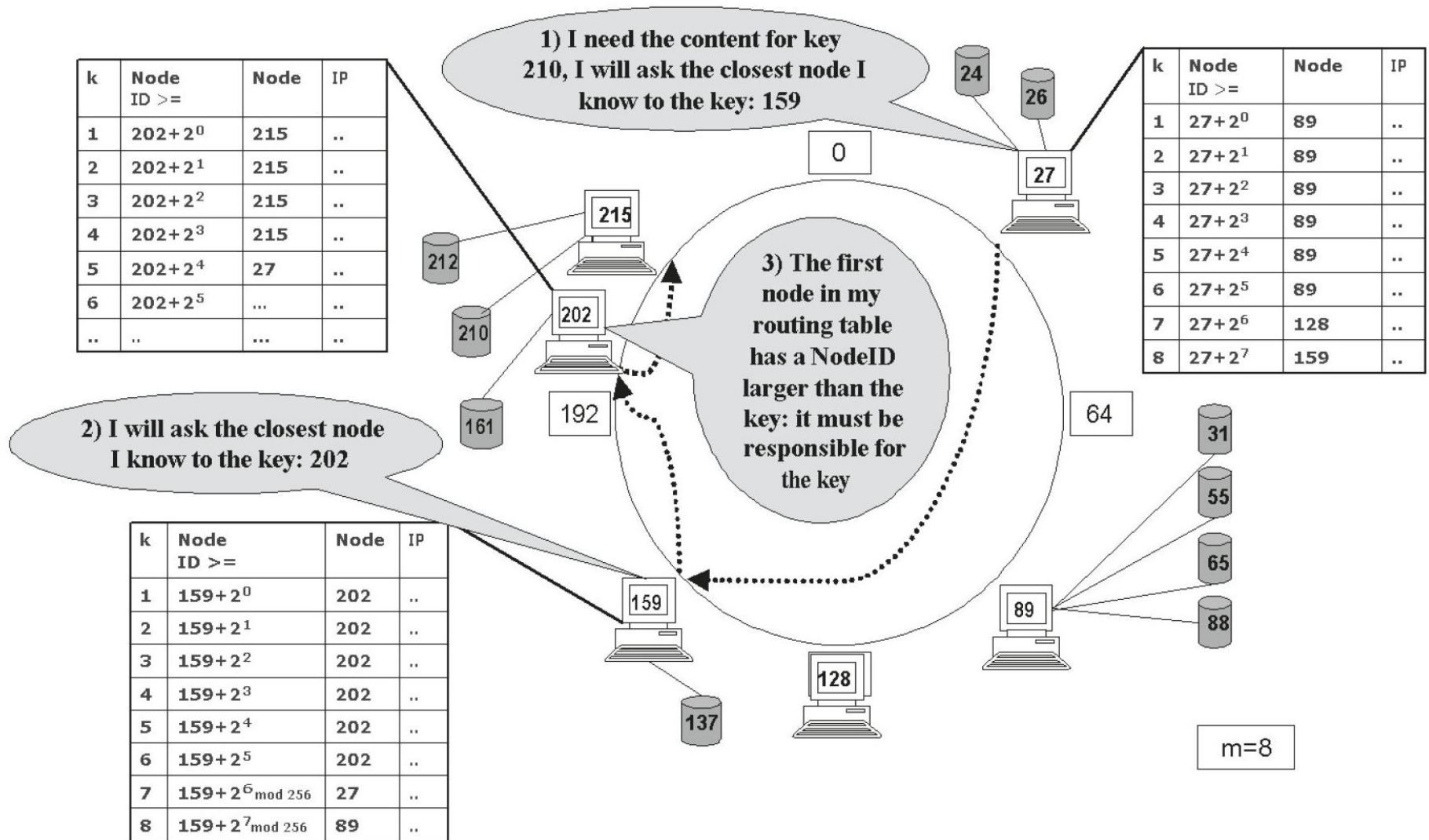
Peer Churn



- To handle peer churn, require each peer to know the IP address of at least its two successors.
- Each peer periodically pings its two successors to see if they are still alive.

- Peer 5 abruptly leaves
- Peer 4 detects; makes 8 its immediate successor; asks 8 who its immediate successor is; makes 8's immediate successor its second successor.
- What if peer 13 wants to join?

Chord: P2P Lookup Service



NoSQL Cassandra DB Applications on Azure

